Applications

Cloud-native application development is an approach to designing, building, deploying, and maintaining services that leverage the resiliency and efficiency of cloud computing technologies to deliver real-time, cost-effective, and secure applications to the end users. In this first module, we introduce cloud-native development and define some of the intrinsic characteristics of this modern paradigm to application development.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Examine cloud-native applications.

02 State the pillars of cloud-native.

03 Discuss candidate applications for cloud-native

Key Terms and Concepts

Key Term or Concept	Definition
Application Programming Interface	An application programming interface is a way for two or more computer programs to communicate with each other. It is a type of software interface, offering a service to other pieces of software.

Key Term or Concept	Definition
Legacy application	A legacy system is any outdated computing system, hardware or software that is still in use. Legacy systems include computer hardware, software applications, file formats and programming languages. However, not all legacy systems are obsolete technologies. Most legacy systems work even if they are outdated, and enterprises will often continue using legacy systems that are critical to their daily functions and business needs.
Single-page application (SPA)	A single-page application is a web application or website that interacts with the user by dynamically rewriting the current web page with new data from the web server, instead of the default method of a web browser loading entire new pages. The goal is faster transitions that make the website feel more like a native app.
Vertical scaling	Vertical scaling refers to increasing a system's capabilities by adding resources to it: processing power, memory, storage, or network speed.

In this module, we introduced cloud-native computing. We provided a definition along with the key capabilities that drive a cloud-native application. We looked at the types of applications that might justify this investment and effort.

In the next modules, with the introduction behind, we now dive into a much more detailed look at cloud native.

References

The references listed here, coupled with the course developer's experience, have been used in the development of this module:

n.a. (2021) 3-tier application architecture. TechTarget. Retrieved from **https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application (https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application)**

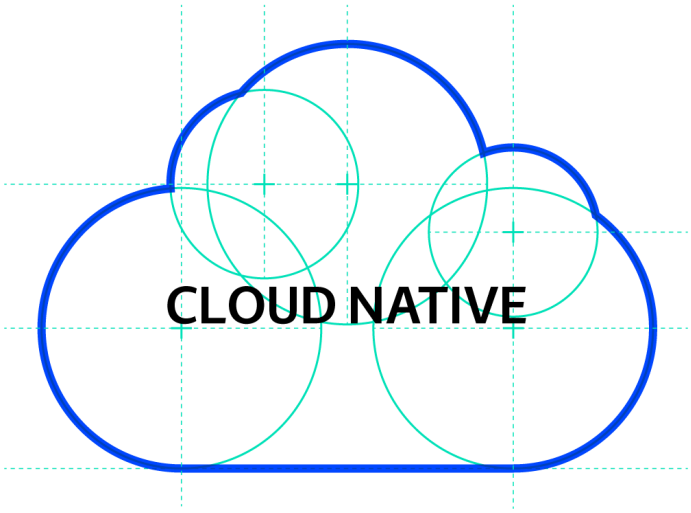
Ki Sun Hwang, Priscilla (2022). Phoenix 'nightmare' still haunting public servants, more than 6 years on. CBC News. Retrieved from **https://www.cbc.ca/news/canada/ottawa/phoenix-pay-system-issues-impact-public-servants-six-years-later-1.6457840 (https://www.cbc.ca/news/canada/ottawa/phoenix-pay-system-issues-impact-public-servants-six-years-later-1.6457840)**

Team Copado (2022). Pets vs. Cattle: More Than an Analogy for Modern Infrastructures. Copado. Retrieved from **https://www.copado.com/devops-hub/blog/pets-vs-cattle-more-than-an-analogy-for-modern-infrastructures (https://www.copado.com/devops-hub/blog/pets-vs-cattle-more-than-an-analogy-for-modern-infrastructures)**

Candidate apps for cloud native

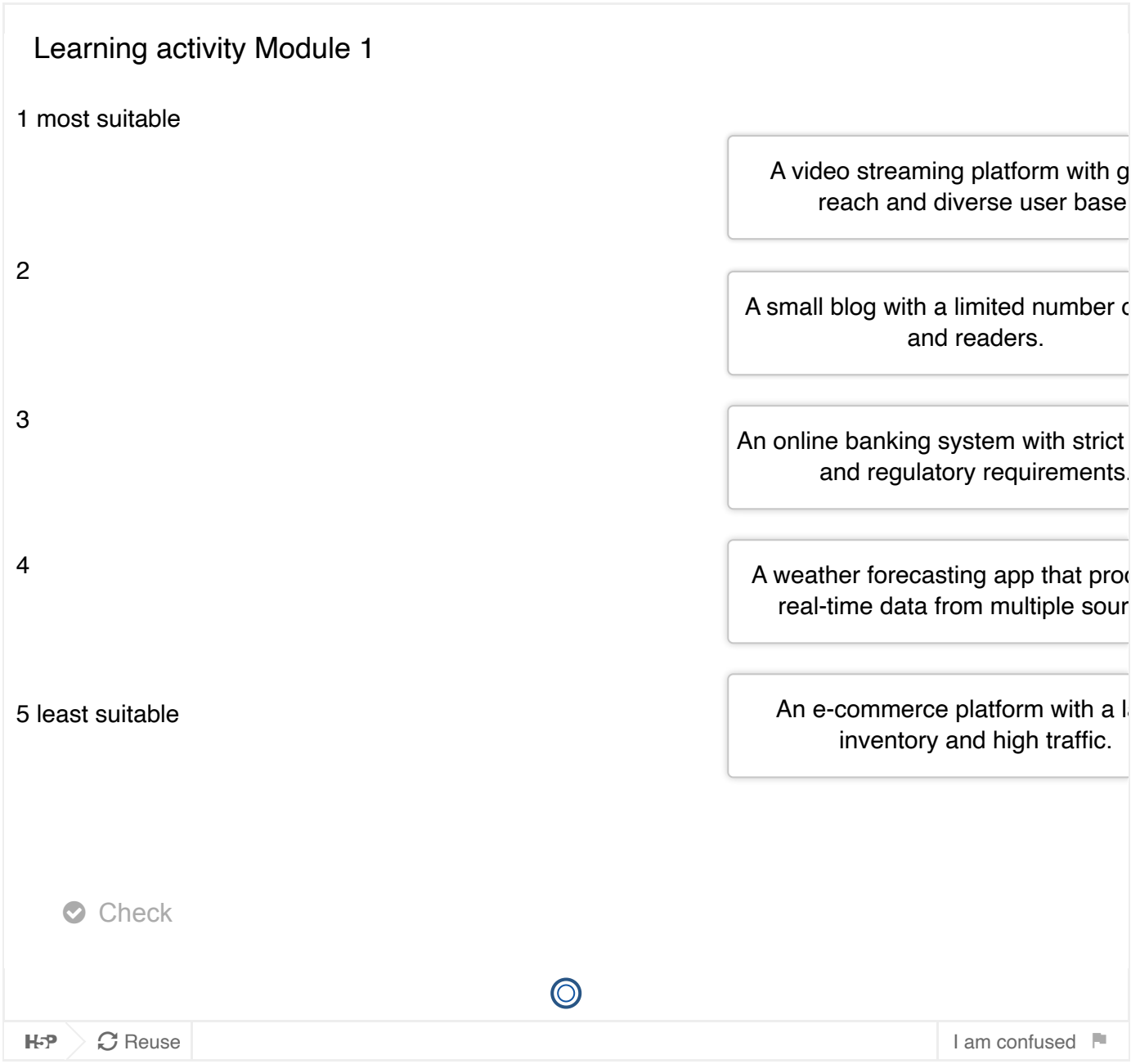
What type of application might be a candidate for cloud native? Here are some considerations that could indicate that an app is a good fit for a cloud native approach:

- Strategic enterprise systems that need to constantly evolve business capabilities/features.
- An application that requires a high release velocity - with high confidence.
- A system with where individual features must release without a full redeployment of the entire system.
- An application developed by teams with expertise in different technology stacks.
- An application with components that must scale independently.



Learning activity

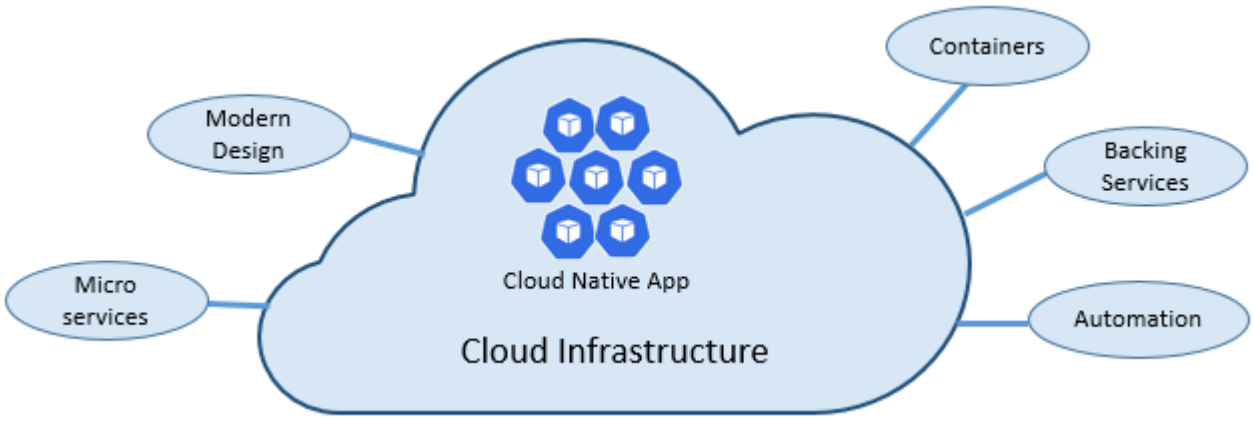
Review the following scenarios and individually rank them from most suitable (1) to least suitable (5) for cloud-native development.



Explore different perspectives on the suitability of each application for cloud-native development. Consider the key factors when evaluating the suitability of an application for cloud-native development, such as scalability, data processing requirements, or the need for real-time responsiveness.

Pillars of cloud native

As we mentioned, cloud-native is based on speed and agility. These attributes derive from many factors, first of which is the cloud infrastructure. In this section, we will introduce the foundational pillars that provide the bedrock for cloud-native systems.



The cloud

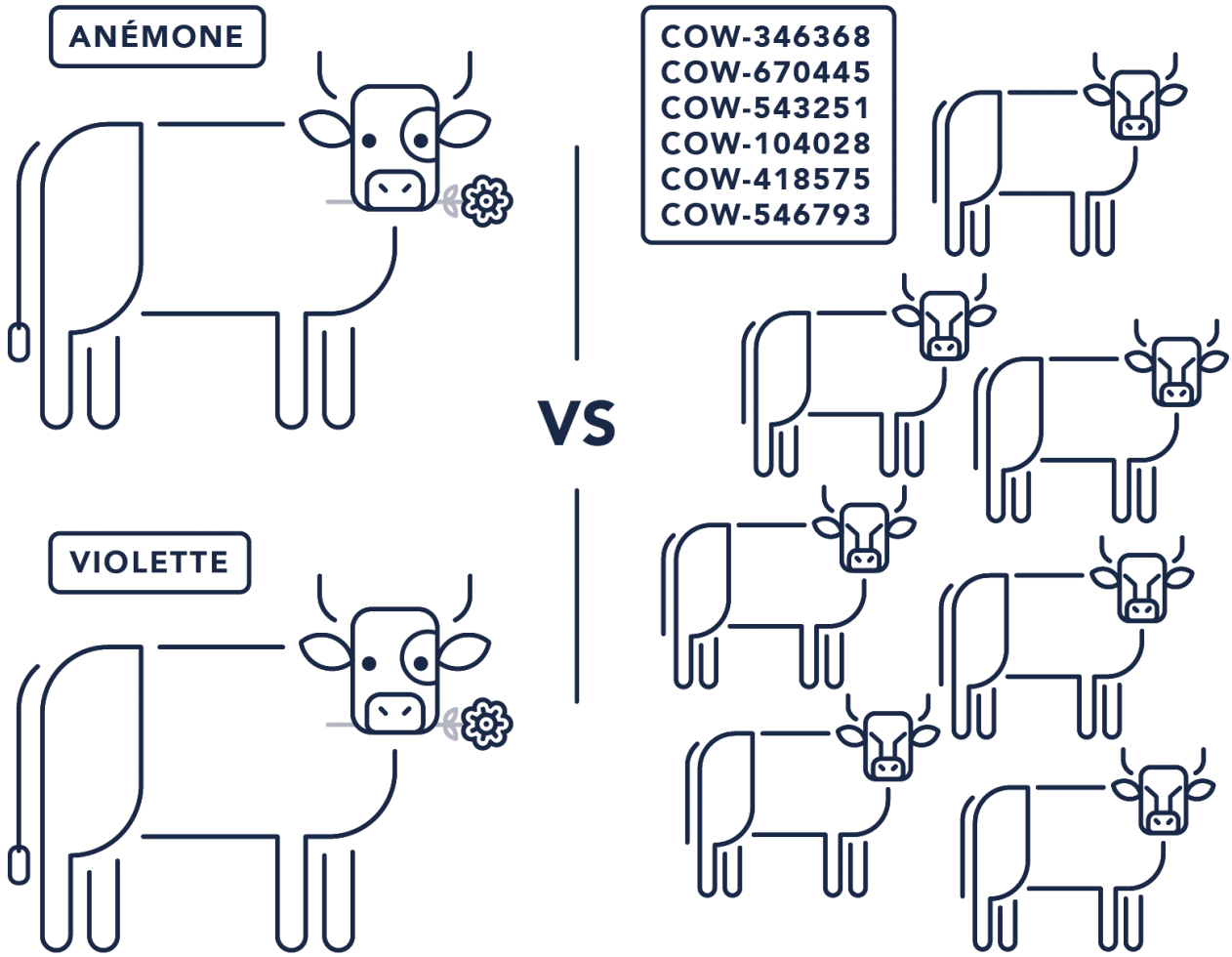
Cloud-native systems take full advantage of the cloud service model. Designed to thrive in a dynamic, virtualized cloud environment, these systems make extensive use of Platform as a Service (PaaS) computing infrastructure and managed services.

They treat the underlying infrastructure as disposable; provisioned in minutes and resized, scaled, or destroyed on demand via automation.



Pets vs. Cattle

A popular approach to explain how traditional infrastructure differs from cloud computing is the “Pets vs. Cattle” analogy.



The basis of the analogy is to consider that in a traditional data center, servers are treated as Pets: they are attributed a physical machine, given a meaningful name, and cared for. When additional resources are needed, they are provided to the same machine already in use, i.e., scaling up. If the server becomes sick, you nurse it back to health. Should the server become unavailable, everyone notices.

The cattle service model is different. You provision each instance as a virtual machine or container. They're identical and assigned a system identifier such as Service-01, Service-02, and so on. Additional resources are provided by creating more of them, i.e., scaling out. When one becomes unavailable, nobody notices.

The cattle model embraces immutable infrastructure. Servers aren't repaired or modified. If one fails or requires updating, it's destroyed and a new one is provisioned – all done via automation.

Cloud-native systems embrace the Cattle service model. They continue to run as the infrastructure scales in or out with no regard to the machines upon which they're running.

The analogy has been used in several articles describing the benefits of modern cloud infrastructures. An example of such an article, that builds on the analogy to tackle “Infrastructure as code” can be found here:

PETS VS. CATTLE: MORE THAN AN ANALOGY FOR MODERN INFRASTRUCTURES (HTTPS://WWW.COPADO.COM/DEVOPS-HUB/BLOG/PETS-VS-CATTLE-MORE-THAN-AN-ANALOGY-FOR-MODERN-INFRASTRUCTURES)

Modern design

A widely accepted methodology for constructing cloud-based applications is the Twelve-Factor Application. It describes a set of principles and practices that developers follow to construct applications optimized for modern cloud environments. Special attention is given to portability across environments and declarative automation.

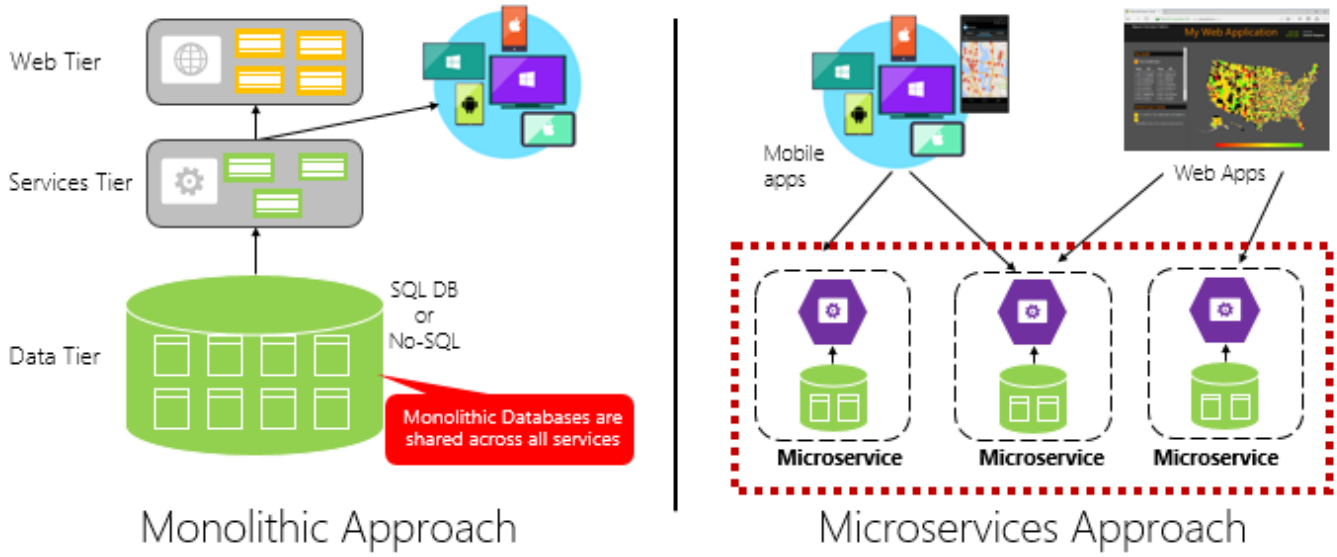
While applicable to any web-based application, many practitioners consider Twelve-Factor a solid foundation for building cloud-native apps. Systems built upon these principles can deploy and scale rapidly and add features to react quickly to market changes.

A detailed exploration of the twelve factors will be done in the next module.

Single Codebase	**Explicit Dependencies**	**Externally Managed Configuration**	**Backing Services**
Build, release, run	**Processes**	**Port Binding**	**Concurrency**
Disposability	**Dev/Prod Parity**	**Log as event streams**	**Admin Processes**

Microservices Architecture

Cloud-native systems embrace microservices, a popular architectural style for constructing modern applications.



Note how the monolith is composed of a layered architecture, which executes in a single process. It typically consumes a relational database.

The microservice approach, however, segregates functionality into independent services, each with its own logic, state, and data. Each microservice hosts its own datastore.

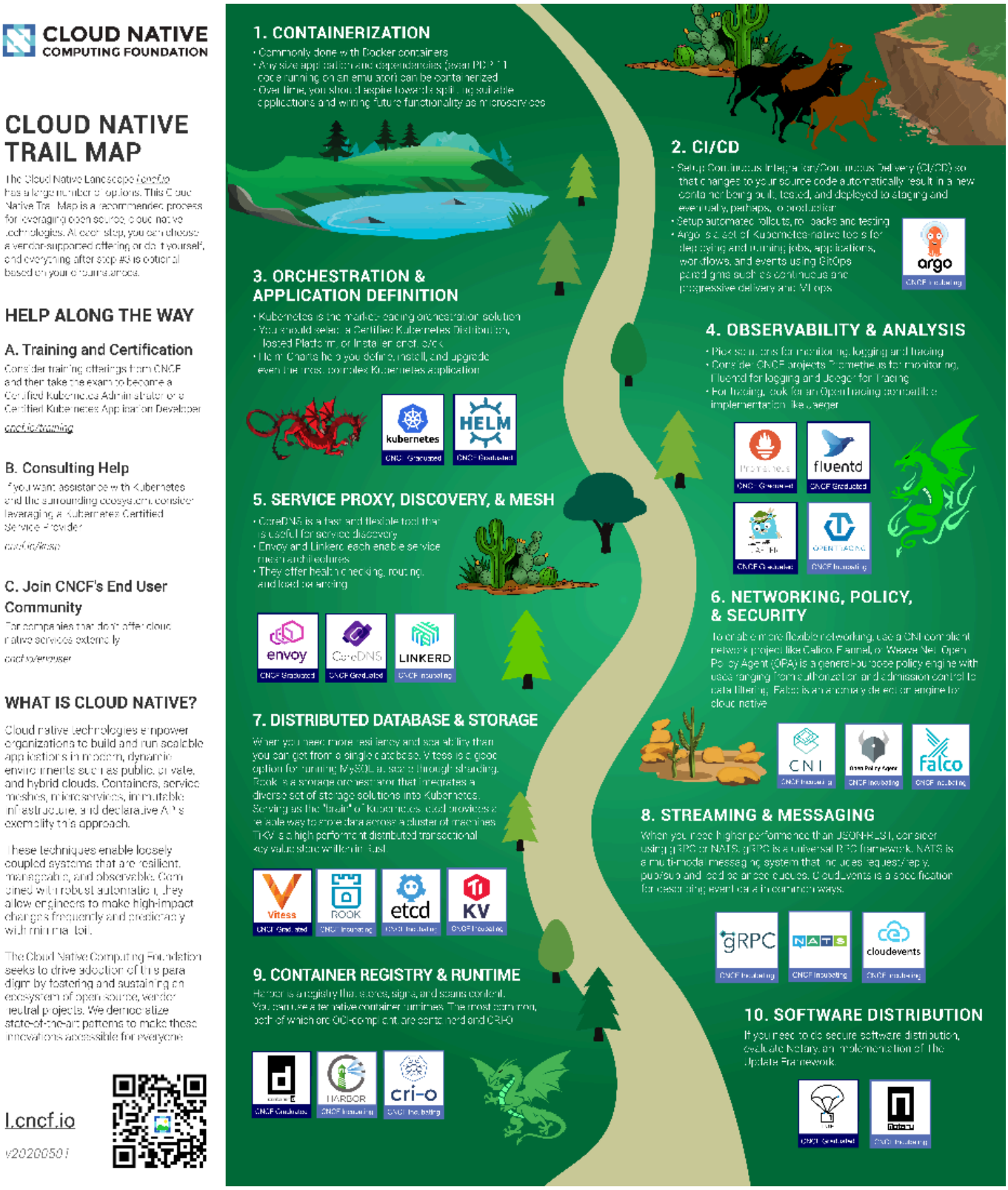
Containers for cloud-native applications

Containerizing a microservice is simple and straightforward.

The Cloud Native Computing Foundation places microservice containerization as the first step in their Cloud-Native Trail Map - guidance for enterprises beginning their cloud-native journey.

The cloud native trail map is a GitHub project of CNCF, available here: **(https://github.com/cncf/trailmap)**

CNCF/TRAILMAP (HTTPS://GITHUB.COM/CNCF/TRAILMAP)



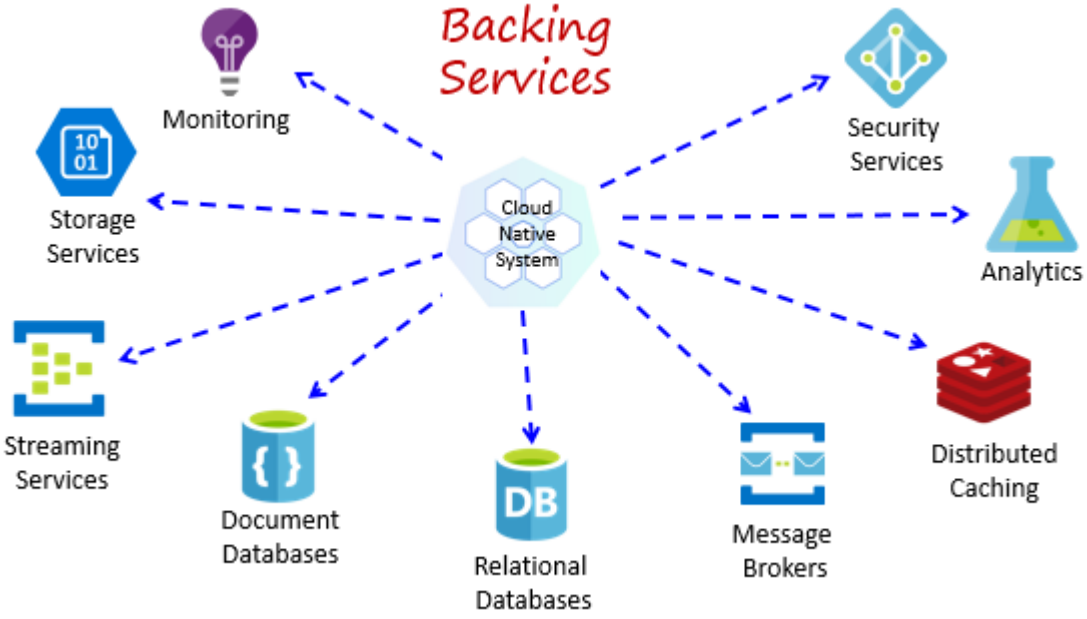
The code, its dependencies, and runtime are packaged into a binary called a container image. Images are stored in a container registry, which acts as a repository or library for images. A registry can be located on your development computer, in your data center, or in a public cloud. Docker itself maintains a public registry via Docker Hub.

Backing services

Cloud-native systems depend upon many different ancillary resources, such as data stores, message brokers, monitoring, and identity services. These services are known as backing services.

You could host your own backing services, but then you'd be responsible for licensing, provisioning, and managing those resources. The cloud provider operates the resource at scale and bears the responsibility for performance, security, and maintenance.

Cloud-native systems favor managed backing services from cloud vendors. The savings in time and labor can be significant. The operational risk of hosting your own and experiencing trouble can get expensive fast.
This graphic shows some of the most common backing services.



Cloud providers offer a rich assortment of managed backing services. Instead of owning the service, you simply consume it.

Monitoring, redundancy, and availability are built into the service.

Providers guarantee service level performance and fully support their managed services - open a ticket and they fix your issue.

Automation

Cloud-native systems embrace microservices, containers, and modern system design to achieve speed and agility, but that's only part of the story:

- How do you provision the cloud environments upon which these systems run?

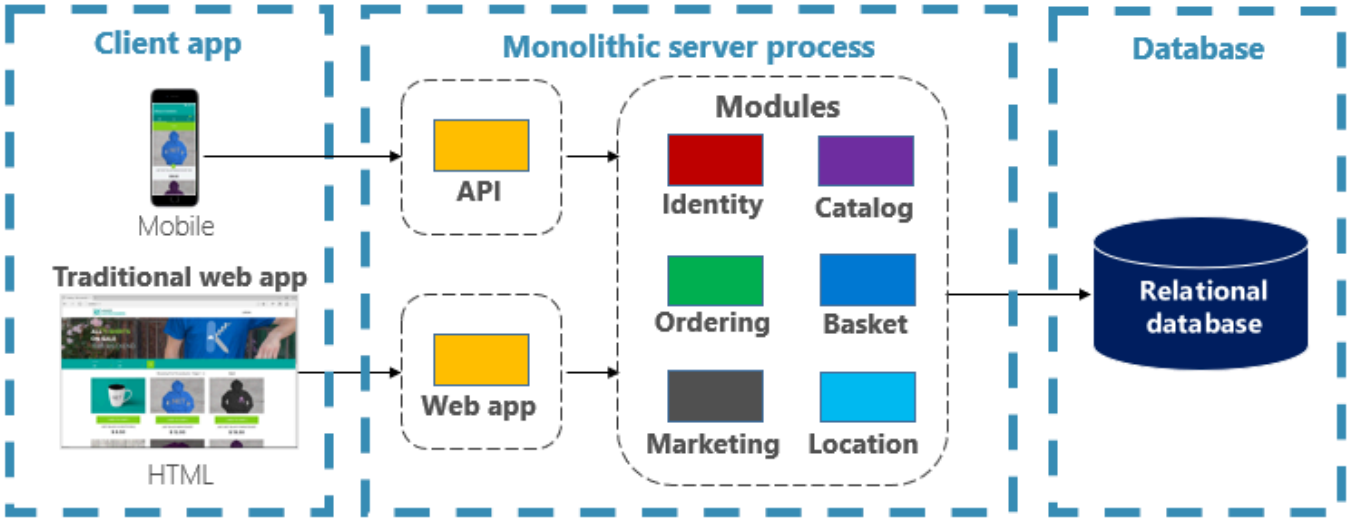
- How do you rapidly deploy app features and updates?

- How do you round out the full picture?
  The answer is to automate infrastructure and deployments.

The automation of infrastructure is known as “infrastructure as code”, or IaC. With IaC, you automate platform provisioning and application deployment. You essentially apply software engineering practices such as testing and versioning to your DevOps practices. Your infrastructure and deployments are automated, consistent, and repeatable.

Application design

When tasked with building an information system that will deliver services, the traditional approach is to use a monolithic design.



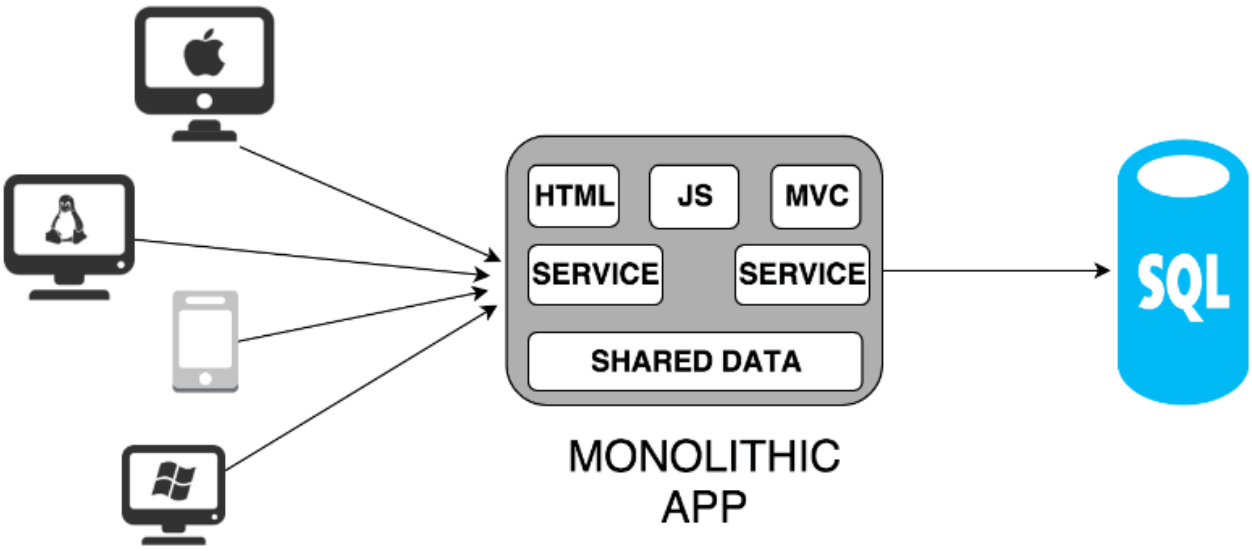
The name could be seen as slightly misleading since a monolithic application implies that it is a single executable being used to run the system. When considering web application, this becomes the typical three-tier architecture that includes the database, the business logic running on the web server, and the presentation tier running on the client's browser. If the web server is running the entire application, it is seen as a monolithic design.

Advantages of a monolithic design.

The 3-tier model is very popular. It was definitely a step-up from buying software on CDs and running it locally on a system, hoping that it would be compatible. The use of a web-deployed application makes compatibility with a myriad system much easier to achieve, and updates to the application happen mostly at the business logic layer without affecting the client app.

The design and development of applications using this design is straightforward. It is easy to:

- Build
- Test
- Deploy
- Troubleshoot
- Scale vertically



Fear cycle

Of course, the monolithic characteristic is associated to the business logic of the application, i.e., the code and functionalities running on the web server. As the application evolves over time, different people will work on it. Because everything in the application is tied together, changes can have unexpected consequences. Side effects can be costly to manage and fix. The application can become convoluted, bloated, and difficult to understand. The documentation, if it exists, is not always kept up to date with the changes.

At some point, no one knows the entirety of the application. The application might get so complicated as to prohibit one single person to understand everything going on. This is where the fear cycle set in:

- Administrators are reluctant to move forward with changes because of the potential impact to other parts of the application.
- New features become tricky, time-consuming, and expensive to implement.
- Each new release is brought to its simplest form and requires a full deployment of the entire application.

Legacy application

Once the application reaches a stage where it becomes very difficult to maintain and evolve, and new technology cannot be employed because of compatibility issues, the application gets designated as a legacy application. This can mean a number of things, but typically:

- The application is unreliable, and some components are unstable.
- The application cannot be updated using new frameworks and technology.
- It needs replacing or decommissioning and kept functional until such a time where money and resources are available to do this.

The risk of a catastrophic crash increases the longer an organization waits to address a legacy application and replace it.

Reading:
The Government of Canada undertook a massive project to replace their outdated pay system with the ill-fated Phoenix Pay system. The new system was launched in 2016 to an extremely high rate of errors and inaccuracies, that are still not fixed to this day. A big part of the issue was that the previous pay system had become so customized and convoluted that understanding it enough to reverse engineer it was almost impossible. Read the following article to get a sense of the complications experienced during the implementation of the project. **(https://www.cbc.ca/news/canada/ottawa/phoenix-pay-system-issues-impact-public-servants-six-years-later-1.6457840)**

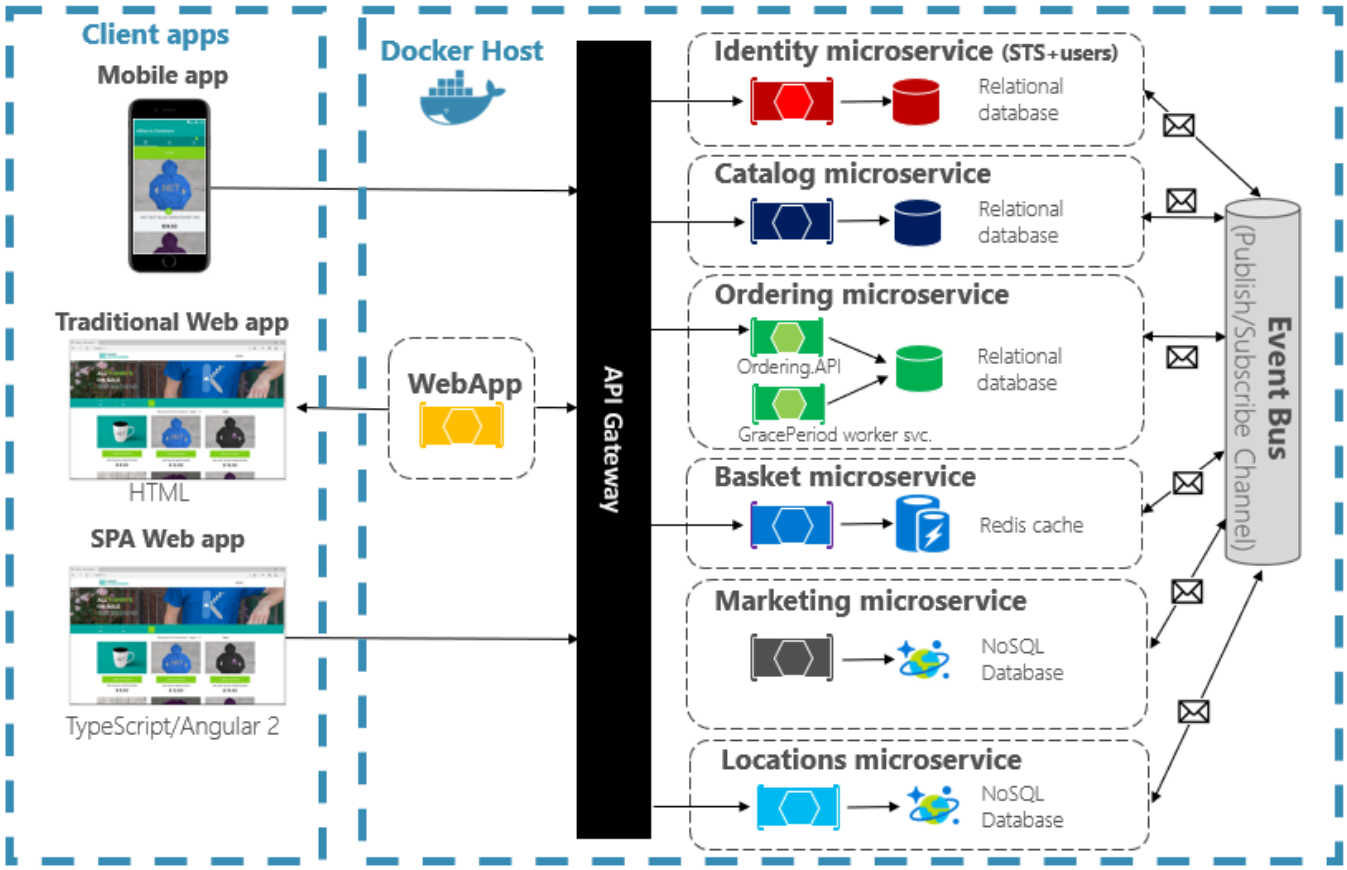
PHOENIX 'NIGHTMARE' STILL HAUNTING PUBLIC SERVANTS

(HTTPS://WWW.CBC.CA/NEWS/CANADA/OTTAWA/PHOENIX-PAY-SYSTEM-ISSUES-IMPACT-PUBLIC-SERVANTS-SIX-YEARS-LATER-1.6457840)

Solution: cloud-native approach

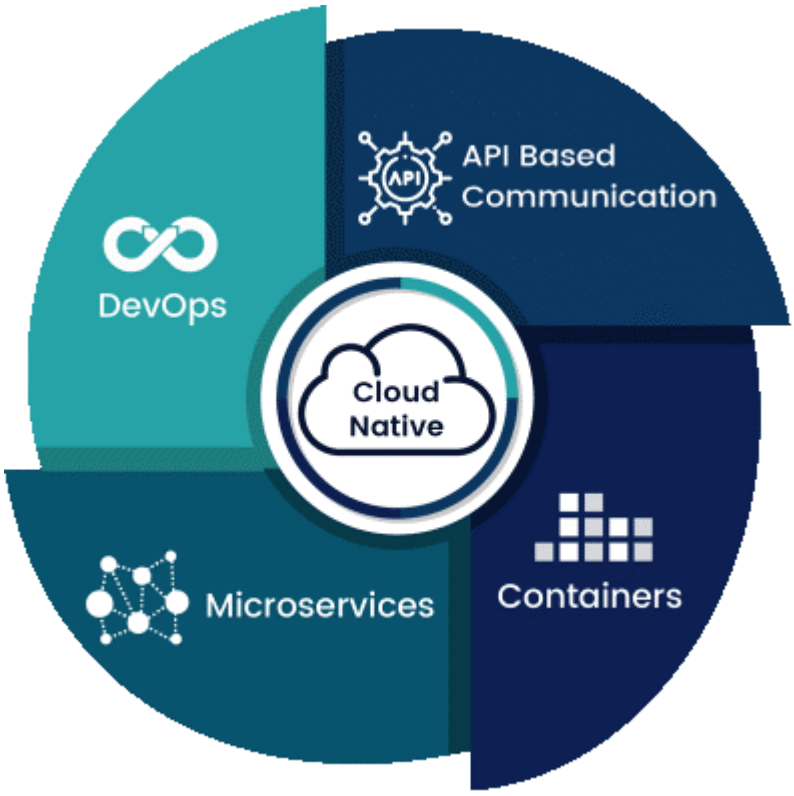
To avoid this scenario, systems can be designed to make the most out of cloud-native computing. Within a short time, cloud-native has become a driving trend in the software industry. It's a new way to construct large, complex systems. The approach takes full advantage of modern software development practices, technologies, and cloud infrastructure. Cloud native changes the way you design, implement, deploy, and operationalize systems.

In a cloud-native design, the applications use microservices that run independently to accomplish tasks for the application. The applications use an event-driven architecture to invoke services. An event bus sends and receives messages from the microservices to coordinate the application.



The concept of cloud-native computing combines new approaches and technologies such as:

- **DevOps**: a combination of development and operations, designed for agile development and continuous integration and deployment.
- **API based communication**: components communicate and exchange information using API calls instead of using a tightly coupled interface.
- **Microservices**: instead of a monolithic server process, multiple smaller services that provide a specific functionality are combined to create the application.
- **Containers**: the software is packaged with all the necessary elements to be able to run in any environment, for easier deployment and migration.



Cloud Native Computing Foundation (CNCF)

Unlike the continuous hype that drives our industry, cloud native is for-real. Consider the Cloud Native Computing Foundation (CNCF), a consortium of over 400 major corporations. Its charter is to make cloud-native computing ubiquitous across technology and cloud stacks. As one of the most influential open-source groups, it hosts many of the fastest-growing open source-projects in GitHub. From the website of the CNCF:

- Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.

- These techniques enable loosely coupled systems that are resilient, manageable, and observable. Combined with robust automation, they allow engineers to make high-impact changes frequently and predictably with minimal toil.

- The Cloud Native Computing Foundation seeks to drive adoption of this paradigm by fostering and sustaining an ecosystem of open source, vendor-neutral projects. We democratize state-of-the-art patterns to make these innovations accessible for everyone.

CNCF hosts critical components of the global technology infrastructure and are a major player in driving and facilitating the adoption of cloud-native computing. Find out more about the organization here: **(https://www.cncf.io/)**

CNCF (HTTPS://WWW.CNCF.IO/)

Cloud-native computing

The CNCF fosters an ecosystem of open-source and vendor-neutrality.
Following that lead, this course presents cloud-native principles, patterns, and best practices that are technology agnostic.
At the same time, we discuss some of the services and infrastructure available from major cloud service providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).







Google Cloud

What is cloud native?

The Cloud Native Computing Foundation provides the official definition:

- Cloud-native technologies empower organizations to **build** and **run** scalable applications in modern, dynamic environments such as **public**, **private**, and **hybrid** clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.

- These techniques enable loosely coupled systems that are resilient, manageable, and observable. Combined with robust automation, they allow engineers to make high-impact changes frequently and predictably with minimal toil.

Cloud native is about speed and agility.

Industry examples

Here are some companies who have implemented cloud-native techniques. Think about the speed, agility, and scalability they've achieved.

NETFLIX

Netflix has over 600 services in production. New deployments to these services occur 100 times per day.

Uber

Uber has over 1,000 services in production. Deployments occurs several thousand times each week.



WeChat has over 3,000 services in production. Deployments to these services occur 1,000 times a day.

In this lesson, we covered the modern design pillar of cloud native. You learned about the Twelve-Factor methodology, factors that reflect today's modern cloud application design, principles, patterns, and best practices for cloud-native app design, and infrastructure and operational concerns.

In the next lesson, you will learn more about microservices.

References

The references listed here, coupled with the course developer's experience, have been used in the development of this module:

Idowu, Toye (2019). The 12-Factor App Methodology Explained. BMC. Retrieved from **https://www.bmc.com/blogs/twelve-factor-app/ (https://www.bmc.com/blogs/twelve-factor-app/)**

Meren, Libby (2021). Defining a modern app. Cloud Native Computing Foundation. Retrieved from **https://www.cncf.io/blog/2021/11/02/defining-a-modern-app/ (https://www.cncf.io/blog/2021/11/02/defining-a-modern-app/)**

Wiggins, Adam (2017). The Twelve Factors. 12factor. Retrieved from **https://12factor.net/ (https://12factor.net/)**

Learning activity



Reflect on and provide answers to the following questions:

1. What is the foremost pillar of cloud-native application design and delivery?

Click to see the answer.

2. What do the twelve factors represent?

Click to see the answer.

3. How are configurations handled according to the twelve factors?

Click to see the answer.

4. What is an example of a backing service?

Click to see the answer.

5. How does port binding isolate microservices?

Click to see the answer.

The Twelve Factors

Modern applications are more and more commonly delivered as a service using cloud computing technology, in the “software-as-a-service”, or SaaS business model. The twelve factors are a methodology for building SaaS apps that:

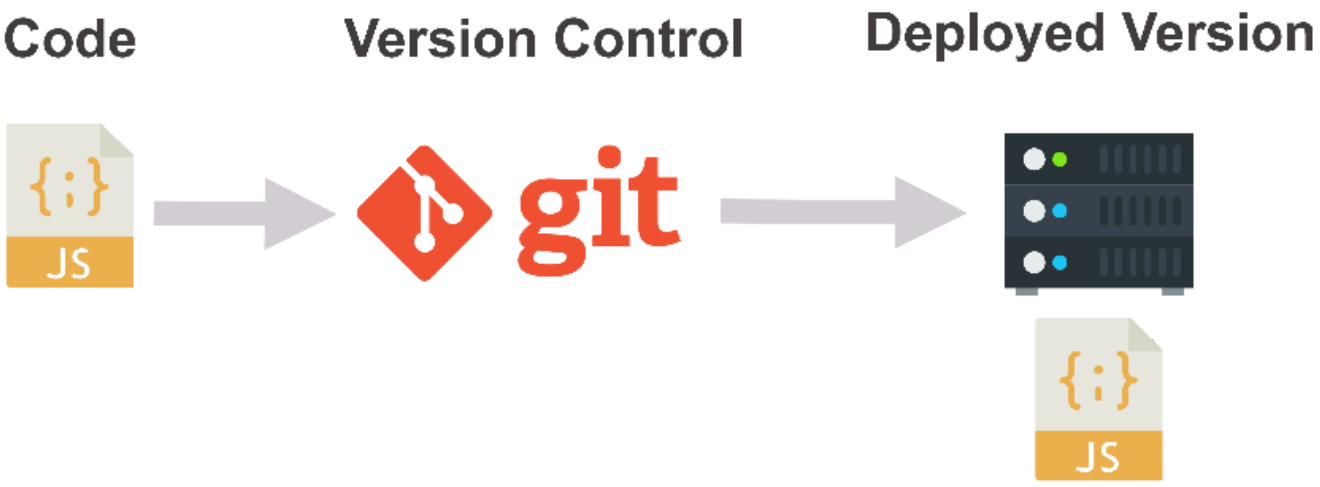
- Use declarative formats for setup automation, to minimize time and cost for new developers joining the project.
- Have a clean contract with the underlying operating system, offering maximum portability between execution environments.
- Are suitable for deployment on modern cloud platforms, obviating the need for servers and systems administration.
- Minimize divergence between development and production, enabling continuous deployment for maximum agility.
- Can scale up without significant changes to tooling, architecture, or development practices.

Let’s look at each of the twelve factors of the methodology.

Factor 1: Code Base

This factor is about the code base of microservices. The principle states that:

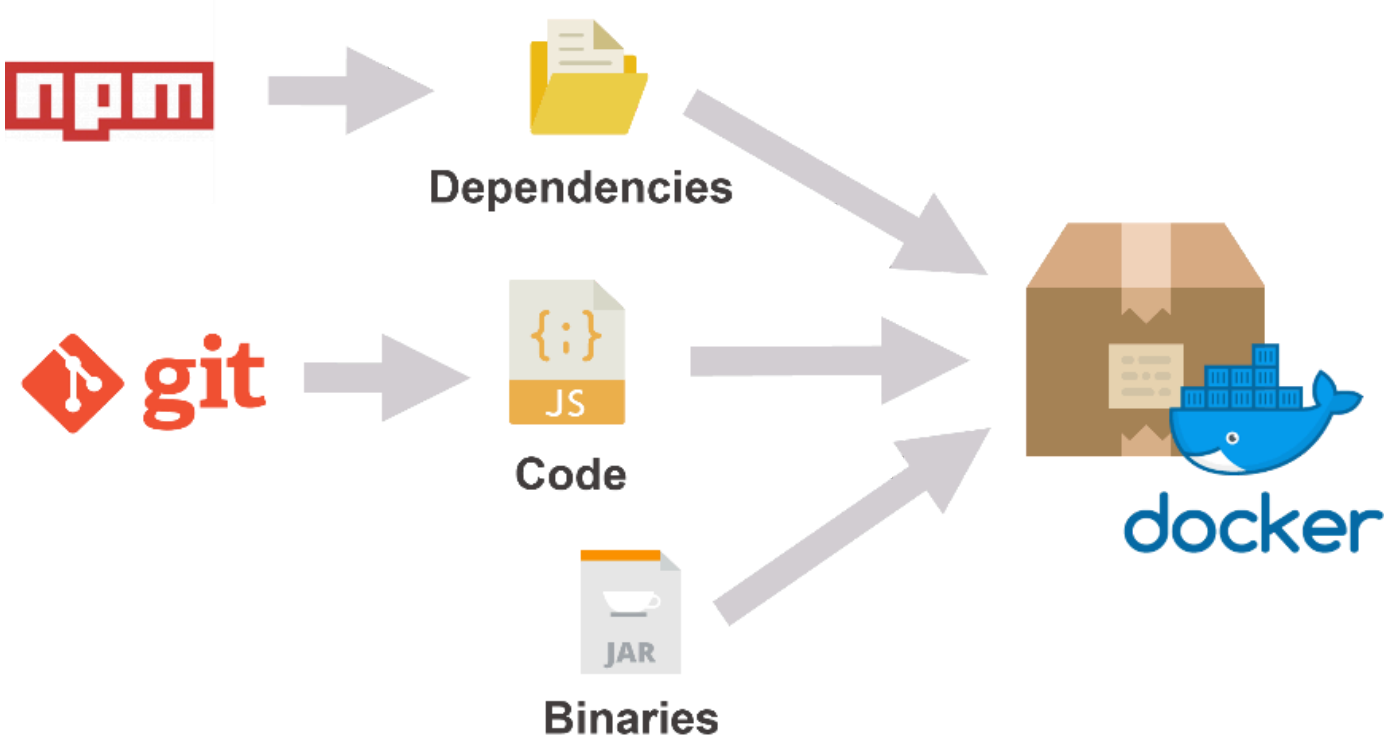
- There is a single code base for each microservice, stored in its own repository.
- The code base is tracked with version control, and it can be deployed to multiple environments (QA, Staging, Production).



Factor 2: Dependencies

This factor concerns the dependencies of the microservices and aims to provide autonomy to the microservice when it comes to its deployment environment and dependencies. The principle states that:

- Each microservice isolates and packages its own dependencies, embracing changes without impacting the entire system.
- The host is never depended upon to provide a required dependency.
- Application deployments should carry all their dependencies with them.



Factor 3: Configurations

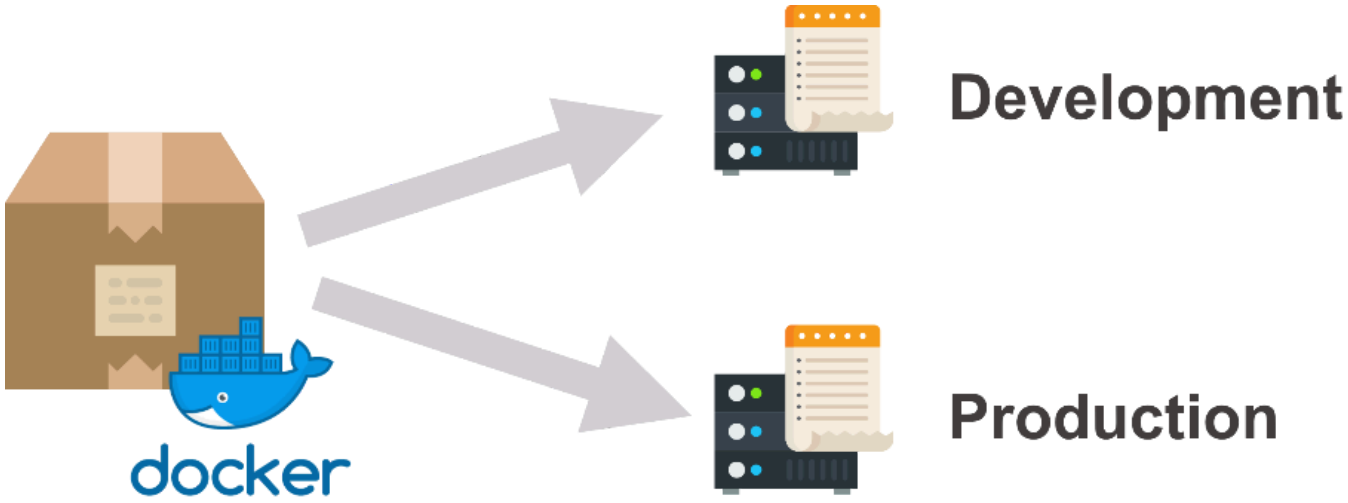
Configuration is a key part of the management and optimization of microservices. Lack of continuity in the configuration of microservices can lead to security issues, breakdowns, data loss, and more. This principle of the twelve factors states that:

- Configuration information is moved out of the microservice and externalized through a configuration management tool outside of the code.
- The same deployment can propagate across environments with the correct configuration

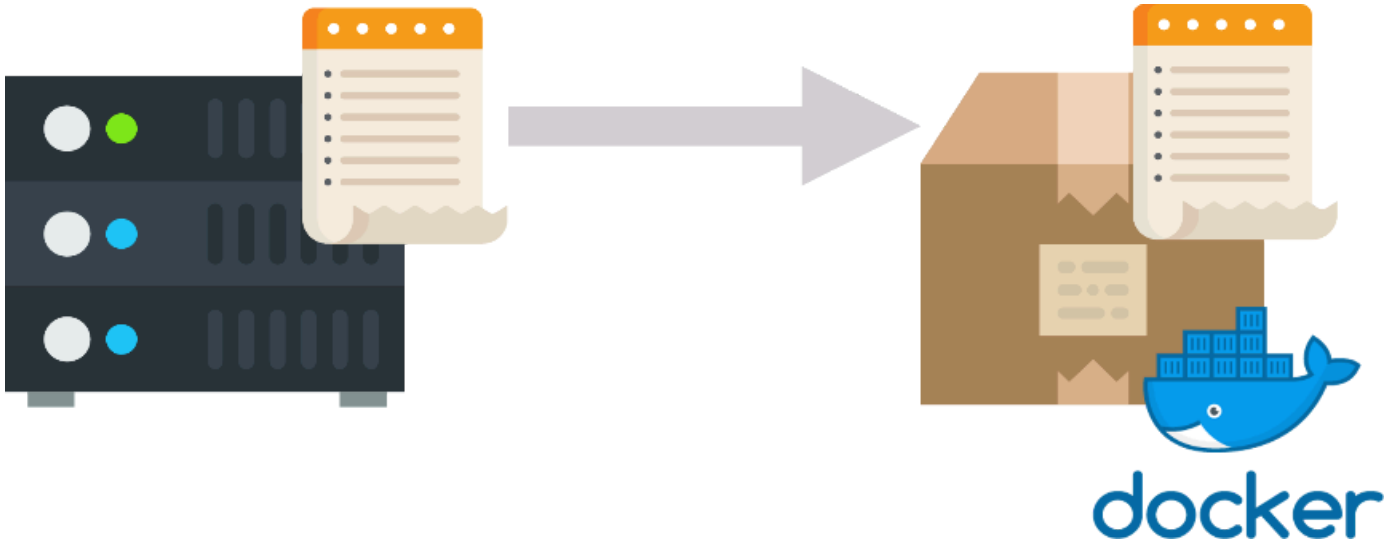
applied.

- At runtime the container gets its configuration from the environment.

Large image will automatically adjust to its parent to fit small screen size. Recommended image size: at least 1200 pixels wide.



The same container is deployed to both environments.

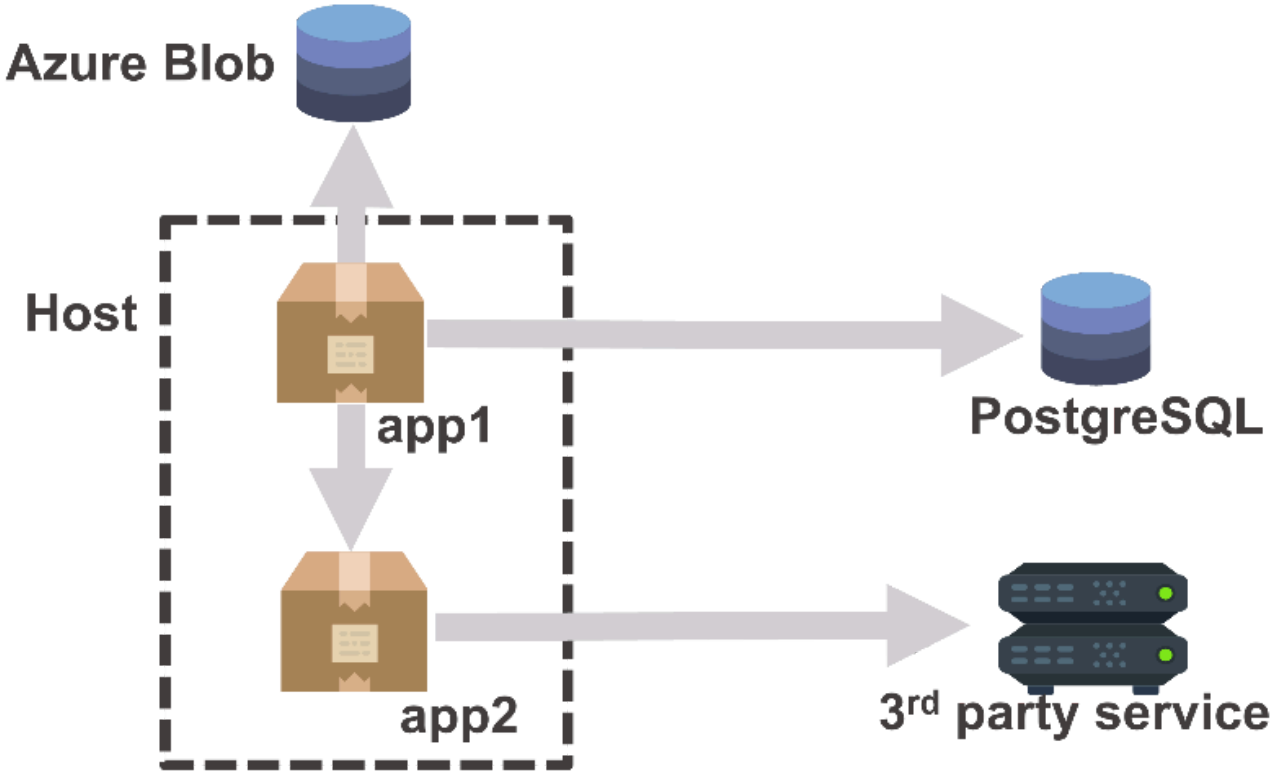


Configuration is part of the environment on the host.

Factor 4: Backing Services

Backing services are resources provisioned by cloud computing environments. They are abstracted and decoupled from the applications in a cloud-native approach. This factor states that:

- Ancillary resources (data stores, caches, message brokers) should be exposed via an addressable URL.
- Enable the resource to be interchangeable by decoupling it from the application.
- Treat local services just like remote third-party ones.

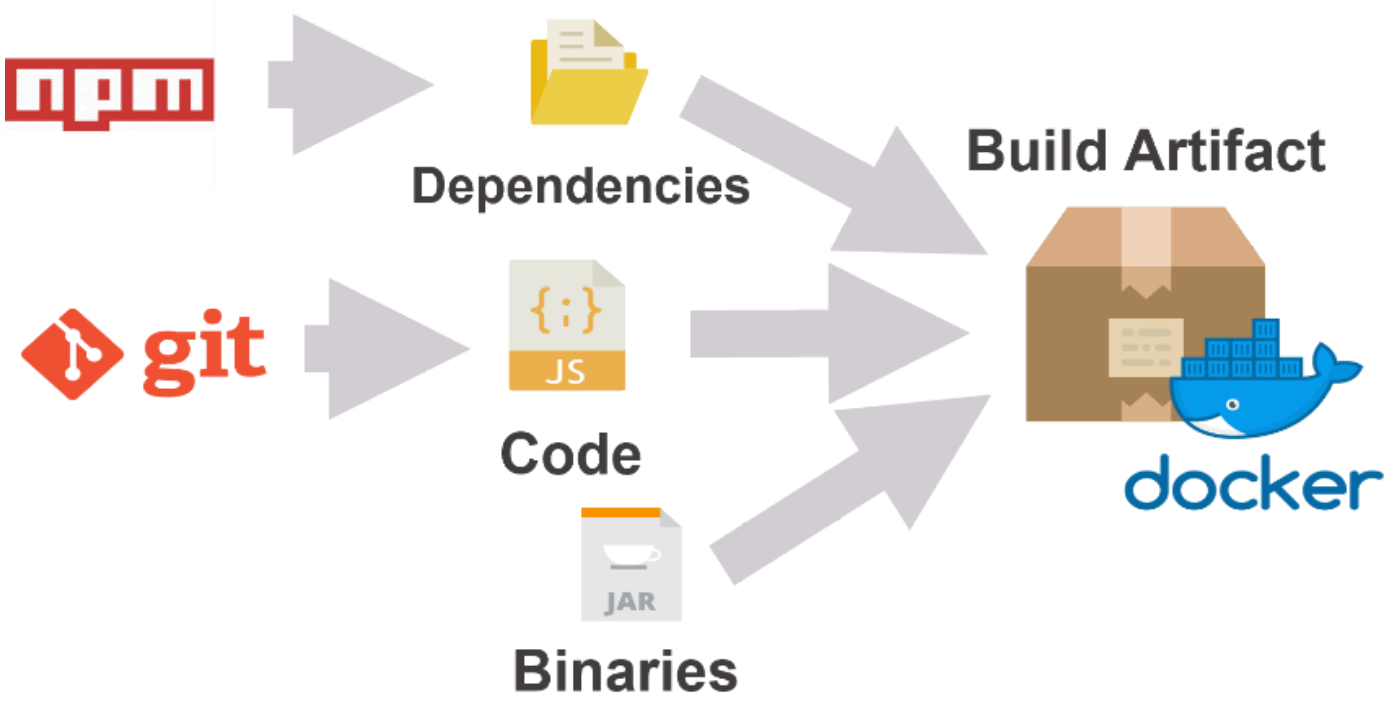


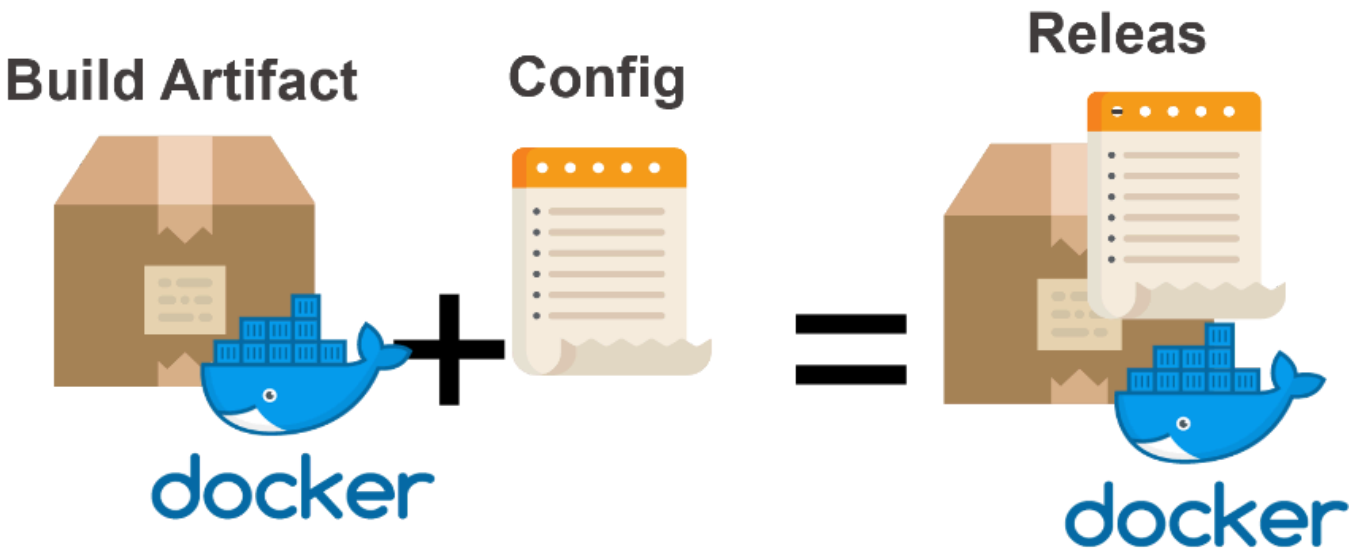
Factor 5: Build, Release, Run

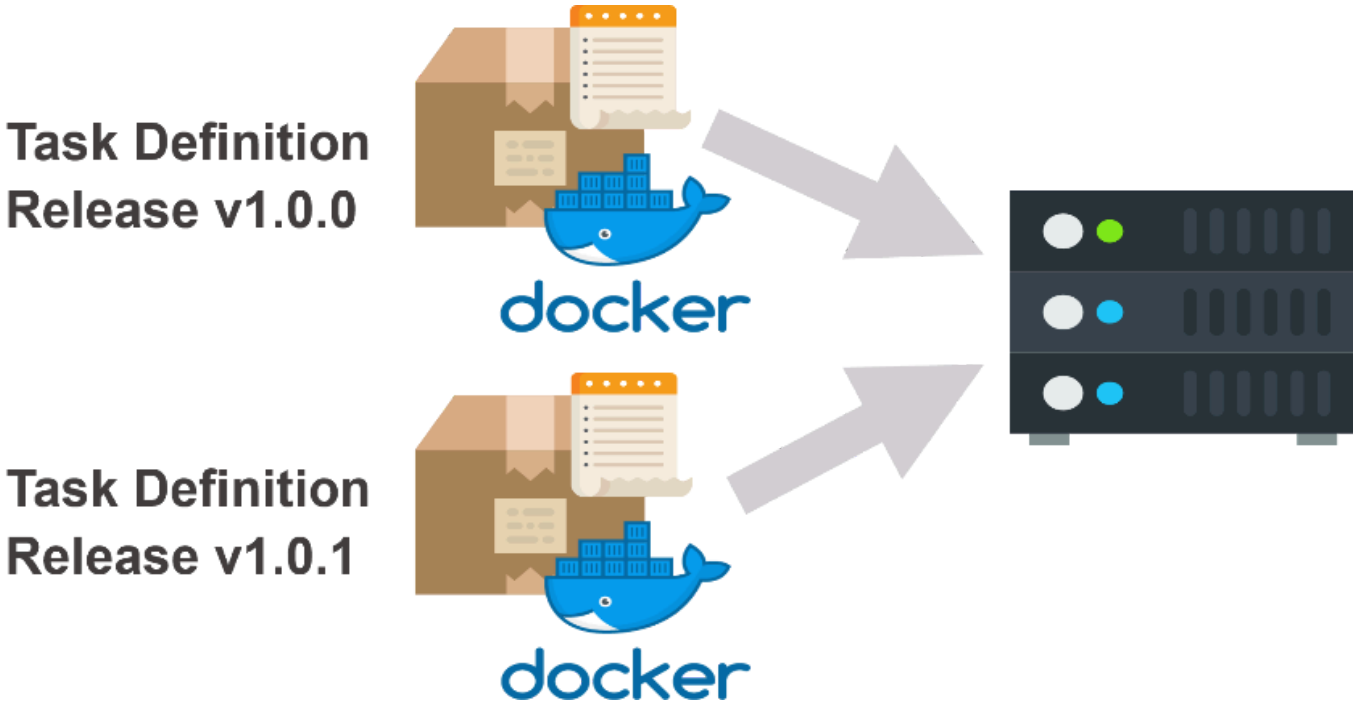
Modern application design espouses the DevOps principles of continuous integration and deployment/delivery. This is mainly applied by the creation of a CI/CD pipeline for the development and deployment of releases. This principle of the twelve factors states that:

- Each release must enforce a strict separation across the build, release, and run stages.
- Each should be tagged with a unique ID and support the ability to roll back.

Modern CI/CD systems help fulfill this principle.


npm
Dependencies
Build Artifact
git
JS
Code
docker
JAR
Binaries


Build Artifact
Config
Releas
docker
docker

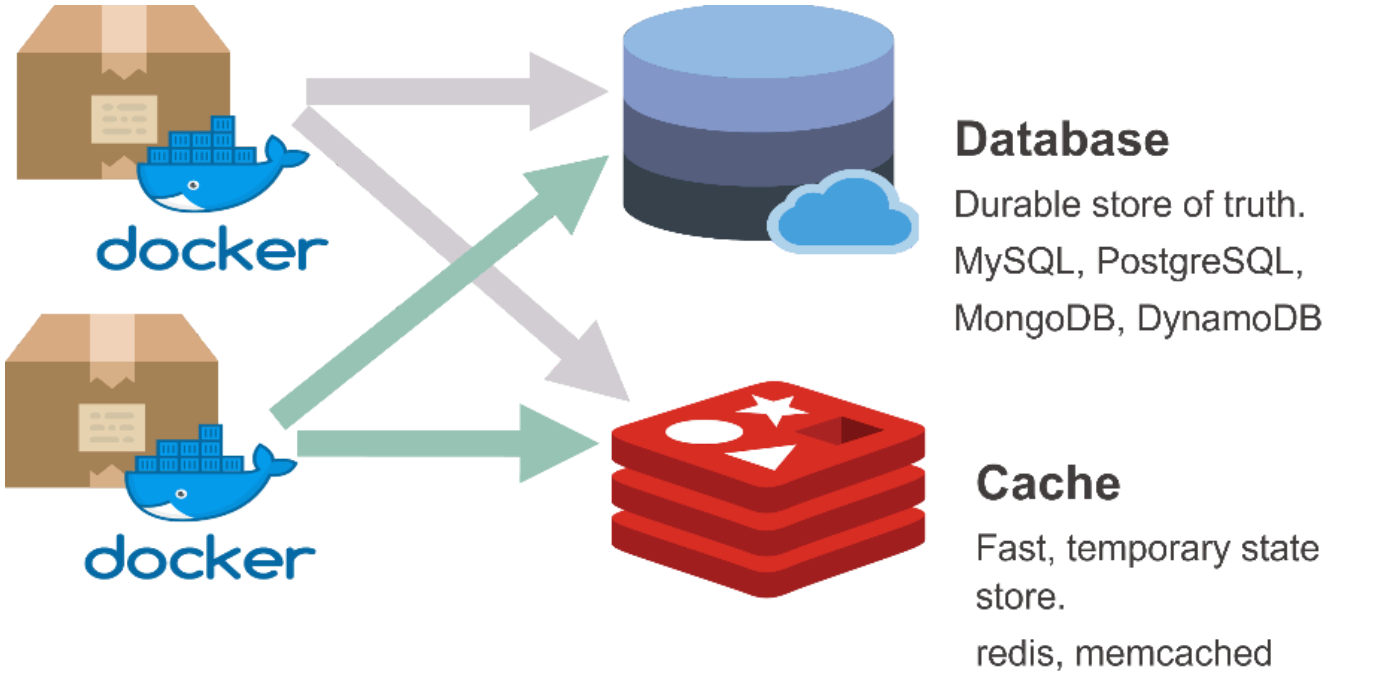


Factor 6: Processes

Microservices create processes when they run, and the efficiency of those processes will determine the capacity of the overall application. If key processes are slow or bogged down, this can create a bottleneck, even with the use of microservices. This principle of the twelve factors states that:

- Each microservice should execute in its own process, isolated from other running services.
- Externalize required state to a backing service such as a distributed cache or data store.

To optimize the speed and efficiency, microservices will leverage localized temporary and volatile caches of data. The durable and long-term storage is provided by relational databases.

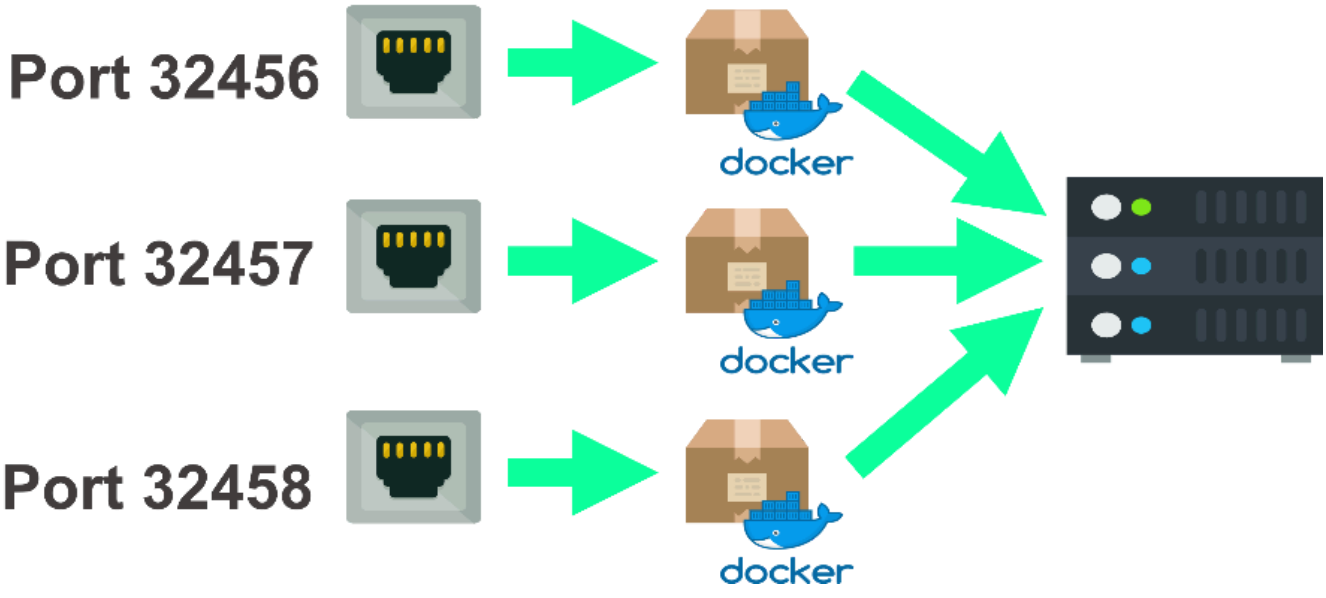


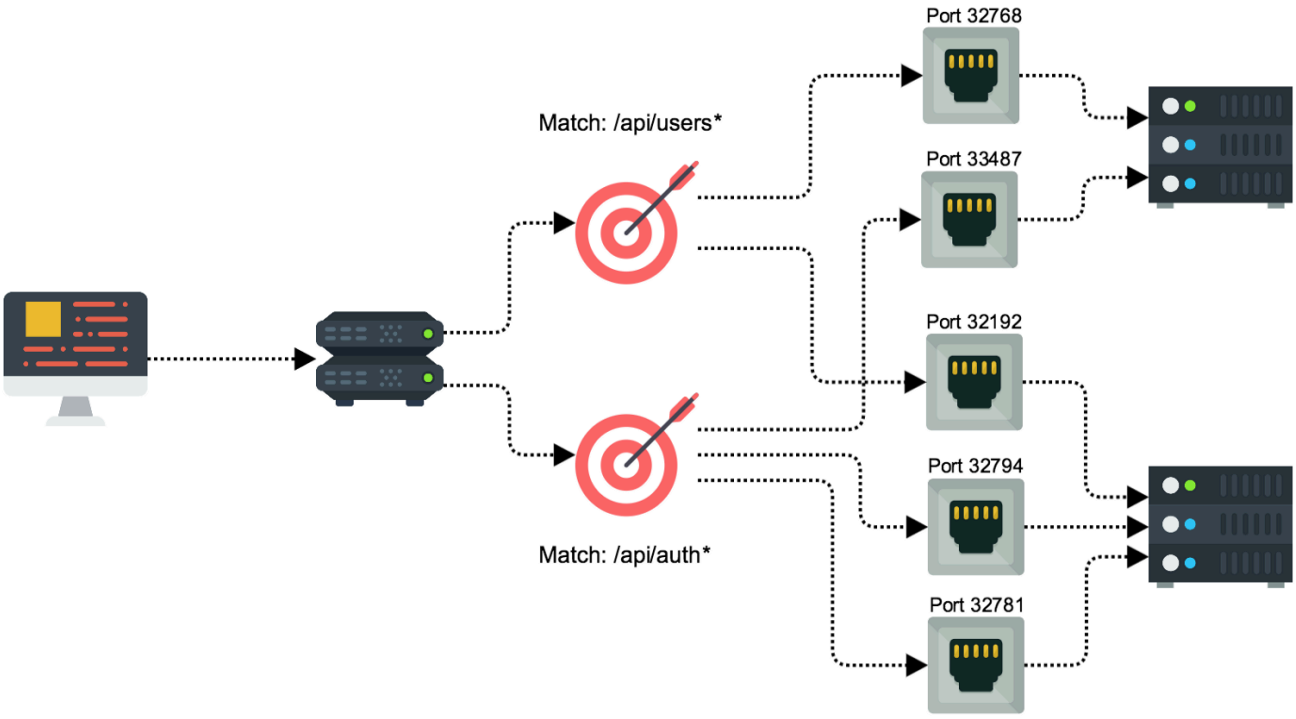
Factor 7: Port Binding

Port binding is a networking concept that isolates applications at a lower level. The isolation is done at the network level instead of the logical and application levels by binding the microservice to its own specific port. This principle of the twelve factors states that:

- Each microservice should be self-contained with its interfaces and functionality exposed on its own port.

Using this approach provides isolation from other microservices at the network level.





Factor 8: Concurrency

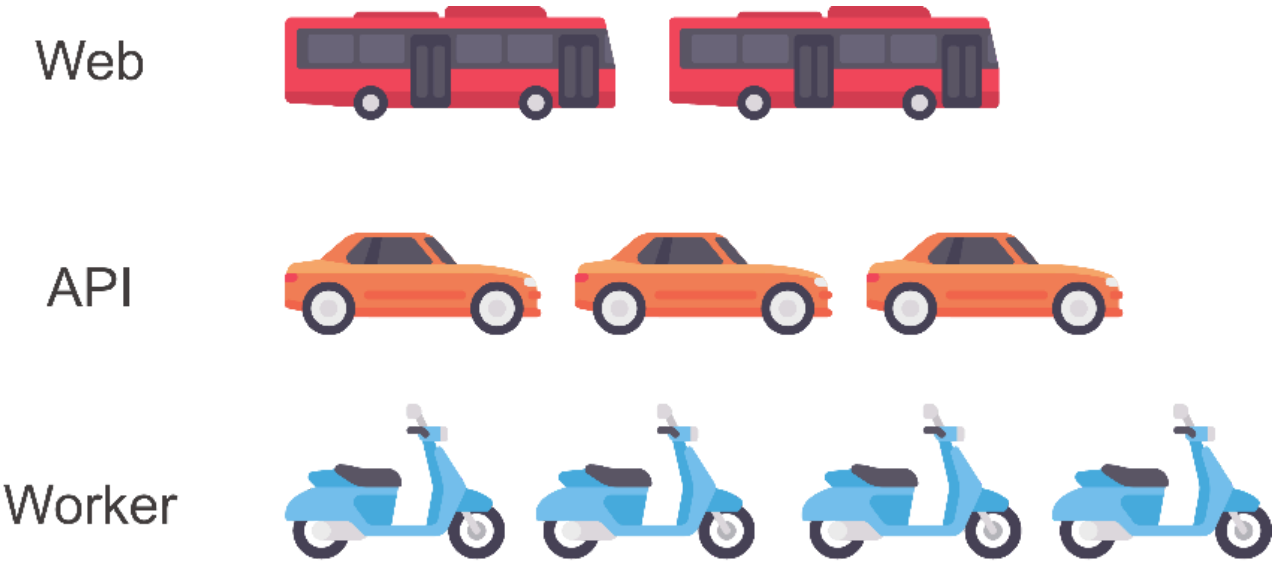
When it comes to scaling the resources that support an application, there are two basic approaches: vertical scaling (scaling up), and horizontal scaling (scaling out). You will recall the analogy of Pets vs. Cattle when it comes to scaling. Cloud computing technologies see the resources as elastic and disposable. Generic instances are created as required and provisioned to support the application; hence the scaling is horizontal, not vertical. This is also the approach used by the twelve factors. This factor states that:

- Scale out services horizontally across multiple identical processes (copies) as opposed to scaling-up a single large instance on the most powerful machine available.
- Develop the application to be concurrent making scaling out in cloud environments seamless.

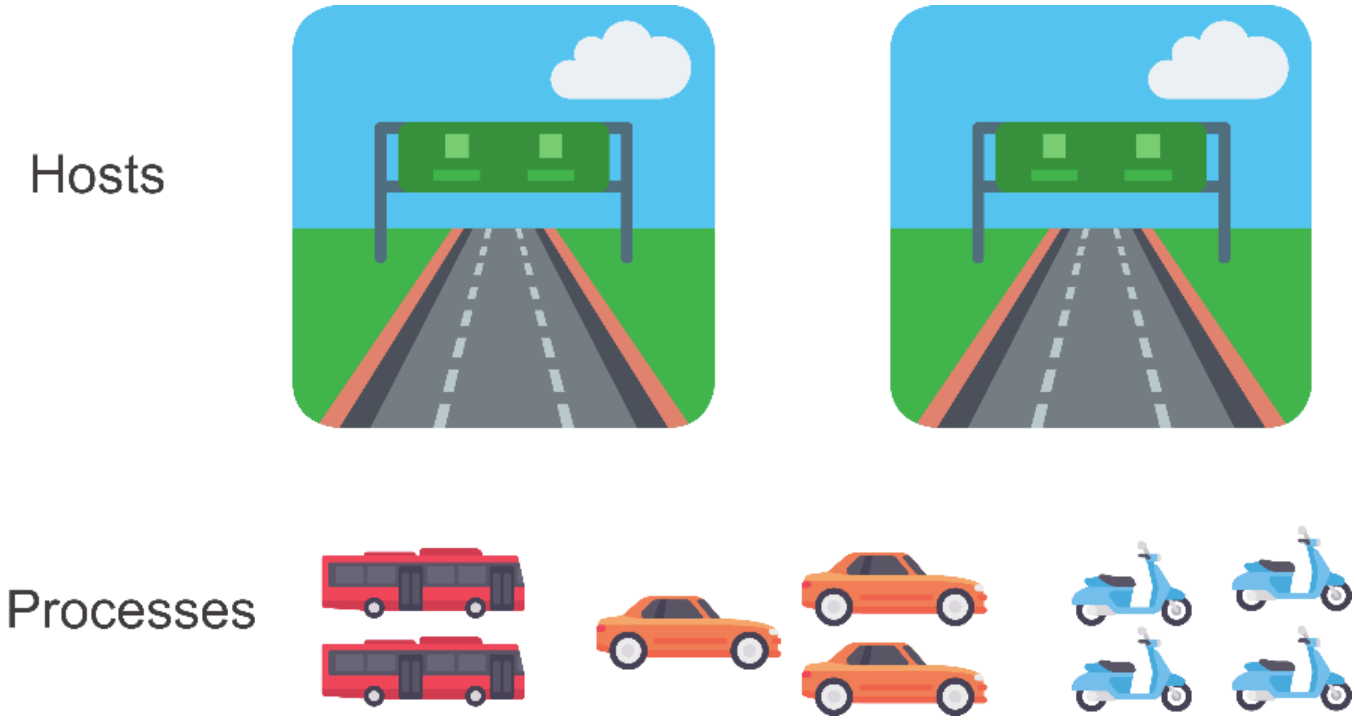
A foundational approach to cloud native application development is to utilize the capabilities of cloud, which include an abstracted, layered infrastructure, that can seamlessly grow and adapt to meet everchanging requirements.



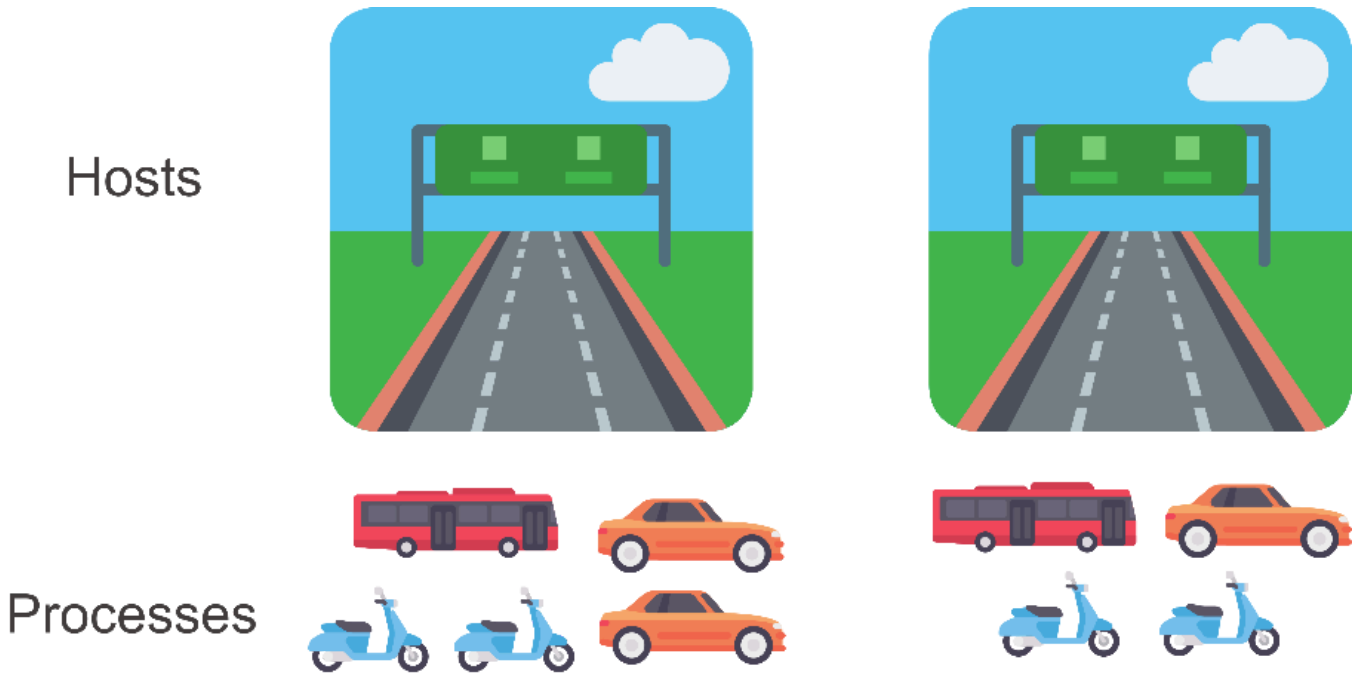
Single-instance deployment of application components.



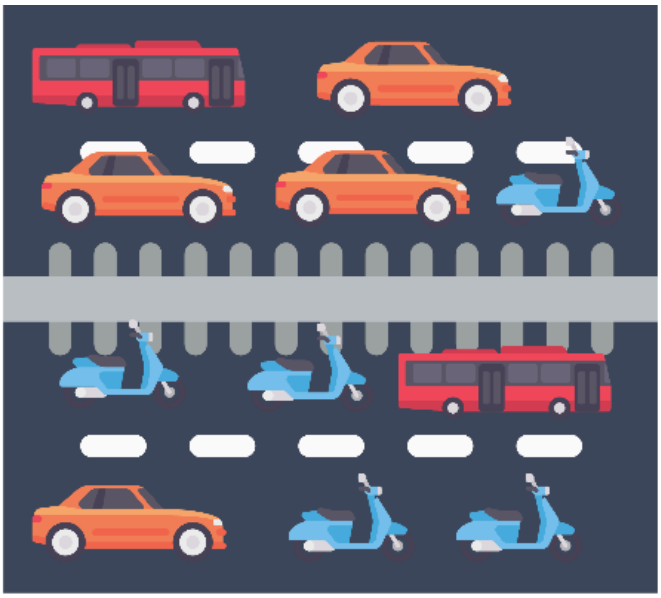
Adding processes to scale the resources horizontally.



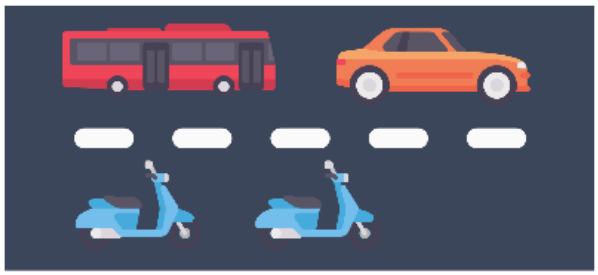
Defining hosts to receive the processes.



Instead of using dedicated hosts for a type of instance, which creates potential single-point-of-failure situations, processes are distributed amongst the instances.



Large Host = More Concurrent Processes



Small Host = Fewer Concurrent Processes

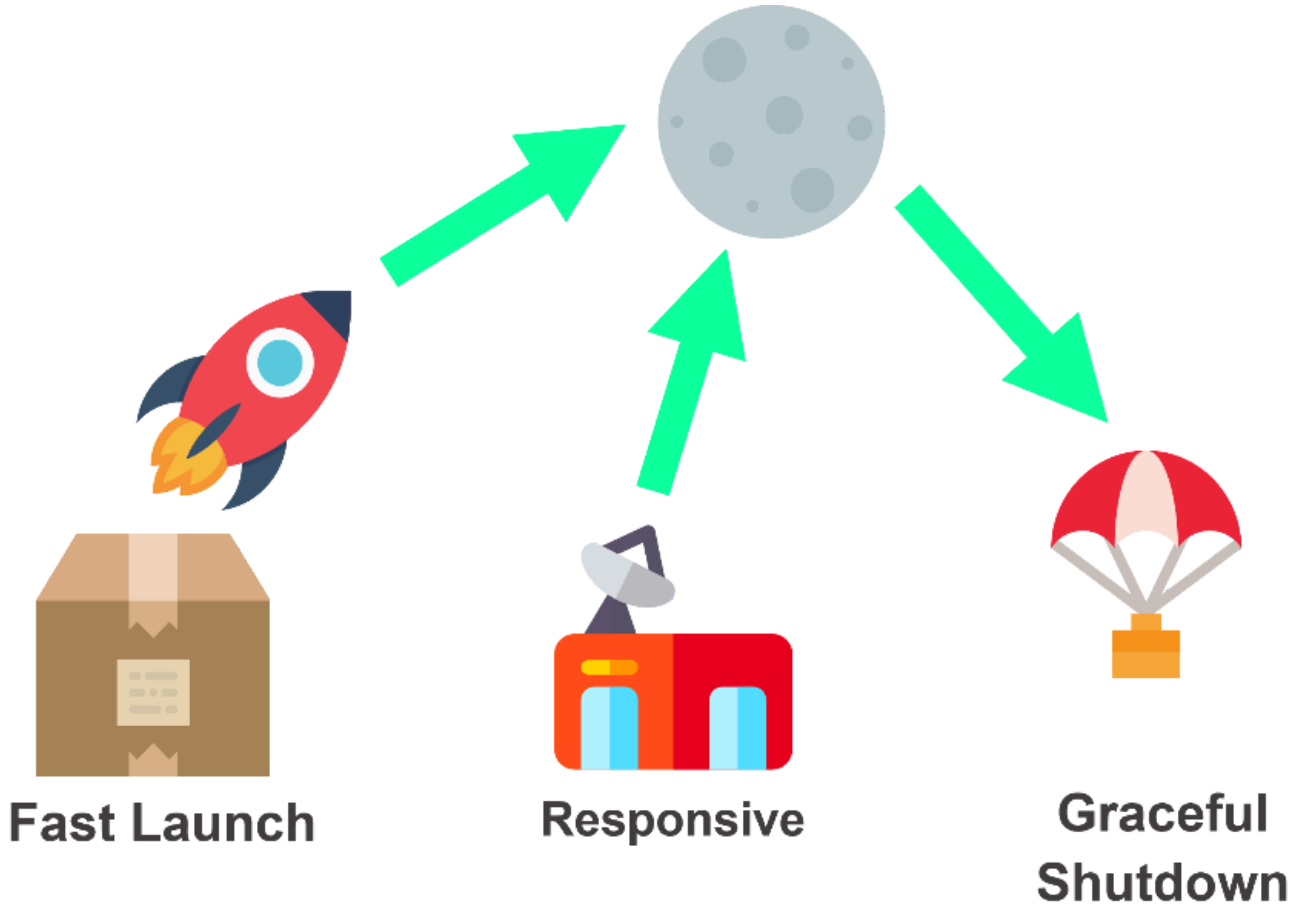
Large hosts can accommodate higher numbers of concurrent processes. This would be a valuable strategy in high demand geographical areas. Smaller hosts are be suitable to bring the service closer to end-users where demand can be met with more modest resources.

Factor 9: Disposability

Along with the principle of scaling out seamlessly to elastically support the application with the necessary resources, this factor is about the disposability of server instances. This principle of the twelve factors states that:

- Service instances should be disposable.
- Favor fast startup to increase scalability opportunities and graceful shutdowns to leave the system in a correct state.

Docker containers along with an orchestrator inherently satisfy this requirement.

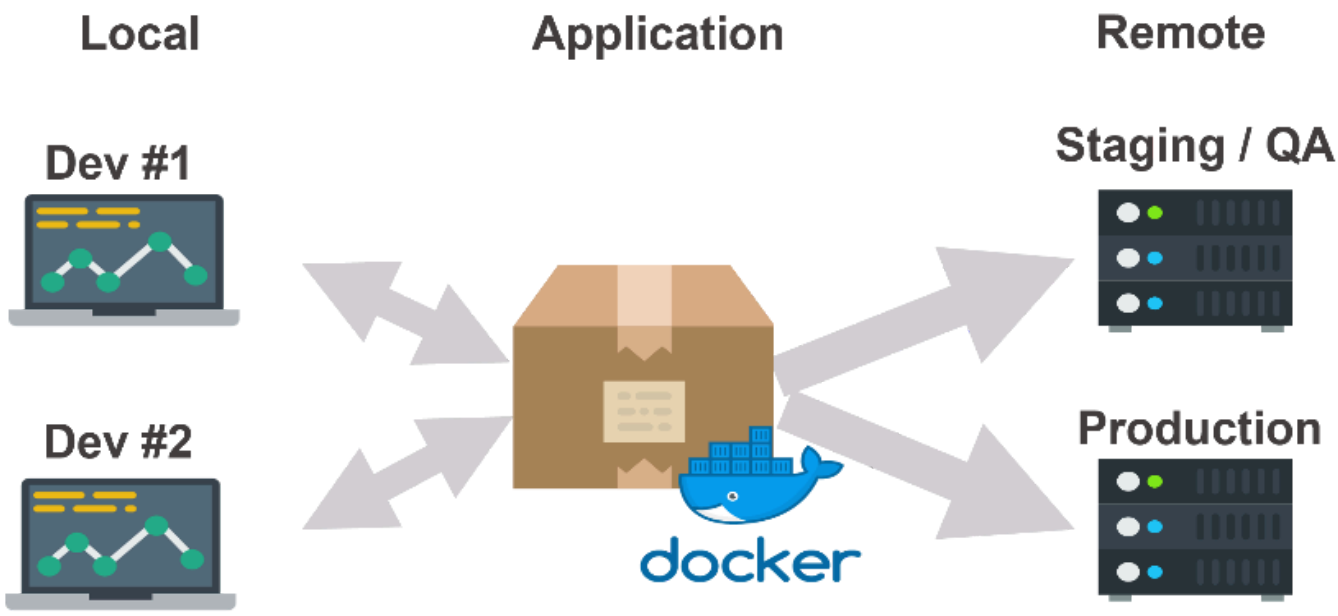


Service instances should be quick to launch, then responsive to demands, and be able to gracefully shut down without affecting the status of the application.

Factor 10: Dev/Prod Parity

A critical aspect of application development is the management of separate environments for development, testing, quality assurance, staging, and production. When the environments and provisioned and managed separately, discrepancies between the environments will inevitably emerge. These discrepancies can be about the configuration, available resources, application settings, usable data, and more. This principle of the twelve factors states that:

- Environments should be kept as similar as possible across the application lifecycle, avoiding costly shortcuts.
  Here, the adoption of containers can greatly contribute by promoting the same execution environment.

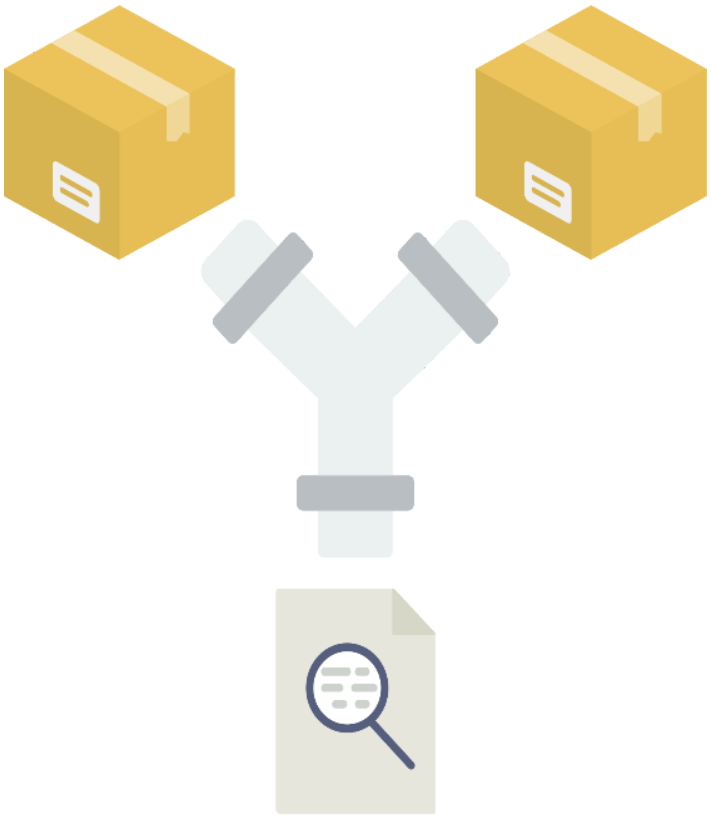


Factor 11: Logging

The notion of logging is transversal to many areas of concern for applications. Many people see logs as a tool for monitoring and detecting unusual or suspicious activities happening in the application ecosystem. For other, they act as a tool to detect failures, overloading, bottlenecks, and other operational concerns that need quick intervention. For the administrators of the system, logs can provide visibility to analyze the application and determine how it must evolve to meet demand, or if instead resources are overallocated and wasted.

In reality, logs are a key tool for all of these activities and more. In a modern application design approach, logs can actively participate to the functioning of a decentralized event-driven application. This principle of the twelve factors states that:

- Logs generated by microservices should be treated as event streams.
- The event streams should be processed with an event aggregator.
- Log data should be propagated to data mining/log management tools like Azure Monitor or Splunk and eventually to long-term archival.



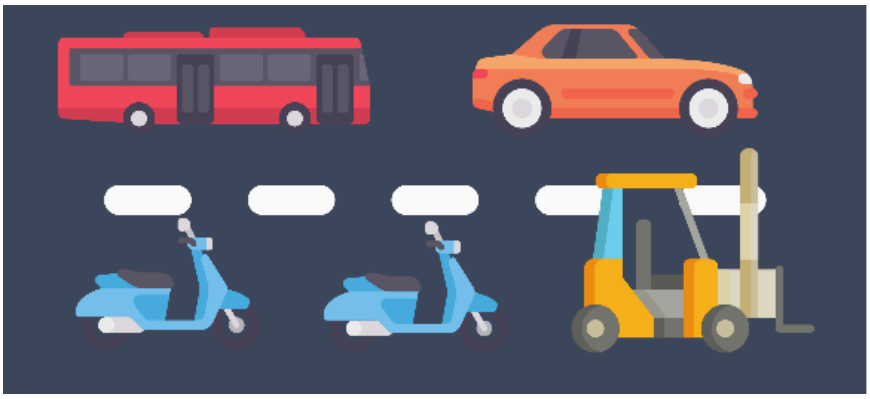
Factor 12: Admin Processes

Processes that run within applications can be separated between regular task and administrative or management tasks. There are key differences between the two types, relating to the criticality of the tasks, the privileges required to run, and the scope of the task. This last principle of the twelve factors states that:

- Administrative/management tasks, such as data cleanup or computing analytics, should run as one-off processes.
- Independent tools should be used to invoke these tasks from the production environment, but separately from the application.



Traditional app design would isolate and treat administrative tasks differently than other tasks. The twelve factors do not recommend this approach.



According to the twelve factors, modern applications should leverage an event-driven architecture, where administrative tasks are invoked and run as any other process.

Additional factors for modern cloud-native apps

There are additional factors that merit mentioning when it comes to cloud native app design and development. They are:

- API First: Make everything a service. Assume your code will be consumed by a front-end client, gateway, or another service.

- Telemetry: On a workstation, you have deep visibility into your application and its behavior. In the cloud, you don't. Make sure your design includes the collection of monitoring, domain-

specific, and health/system data.

- Authentication/ Authorization: Implement identity from the start. Consider role-based access control (RBAC) features available in public clouds.

This is another general-purpose page layout. Unlike the Basic page, this one includes samples of some commonly used page elements.

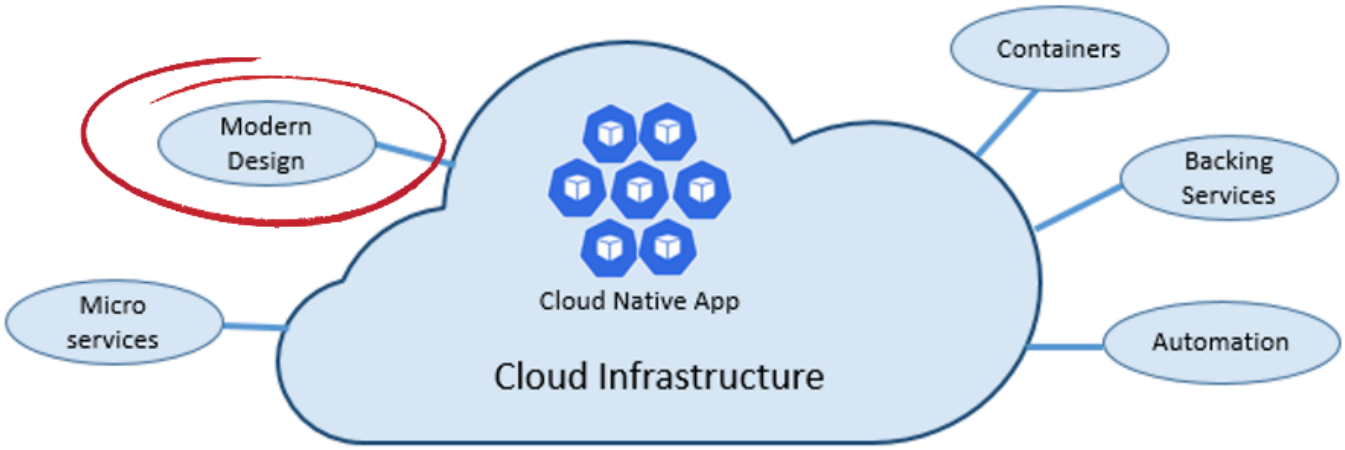
You can copy one, or more, of the specific elements from this page and paste them into other pages. By combining elements, you can create new layouts for your courses.

The pillars of cloud native

You will recall from the previous lesson that cloud native applications are based on pillars, the foremost of which is the use of cloud infrastructure. Other pillars include:

- Modern design
- Microservices
- Containerization
- Backing services
- Automation.

This lesson will focus on the “modern design” pillar, by providing an in-depth analysis of the twelve factor of cloud native application design.



Modern design

Consider the following questions:

- How would you design a cloud-native app?
- What would your architecture look like?
- To what principles, patterns, and best practices would you adhere?
- What infrastructure and operational concerns would be important?

These questions are fundamental to establish a design strategy that can “make or break” the application. As we examined in the previous lesson, the traditional monolithic design is not a good fit for an application that needs to scale rapidly and meet increased requirements for efficiency and delivery.

So how to proceed? In comes the wisdom, knowledge, and experience of industry leaders in cloud computing in the form of the “Twelve-Factor Application”. This forms the basis for the modern design of cloud-native applications.

The following blog post can serve as a quick introduction to the 12-factor app methodology: **(https://www.bmc.com/blogs/twelve-factor-app/)**

THE 12-FACTOR APP METHODOLOGY EXPLAINED (HTTPS://WWW.BMC.COM/BLOGS/TWELVE-FACTOR-APP/)

Twelve-factor application

The Twelve-Factor Application is a widely accepted methodology for constructing cloud-based applications. It is a set of principles and practices to construct applications optimized for modern cloud environments. Special attention is given to portability across environments and declarative automation.

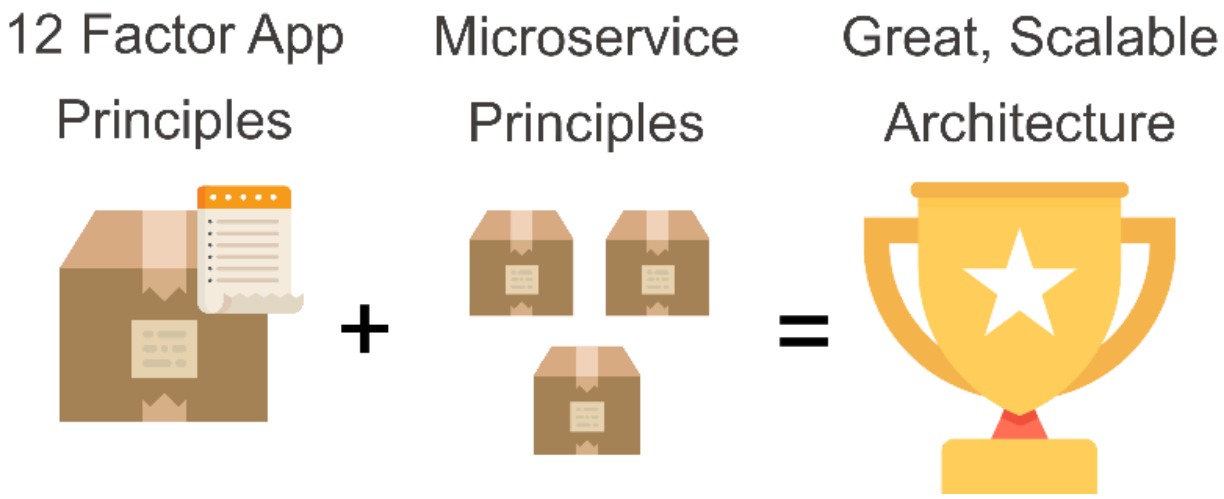
While applicable to any web-based application, many practitioners consider Twelve-Factor a solid foundation for building cloud-native apps. Systems built upon these principles can deploy and scale rapidly and add features to react quickly to market changes.

Single Codebase	Explicit Dependencies	Externally Managed Configuration	Backing Services
Build, release, run	Processes	Port Binding	Concurrency
Disposability	Dev/Prod Parity	Log as event streams	Admin Processes

---

The Twelve-Factor application + Microservice

As you will recall, cloud-native applications shy away from a heavy, clunky, monolithic server design and instead rely on microservices that are easier to design, maintain, and scale.



In this lesson, we will examine each of the 12 factors that compose this modern design paradigm.

In this module, we revisit the industry-recognized twelve-factor approach to cloud-native application development that were briefly introduced in the previous lesson. The twelve factors are a set of principles and practices to construct applications optimized for modern cloud environments. This lesson will describe the twelve-factor methodology.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Describe the Twelve-Factor methodology.

02 Explain factors that reflect today's modern cloud application design.

03 Identify principles, patterns, and best practices for cloud-native app design.

04 Discuss infrastructure and operational concerns.

Key Terms and Concepts

Key Term or Concept	Definition
Continuous integration and deployment/delivery (CI/CD) pipeline	A continuous integration and continuous deployment (**CI/CD (https://www.redhat.com/en/topics/devops/what-is-ci-cd)**) pipeline is a series of steps that must be performed in order to deliver a new version of software. CI/CD pipelines are a practice focused on improving software delivery throughout the software development life cycle via automation.

Key Term or Concept	Definition
Quality Assurance (QA)	Quality assurance is the term used in both manufacturing and service industries to describe the systematic efforts taken to assure that the product delivered to customer meet with the contractual and other agreed upon performance, design, reliability, and maintainability expectations of that customer.
Role-based access control	In computer systems security, role-based access control or role-based security is an approach to restricting system access to authorized users, and to implementing mandatory access control or discretionary access control. Role-based access control is a policy-neutral access-control mechanism defined around roles and privileges

In this lesson, we covered the microservice pillar of cloud native. You learned about the key concepts of Microservices, its principles, advantages, challenges, and examples. In the next lesson, you will learn more about containers.

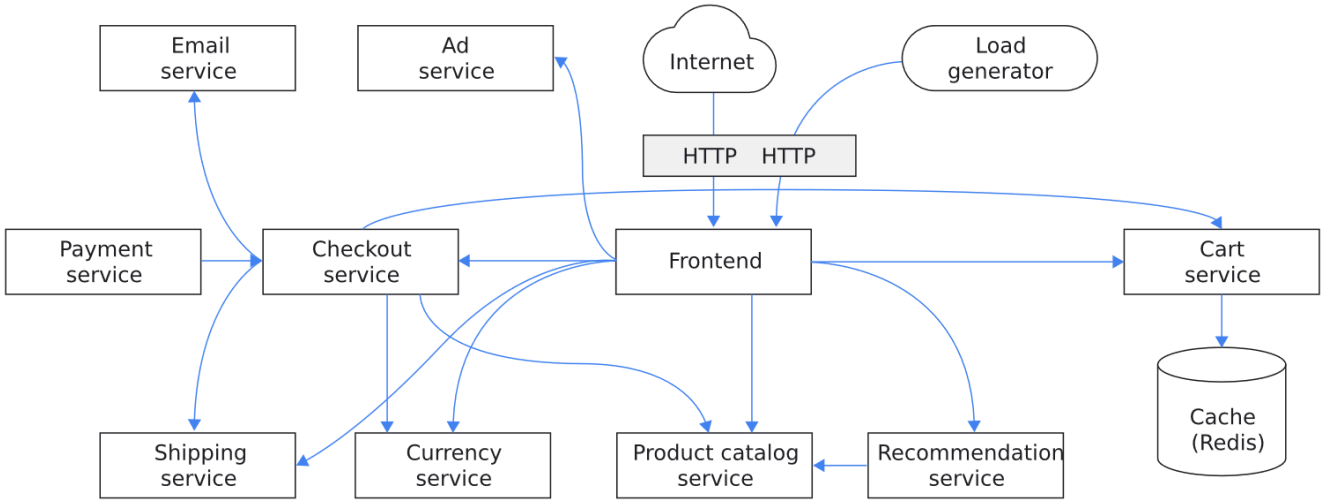
References

The references listed here, coupled with the course developer's experience, have been used in the development of this module:

Reselman, Bob (2022). 5 design principles for microservices. Red Hat Developer. Retrieved from **https://developers.redhat.com/articles/2022/01/11/5-design-principles-microservices (https://developers.redhat.com/articles/2022/01/11/5-design-principles-microservices)**#

Learning Activity

Consider an e-commerce application designed as independent microservices.



Consider the application as an event-driven architecture and consider the functionalities that are represented by each microservice in the application diagram. Reflect on the following questions:

1. What does the “frontend” service represent?

Click to see the answer.

2. What is the event that triggers the cart service?

Click to see the answer.

3. What happens if payment service is not available? Where and when will the application suffer problems?

Click to see the answer.

H5P Reuse I am confused

Advantages of microservices

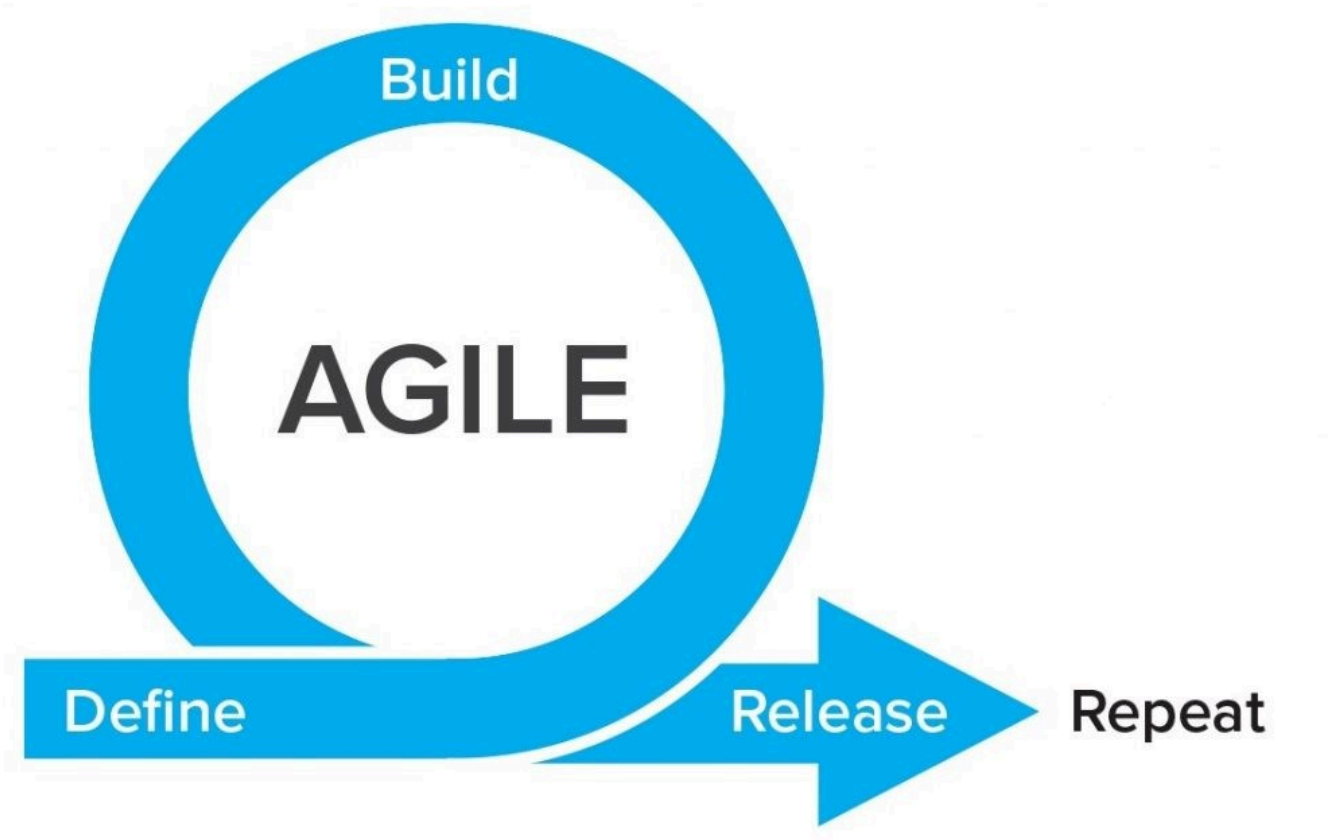
The main advantage of microservices is arguably the agility that they provide at almost each step of the way. You can update a small area of a live application with less risk of disrupting the entire system. The update can be made without a full redeployment of the application.

Fine-grained scaling provides for greater control of your system and helps reduce overall costs as you scale portions of your system, instead of updating an entire monolithic application.

The agility of microservices can be expressed as such:

- Each microservice has an autonomous lifecycle and can evolve independently and deploy frequently. You don't have to wait for a quarterly release to deploy a new features or update.

- Each microservice can scale independently. Instead of scaling the entire application as a single unit, you scale out only those services that require more processing power to meet desired performance levels and service-level agreements.



Challenges of microservices

There are of course challenges to the adoption of a fragmented application design that uses independent microservices instead of a straightforward multifunctional monolithic design. These challenges are essentially:

- Communication
- Resiliency
- Distributed Data
- Secrets

Let's explore each of these challenges.

Communication challenge

Some of the questions that arise when it comes to the communication between microservices include:

- How will front-end client applications communicate with back-end core microservices? Will you allow direct communication? Or might you abstract the back end microservices with a gateway facade that provides flexibility, control, and security?
- How will back-end core microservices communicate with each other? Will you allow direct HTTP calls that can increase coupling and impact performance and agility? Or might you consider decoupled messaging with queue and topic technologies?

This challenge and its potential solutions will be explored in later modules.

Resiliency

A microservices architecture moves your system from in-process to out-of-process network communication. In a distributed architecture, what happens when Service B isn't responding to a network call from Service A? Or, what happens when Service C becomes temporarily unavailable and other services calling it become blocked?

This challenge and its potential solutions will be explored in later modules.

Distributed data

By design, each microservice encapsulates its own data, exposing operations via its public interface. If so, how do you query data or implement a transaction across multiple services?

This challenge and its potential solutions will be explored in later modules.

Secrets

The term “secret” represents any type of data, including metadata, that must be protected against any unauthorized access. This can present a unique challenge in a distributed architecture of independently run and managed microservices. At the heart of this challenge is the question:

- How will your microservices securely store and manage secrets and sensitive configuration data?

This challenge and its potential solutions will be explored in later modules.

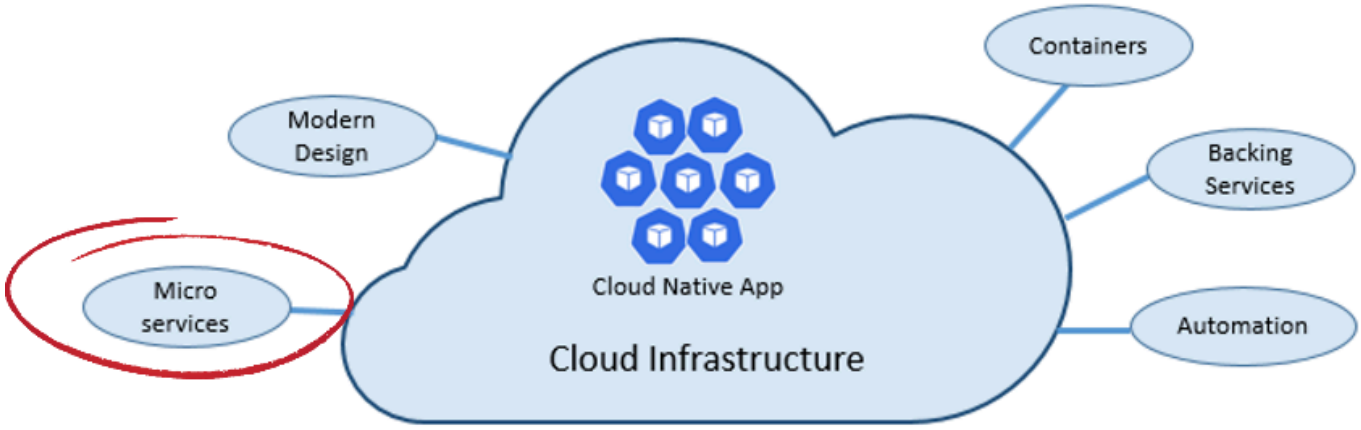
The pillars of cloud native

You will recall from the previous lessons that cloud native applications are based on pillars, the foremost of which is the use of cloud infrastructure.

Other pillars include:

- Modern design
- Microservices
- Containerization
- Backing services
- Automation.

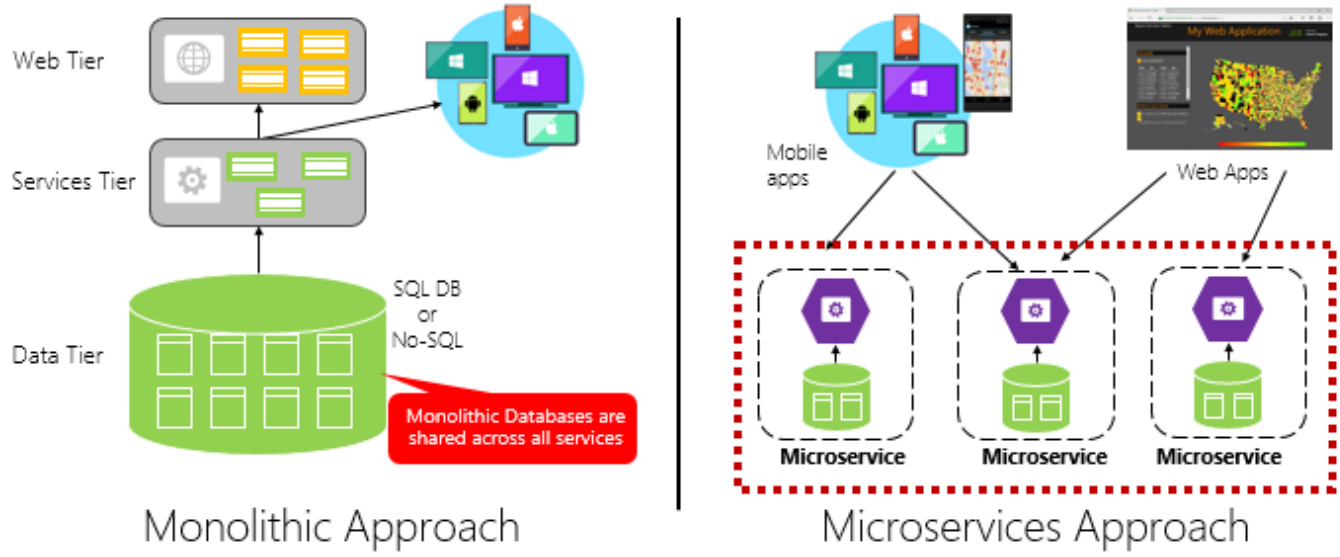
This lesson will focus on the “microservices” pillar of cloud native application design.



Microservices vs Monolithic Architecture

We have already covered the differences between the traditional, monolithic application design and the cloud-native approach to using microservices in the first lesson. You will recall that the monolithic approach is a designation for applications that are self-contained and distributed as a single program. Web applications are not specifically built as such, since there are typically three tiers that serve to create a layered application. The tiers are the data tier, the services tier, also called the business logic, and the presentation tier, typically provided via web interfaces that are compatible with several web clients. This design consumes a relational database that services the entire application with normalized datasets.

The microservice approach, however, segregates functionality into independent services, each with its own logic, state, and data. Each microservice hosts its own datastore.

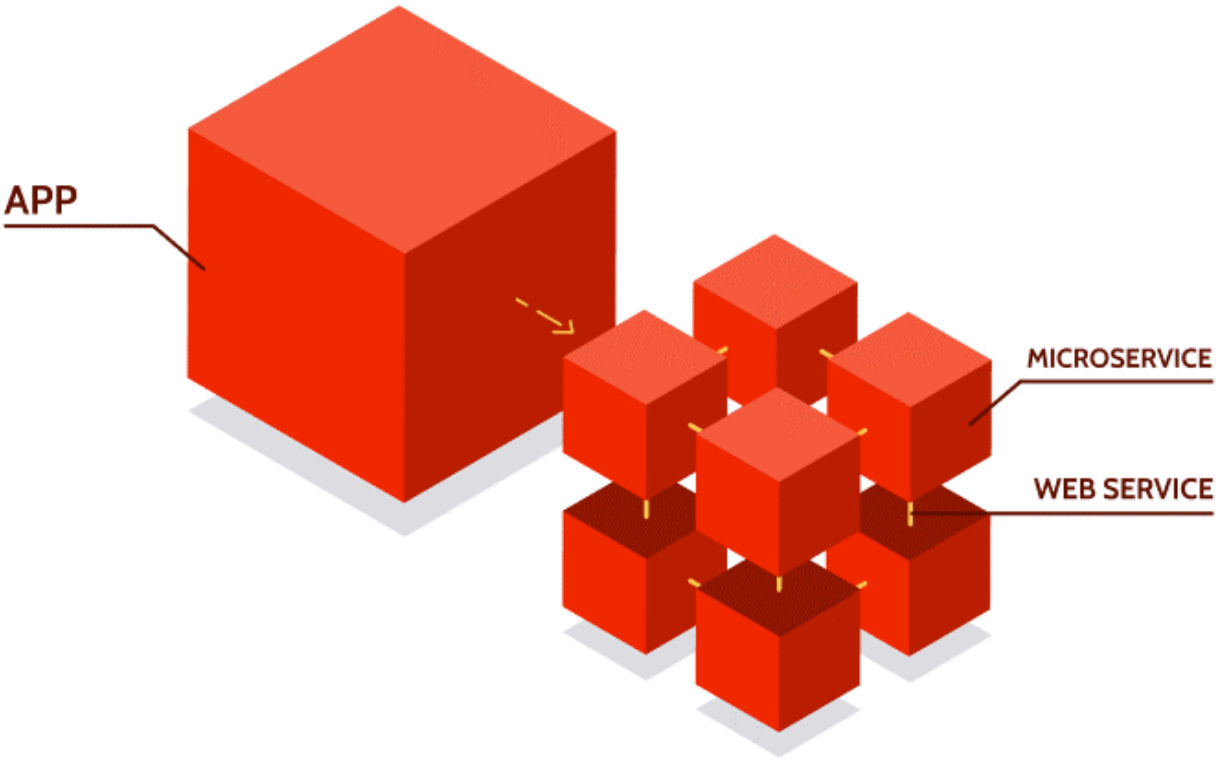


Definition of a microservice

Let’s dig deeper into the concept of a microservice by looking at a complete definition of the concept. Consider the following attributes associated to a microservice:

- A microservice typically implements a set of distinct features or functionality.
- Each microservice is a mini-application that has its own architecture and business logic.

For example, some microservices expose an API that's consumed by other microservices or by the application's clients, such as third-party integrations with payment gateways and logistics.



Microservices Principles

Let's look at the principles of microservices. The principles that you will study are the following:

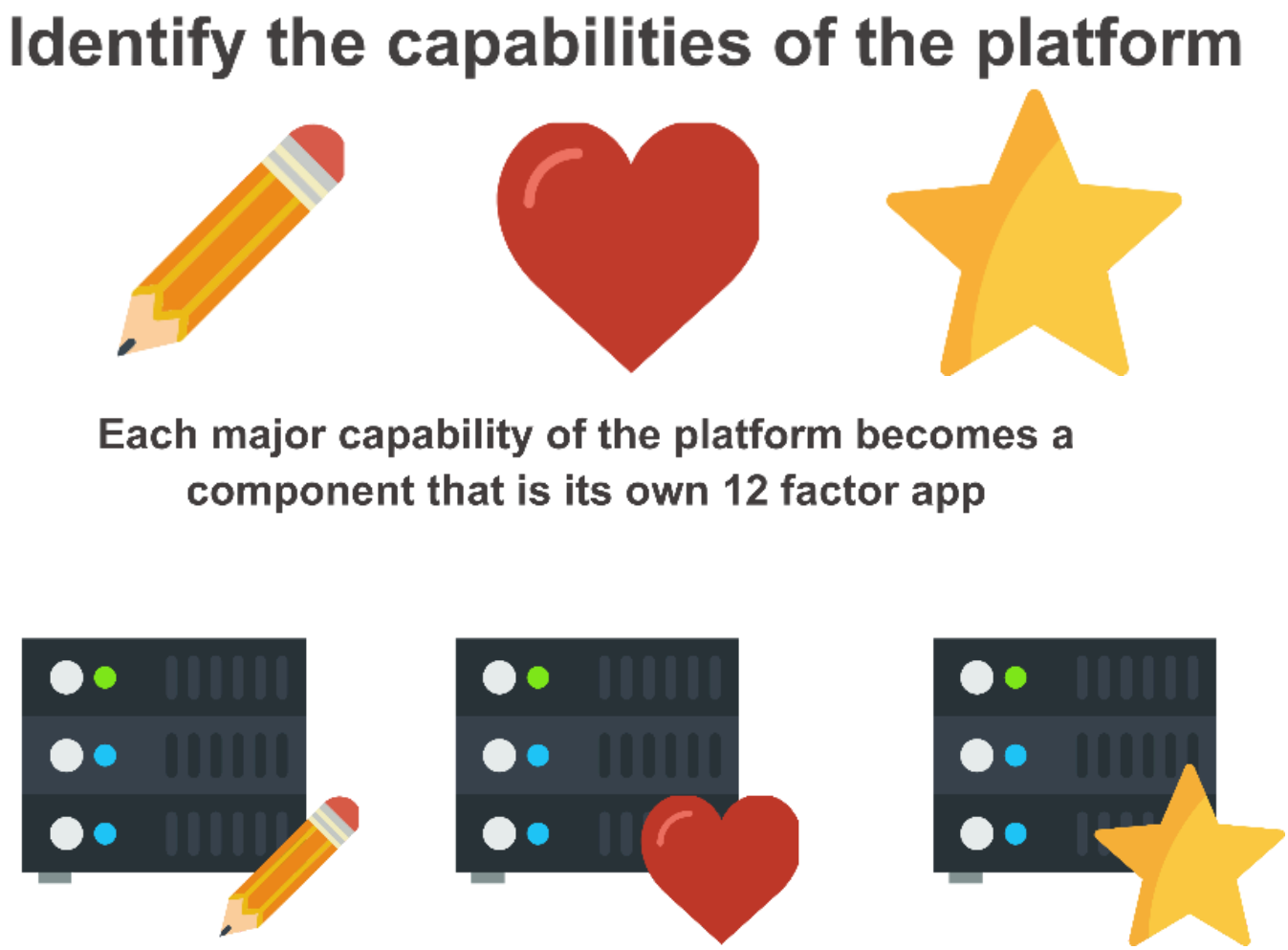
- Componentization
- Product focused
- Decentralization
- Automation

Microservices principles: Componentization

This principle revolves around this statement: "Each microservice is a mini-application that is itself designed and developed in accordance with the 12-factor methodology."

This understanding is at the heart of microservice design. A microservice must make sense on its own, provide a full service or a complete functionality, independently from the applications that will consume its services.

This answers one of the fundamental questions around the design of the application. According to this principle, each major capability of the platform becomes a component that is its own 12-factor app.



Microservices principles: Product focused

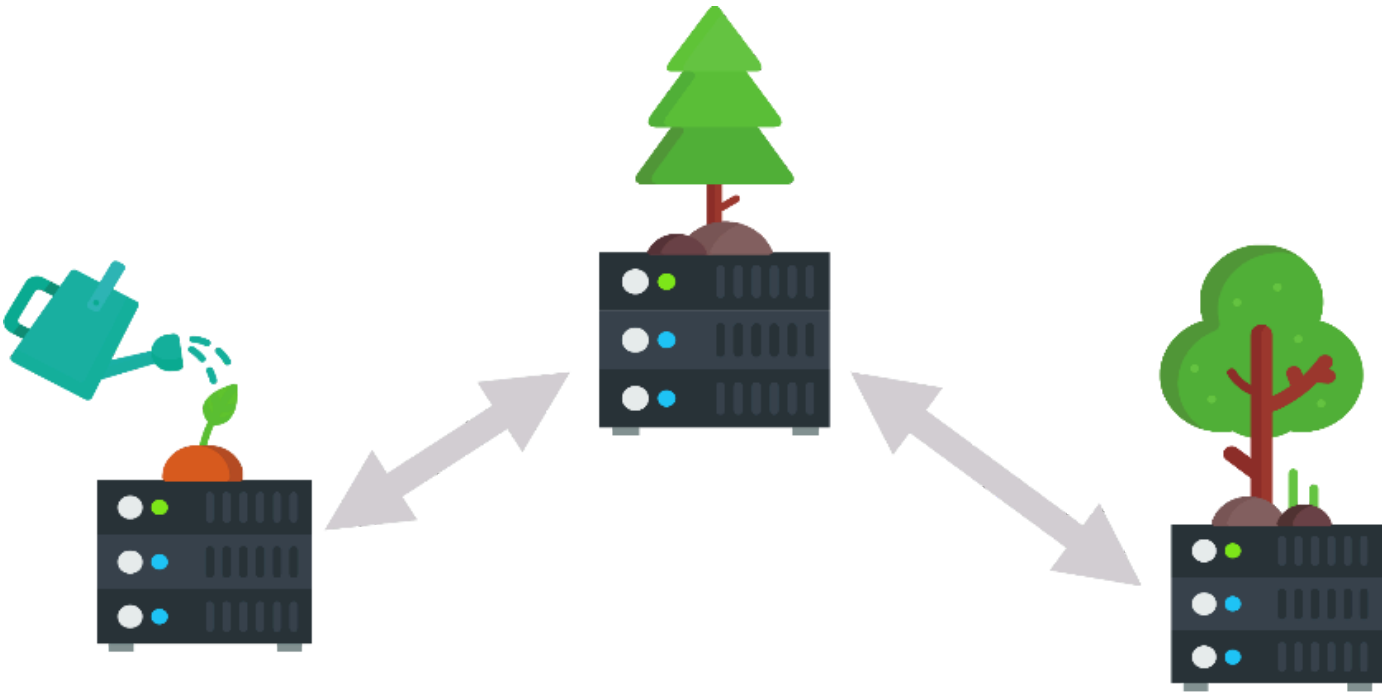
This principle states that:

- Microservices are products not projects.
- Products grow over time.
- Microservices are an ecosystem of connected products that can be at different stages of growth.

It reinforces the notion that the components are not designed to be a limb of the application, but to act on their own, evolve, and meet a functionality.
Recall that this approach allows local services to be treated as third-party services because of this independence between the components.



Monolithic application evolve as a single block, slowing process and increasing the risk of failure due to the intricate nature of interconnecting services that lack autonomy.

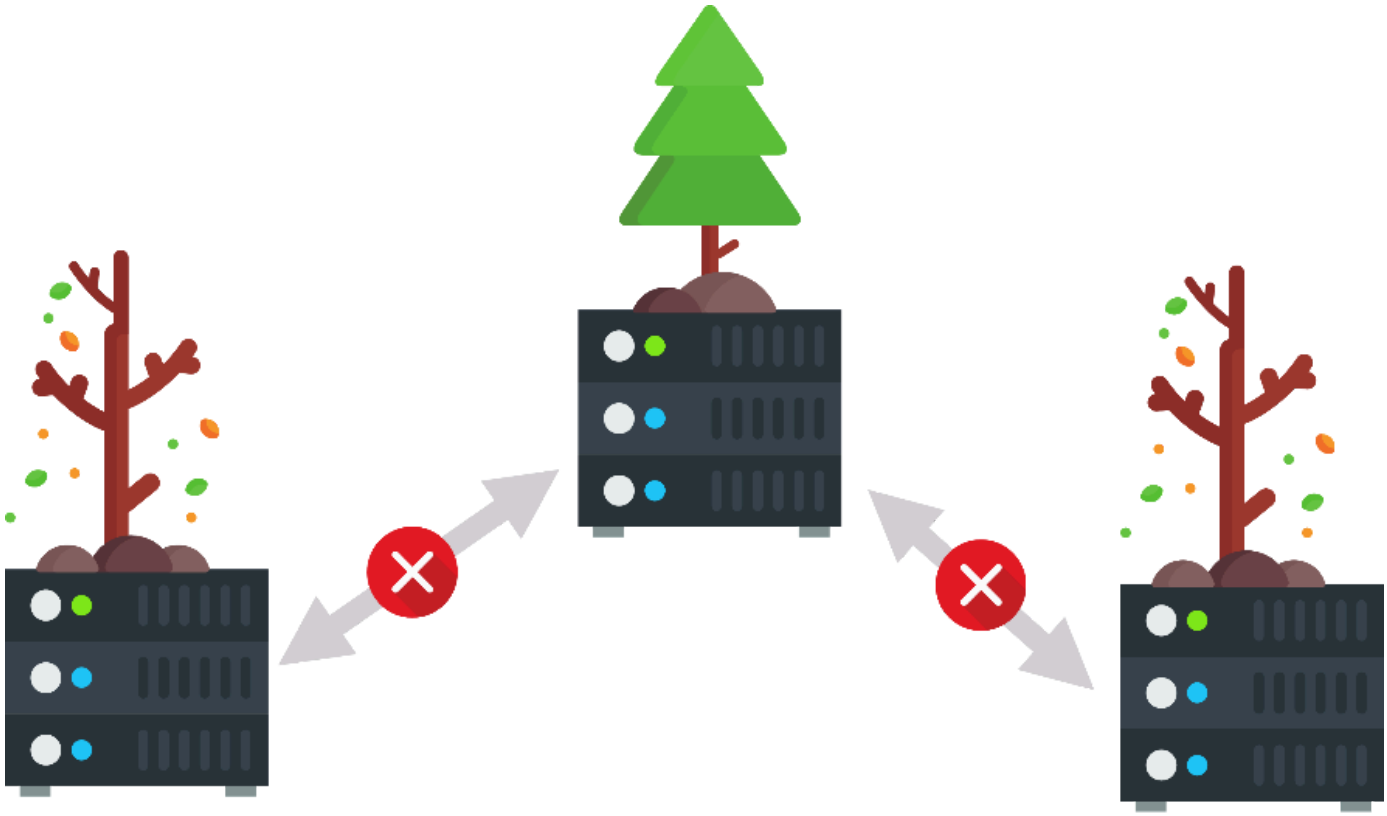


In a microservice approach, each microservice evolves and adapts on its own. The loose coupling between the application components reduces risks of breaking the compatibility between the components when updates are implemented to the different functionalities.

This principle also states:

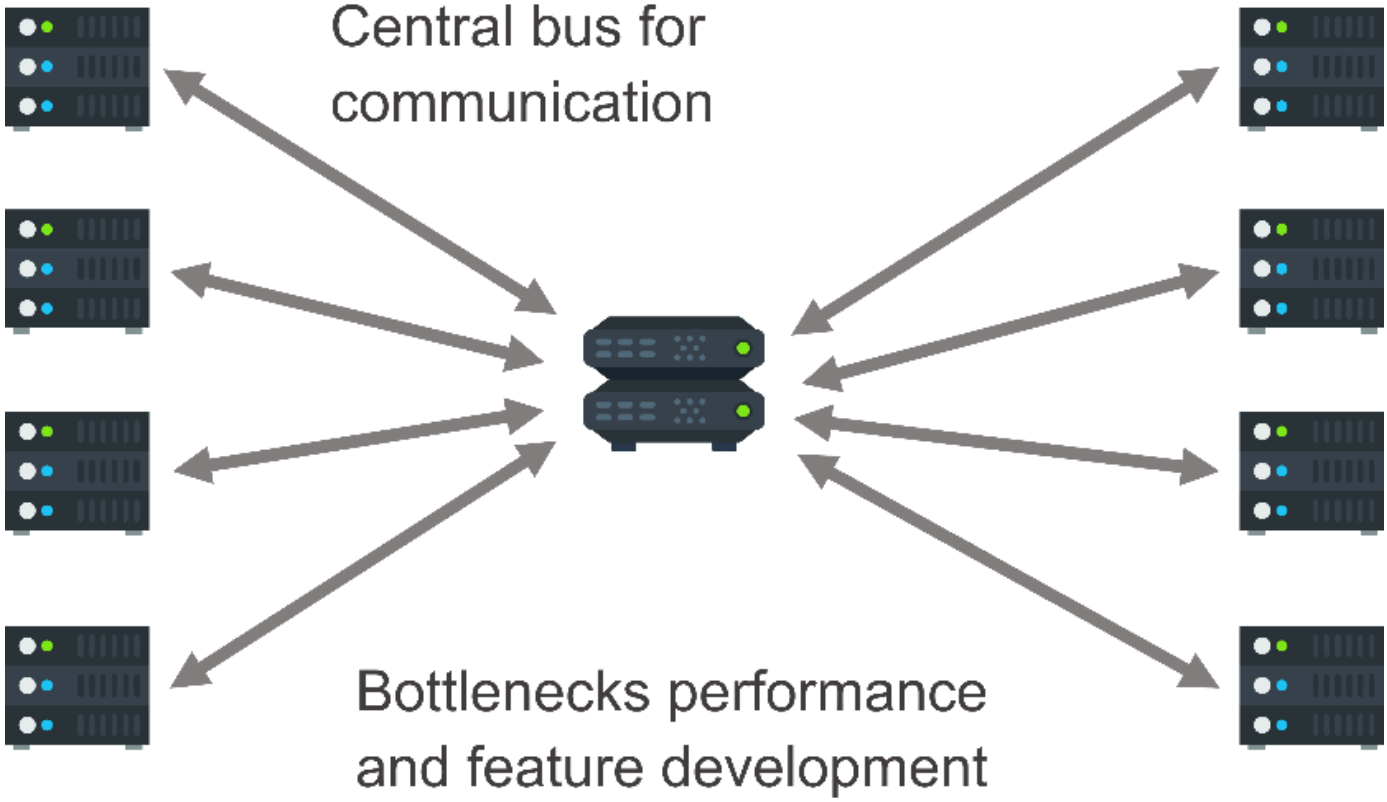
- Don’t create throwaway microservices that become unmaintained and break the ecosystem.

A throwaway service can be a temporary implementation that allows the application to provide its functionality. All microservices must be maintained and kept in running order, otherwise they could corrupt data, breach the integrity of the services, and jeopardize the application.



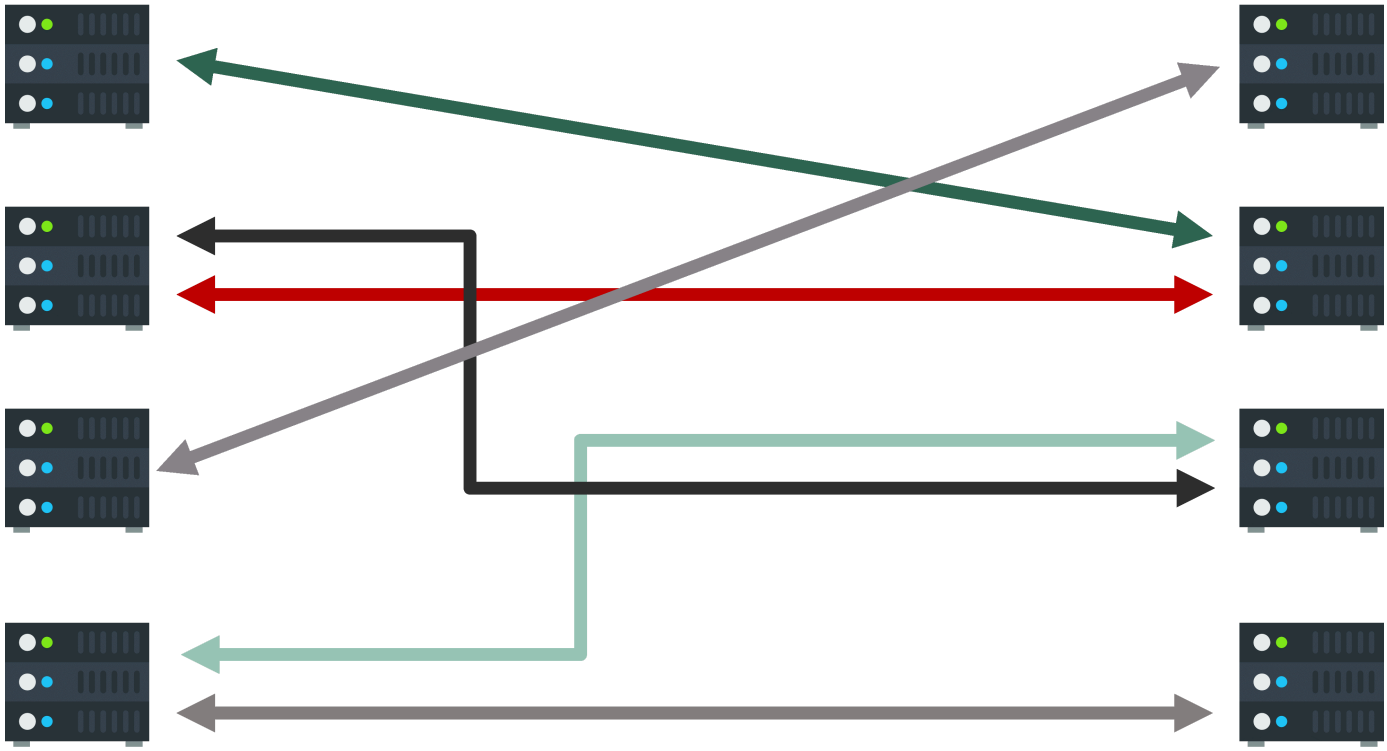
Microservices principles: decentralization

The dilemma of using a centralized or distributed infrastructure can be complicated to answer, as dilemmas typically are. Cloud-native app development favours a decentralized approach, in keeping with cloud computing that is essentially a distributed architecture with built-in redundancy and geographical dispersion. As such, one of the principles of microservices is to be decentralized.



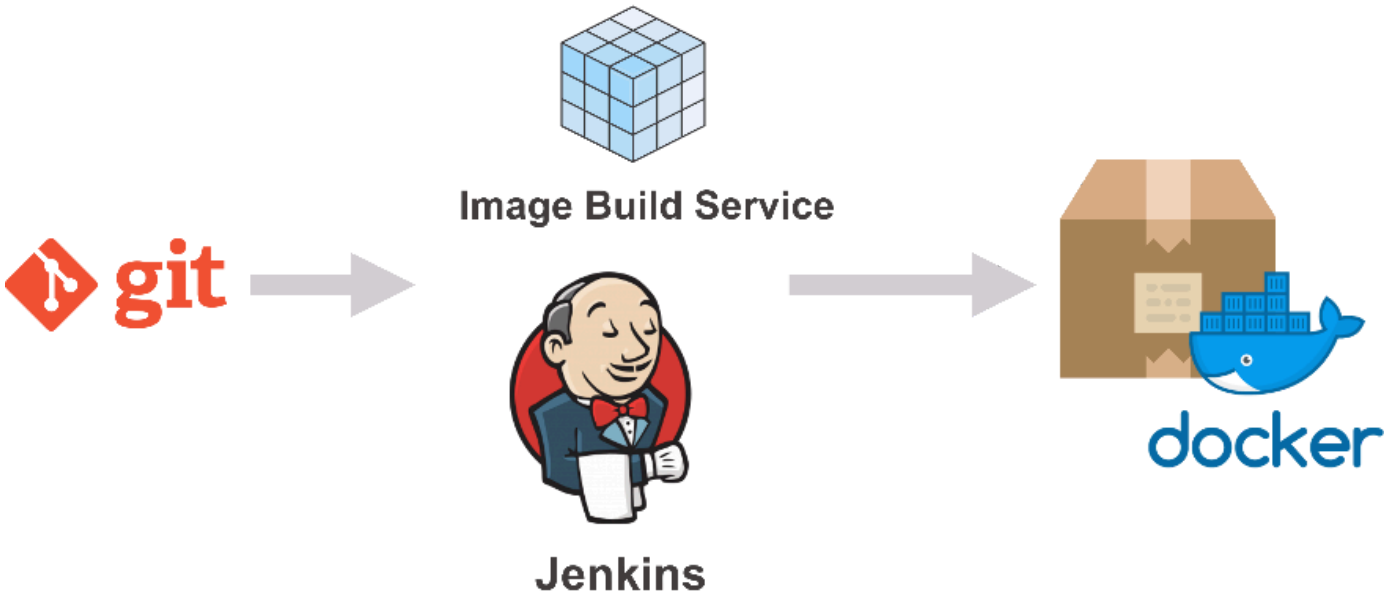
A centralized approach will use a central bus for communication, forcing the components to go through this potential single-point-of-failure. If the central bus is lacking resources, the only way to provision more in a centralized approach is vertical scaling. You will recall that the 12 factor methodology prones the horizontal scaling approach, in keeping with cloud.

A decentralized approach means that microservices can consume each other's resources, without relying on a centralized architecture. In fact, the design of a microservice should allow it to accept and fulfill requests from end users, other microservices, applications, or whichever process is querying its capability.



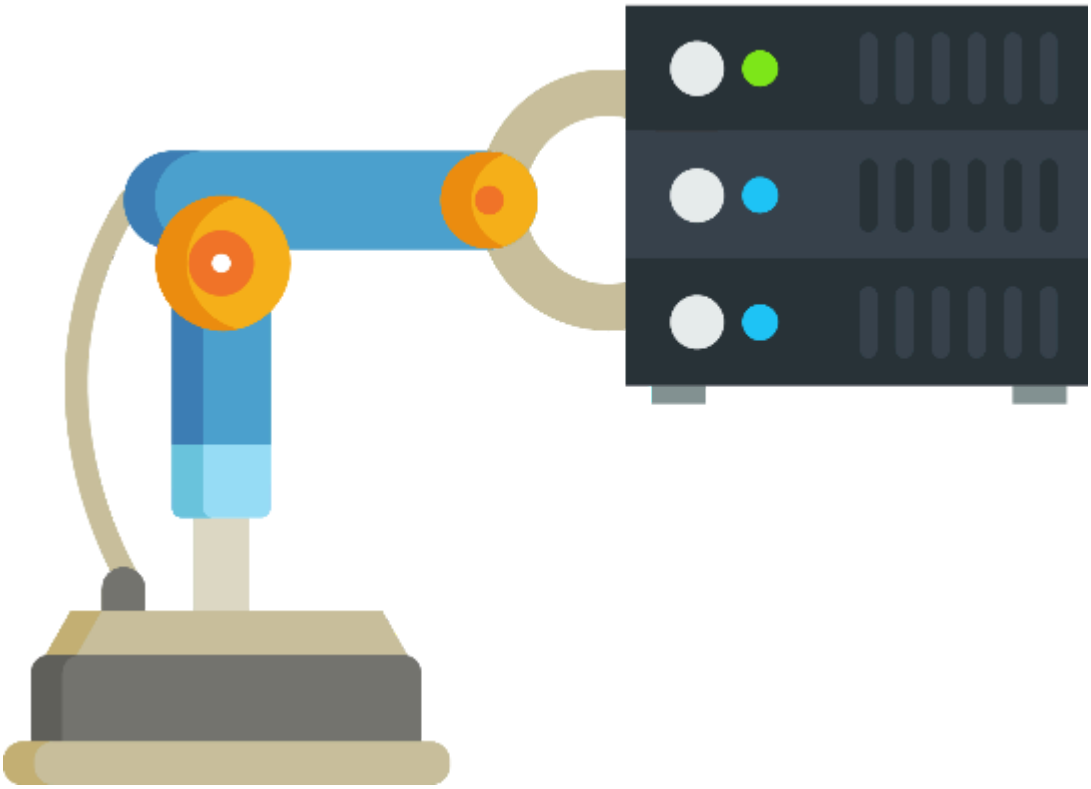
Microservices principles: automation

Automation is a key enabler of cloud computing technologies. It is embedded with the definition of cloud computing. Understandably, it is also a basic principle of microservices. Automation reduces the time to test, deploy, and configure application. Cloud-native application development uses automation at many different levels, including packaging and releasing applications to self-deployable containers.



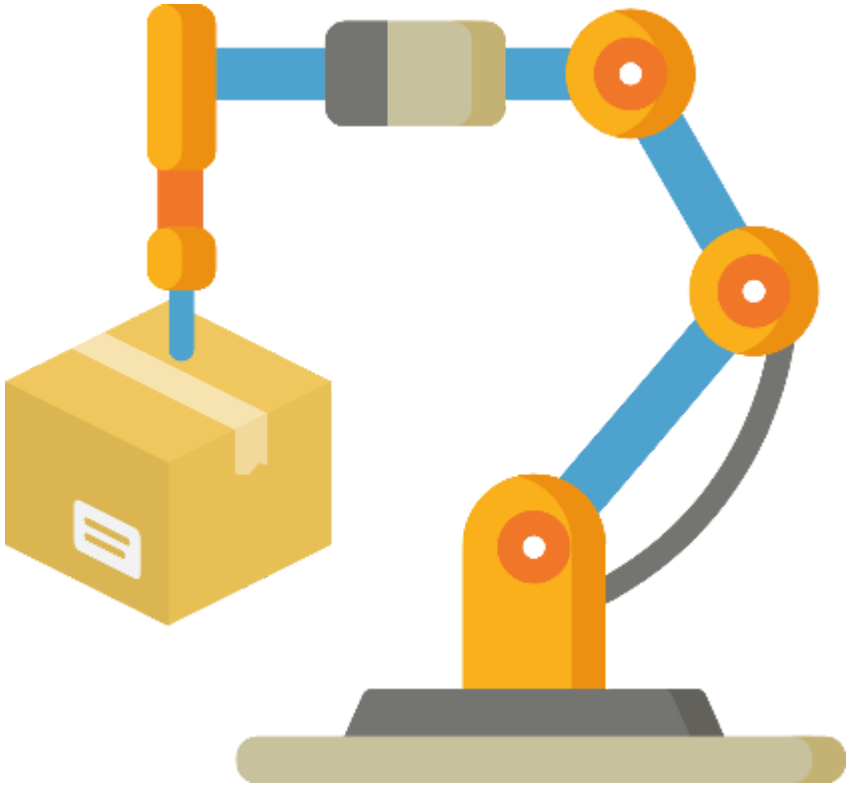
Among the items that can be automated is the provisioning of the servers that host microservice containers. The automation of infrastructure is a great tool to make uniform environments that do not rely on the host for their requirements and dependencies.

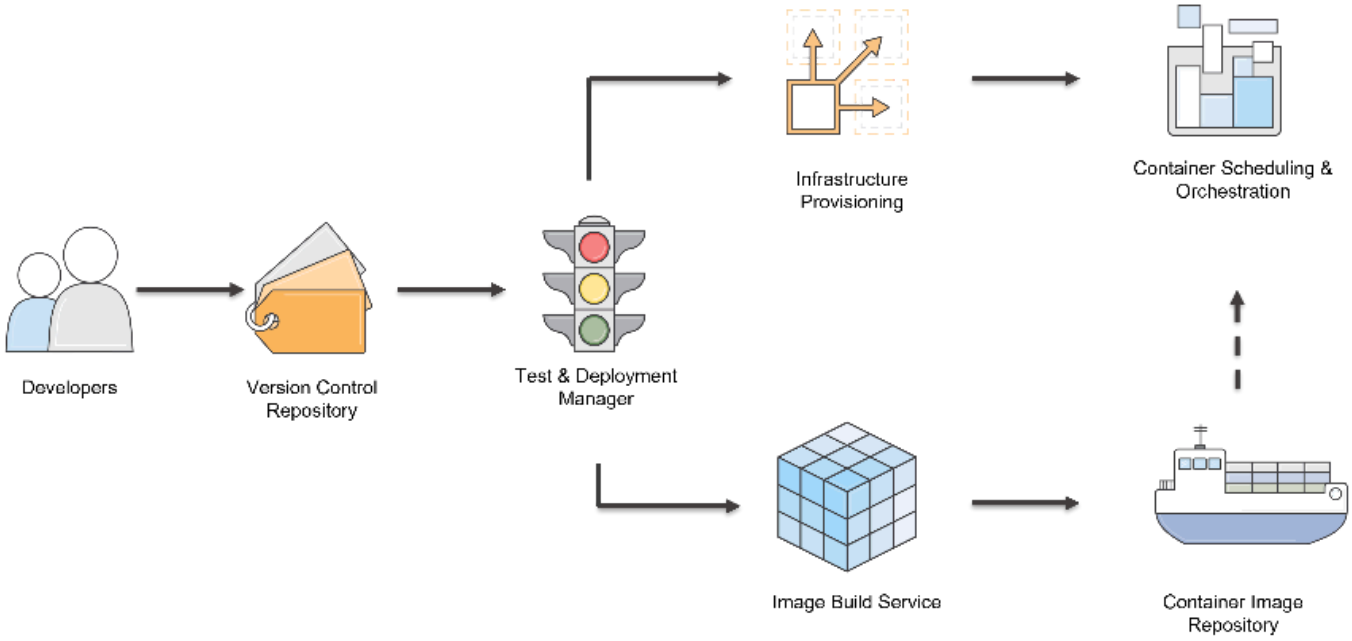
Tools that provide infrastructure automation include Terraform, Ansible, Amazon CloudFormation or Azure Resource Manager.



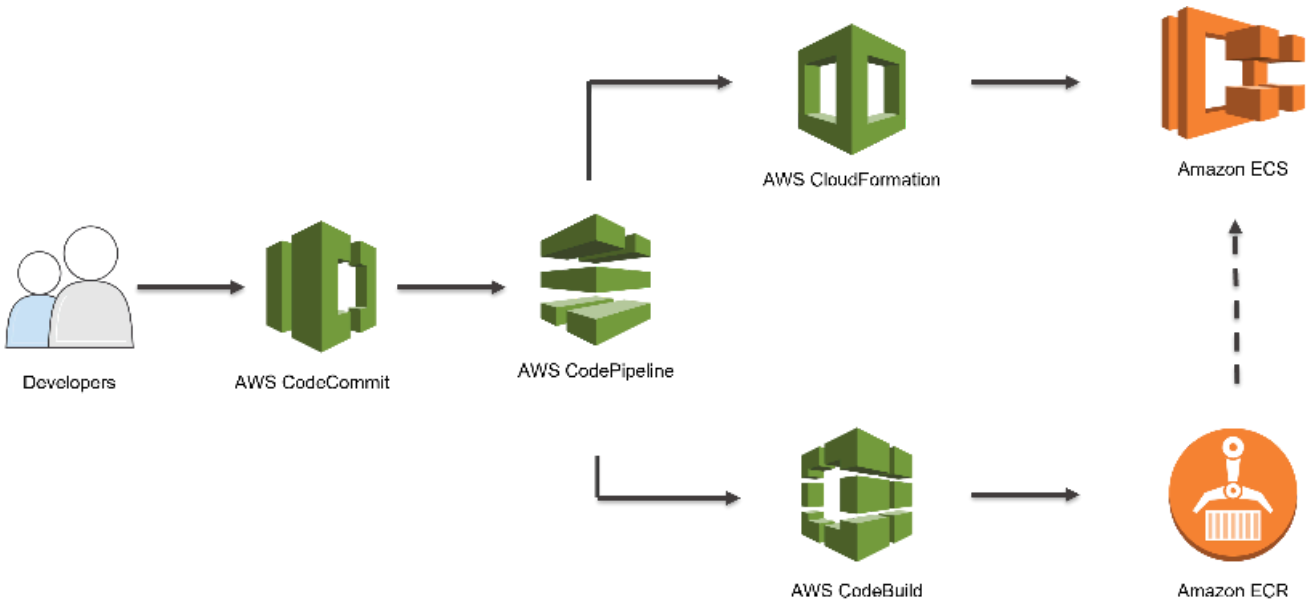
The placement of containerized service process onto hosts should be automated as well.

Tools that provide this automation include Amazon Elastic Container Service, Azure Container Apps, Kubernetes, Docker Swarm.

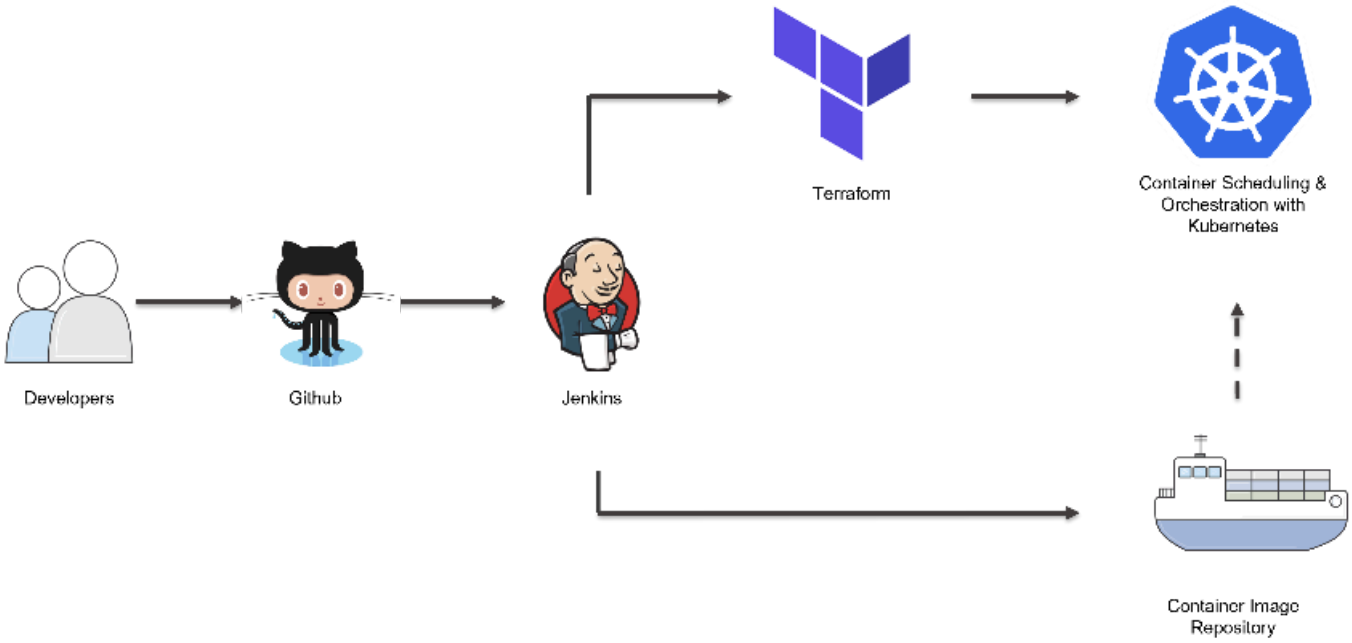




This diagram summarizes the process of releasing and testing a new version of the microservice, automating the provisioning of the infrastructure, the container scheduling, the build service, and the release of the container image.



This diagram proposes the same workflow, but with specific AWS tools that provideeach functionality.



An alternative diagram using different tools is presented here. Remember that in a true cloud-native development approach, applications should be vendor-agnostic.

Characteristics of microservices

To summarize what characterizes microservices, we can state that:

- Each microservice is developed autonomously and can be deployed independently.
- Each microservice is self-contained, encapsulating its own data storage technology, dependencies, and programming platform.
- Each microservice runs in its own process and communicates with others using standard communication protocols such as HTTP/HTTPS, gRPC, WebSockets, or AMQP.
- Microservices compose together to form an application.

For a slightly different perspective on the defining characteristics of microservices, read the following blog article: **(https://developers.redhat.com/articles/2022/01/11/5-design-principles-microservices)**

5 DESIGN PRINCIPLES FOR MICROSERVICES (HTTPS://DEVELOPERS.REDHAT.COM/ARTICLES/2022/01/11/5-DESIGN-PRINCIPLES-MICROSERVICES#)

In the previous lessons, you were introduced to the pillars of cloud-native application design and development. You were also introduced to the twelve-factor application design set of principles for modern app design. In this lesson, we will focus on microservices and how they can enable the trusted and resilient delivery of services to cloud applications.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Recall the contrast between microservices and monolithic applications.

02 Recall the key concepts of microservices.

03 States the principles of microservices.

04 Define the advantages and the challenges of microservices.

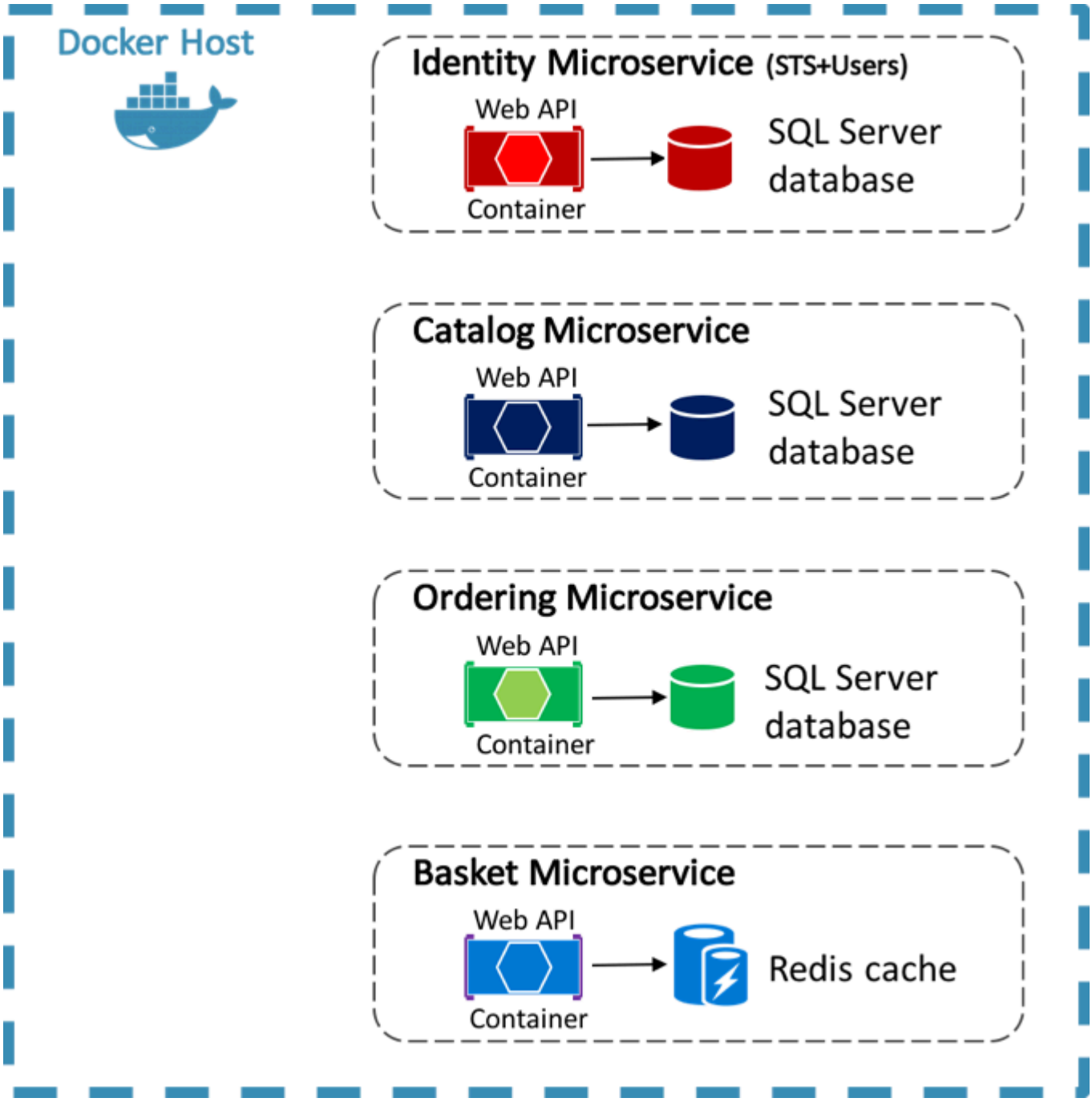
Key Terms and Concepts

Key Term or Concept	Definition
Microservices	An architectural and organizational approach to software development where software is composed of small independent services that communicate over well-defined APIs. Microservices architectures make applications easier to scale and faster to develop.

Key Term or Concept	Definition
Monolithic Applications	A self-contained, single unified software application which is independent from other applications. Multiple components are combined into one large application.

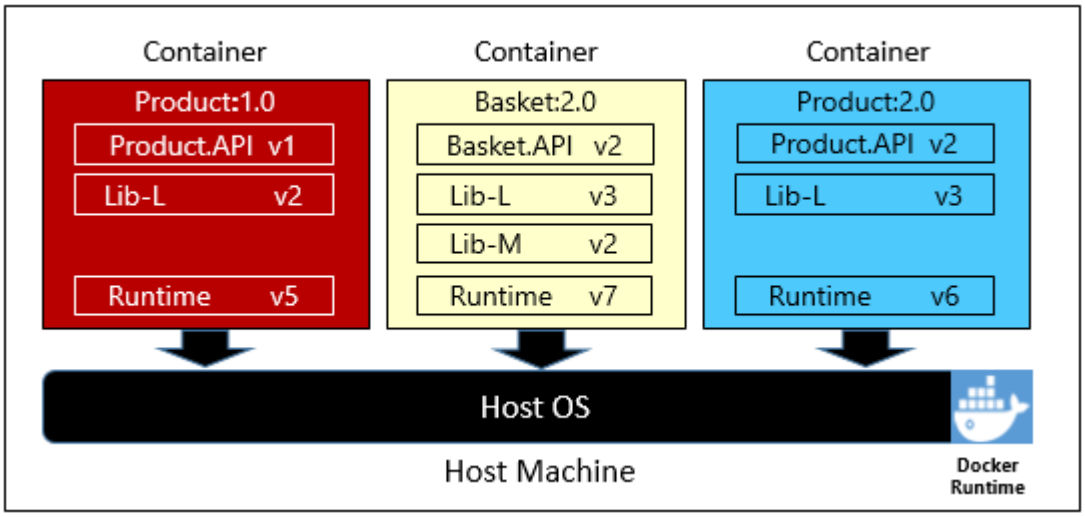
Containerized Microservice

Containerizing a microservice is simple and straightforward. The code, its dependencies, and runtime are packaged into a binary called a container image. Images are stored in a container registry, which acts as a repository or library for images. A registry can be located on your development computer, in your data center, or in a public cloud. Docker itself maintains a public registry via Docker Hub.



Running a containerized microservice

When an application starts or scales, the container image transforms into a running container instance. The instance runs on any computer that has a container runtime engine installed. You can have as many instances of the containerized service as needed.

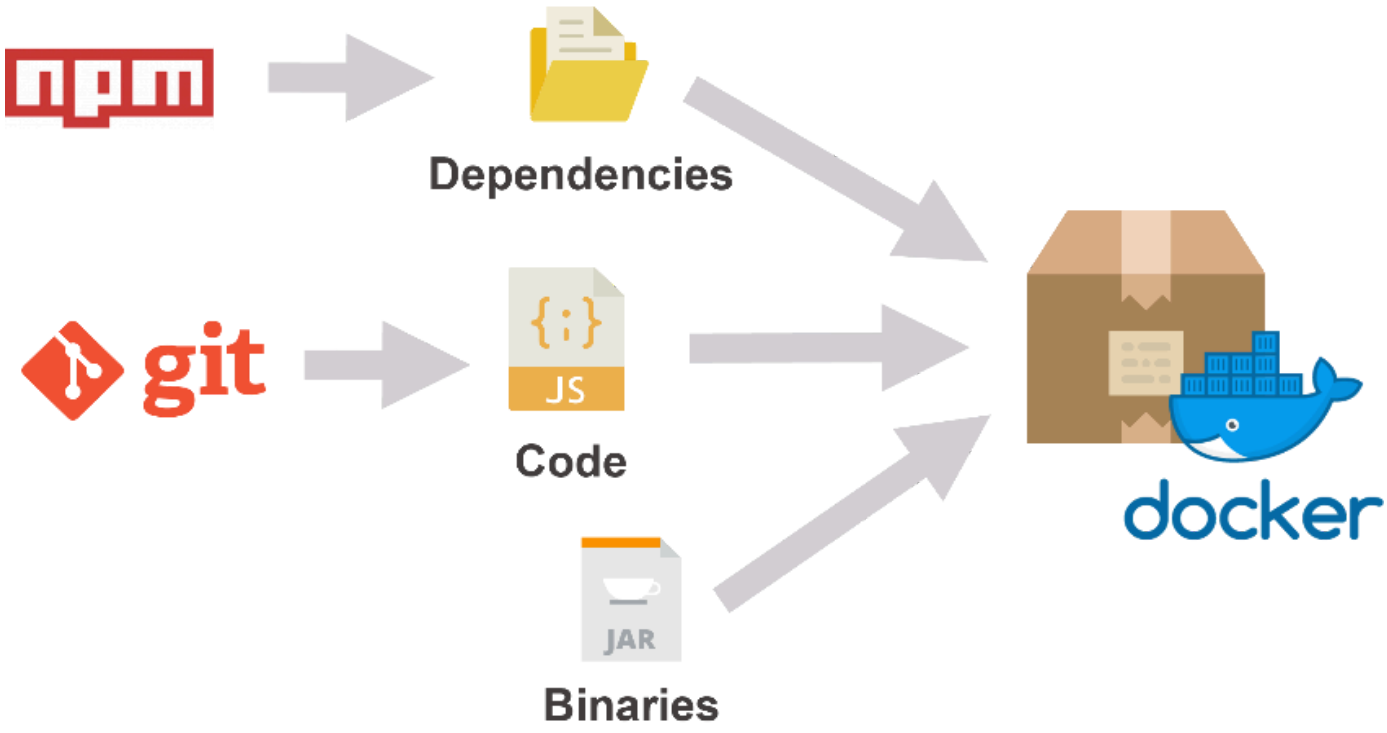


This diagram shows three different microservices, each in its own container, all running on a single host.

Container model

Notice how well the container model embraces the “Dependencies” principle from the Twelve-Factor Application that was covered in earlier lessons:

- Each microservice isolates and packages its own dependencies, embracing changes without impacting the entire system.
- The microservice never depends on the host to have access to a dependency.
- It allows application deployments to carry all their dependencies with them.



Containerization embodies the “Dependencies” principle of the Twelve-Factor Application methodology.

Docker terminology

Here is some of the terminology associated with the use of the Docker container engine. This terminology will become more familiar as you interact with the software in learning activities and labs.

Image: The basis of a Docker container. The content at rest.

Container: The image when it is ‘running.’ The standard unit for app service

Engine: The software that executes commands for containers. Networking and volumes are part of Engine. Can be clustered together.

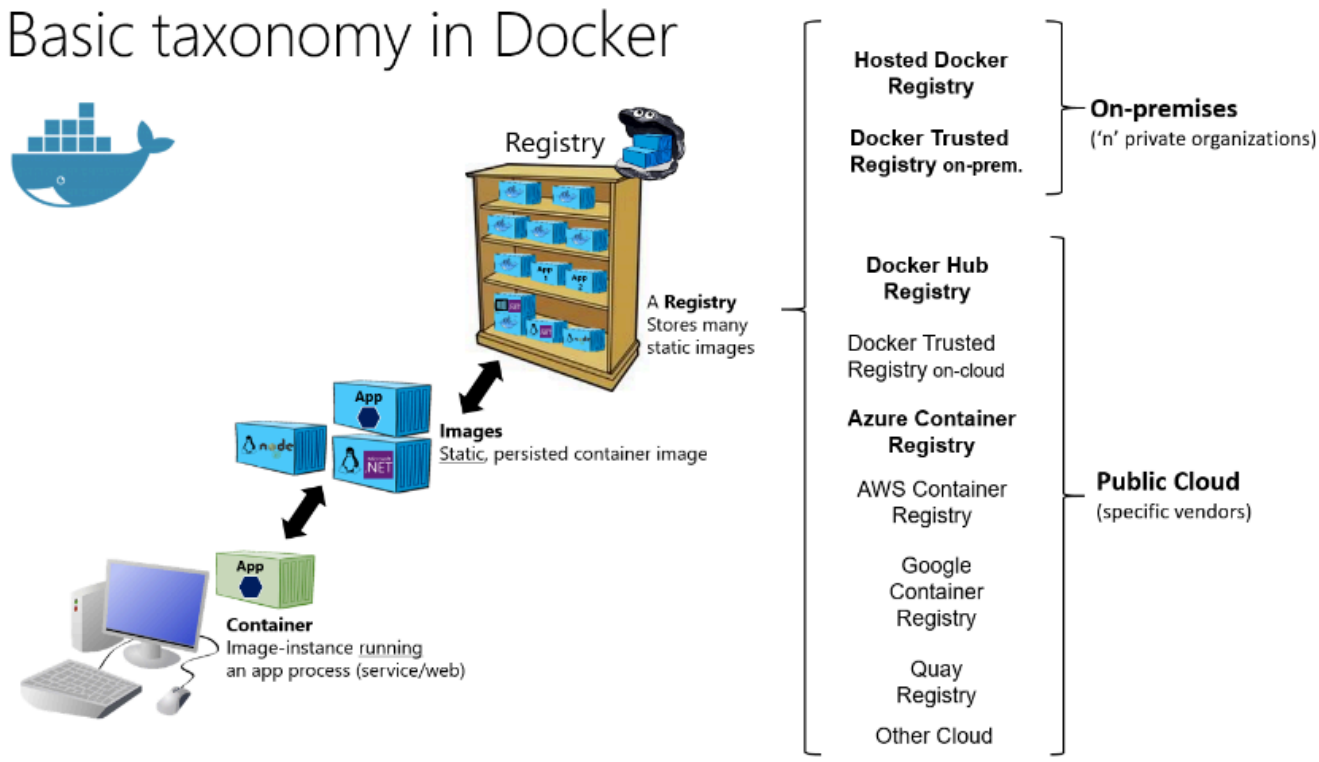
Registry: Stores, distributes and manages Docker images

Control Plane: Management plane for container and cluster orchestration

---

Basic taxonomy in Docker

When looking at a bigger picture, the elements of application packaging and containerization fall into place with the use of a cloud computing environment. The Docker Hub registry holds base images for public cloud vendors that will allow the packaged application to be run against the chosen vendor.



If this module felt overwhelming because you are new to Docker, do not worry, as you will be acquainted with the software and methodology in future lessons and labs. In the meantime, you can discover a bit more about the technology through this resource: **(https://www.howtogeek.com/733522/docker-for-beginners-everything-you-need-to-know/)**

DOCKER FOR BEGINNERS: EVERYTHING YOU NEED TO KNOW (HTTPS://WWW.HOWTOGEEK.COM/733522/DOCKER-FOR-BEGINNERS-EVERYTHING-YOU-NEED-TO-KNOW/)

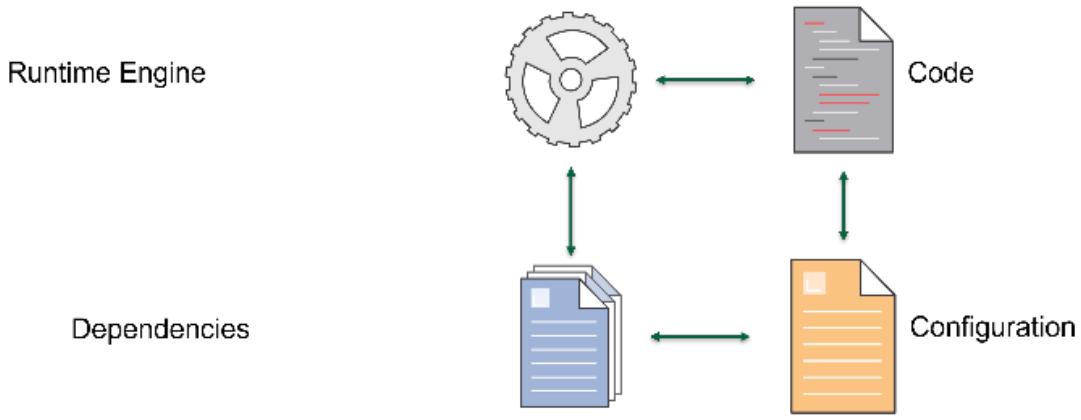
Using containerization

Now you are familiar with the above concepts and how containerization can help to optimize application packaging and deployment, without dedicating a complete operating system to the task. In this section, we will look at the mechanics of containerization.

Application Environment Components

There are four components to consider when packaging an application. These components are:

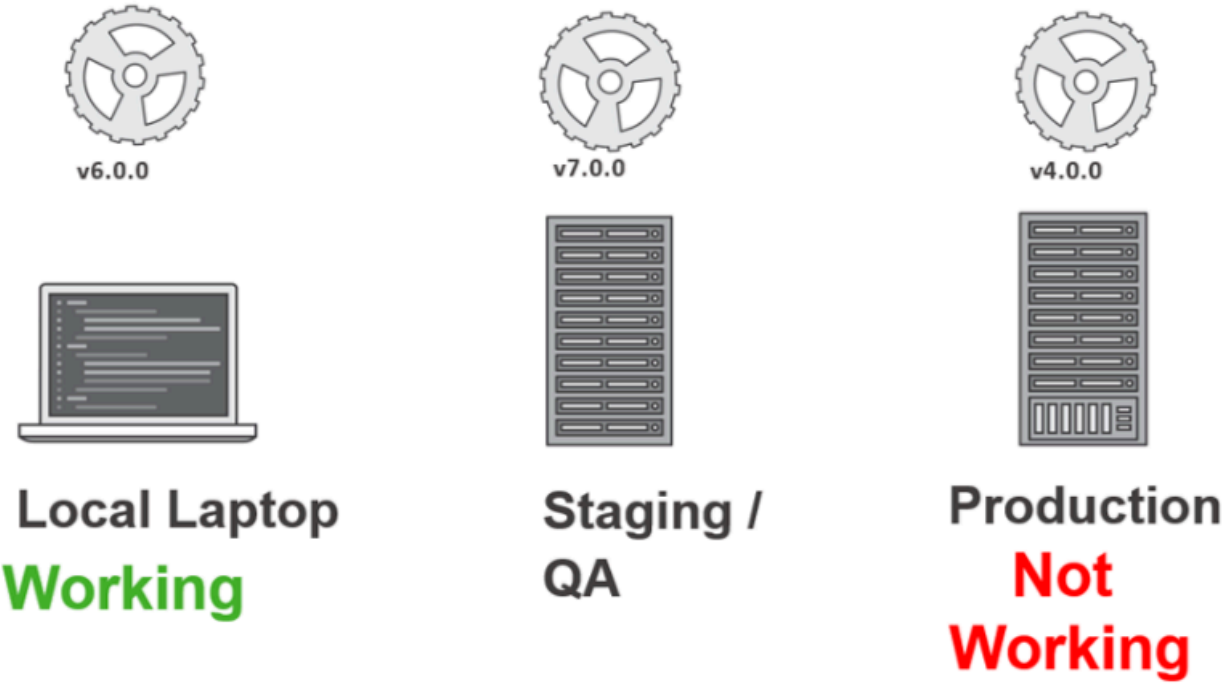
- Runtime engine: This is what will allow the code to run. Examples include the Java Runtime Environment and the Python interpreter.
- Code: The code that makes the application, which can be compiled or scripted for interpretation.
- Dependencies: There is almost no code today that run by itself without using some dependencies. These can be libraries, modules, or packages required by the application.
- Configuration: This defines the application in a declarative way. It provides metadata on the application that informs about its needs and expectations for the environment where it will run.



Migration woes

An all too familiar scenario for experienced developers is to have code running and working well in the development environment, only to fail when it is migrated to another setting. The culprit could be an out-of-date library, a system setting, a different environment variable, and anything in

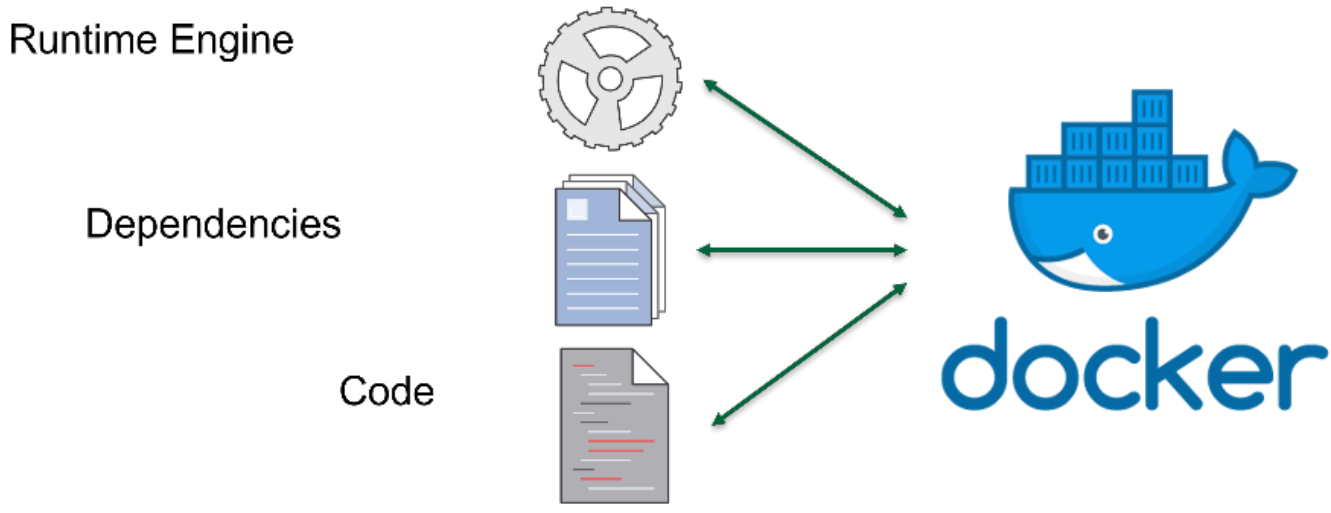
between. Early operating systems were much simpler and very similar from an environment to another. As operating system became increasingly complex, it is almost impossible to faithfully reproduce an environment without a massive effort.



A migration issue: a library used by the application is at version 6.0.0 on the development computer, version 7.0.0 in the staging environment, which could already cause issues, and at version 4.0 in the production environment, breaking down the application.

Containerization as a solution

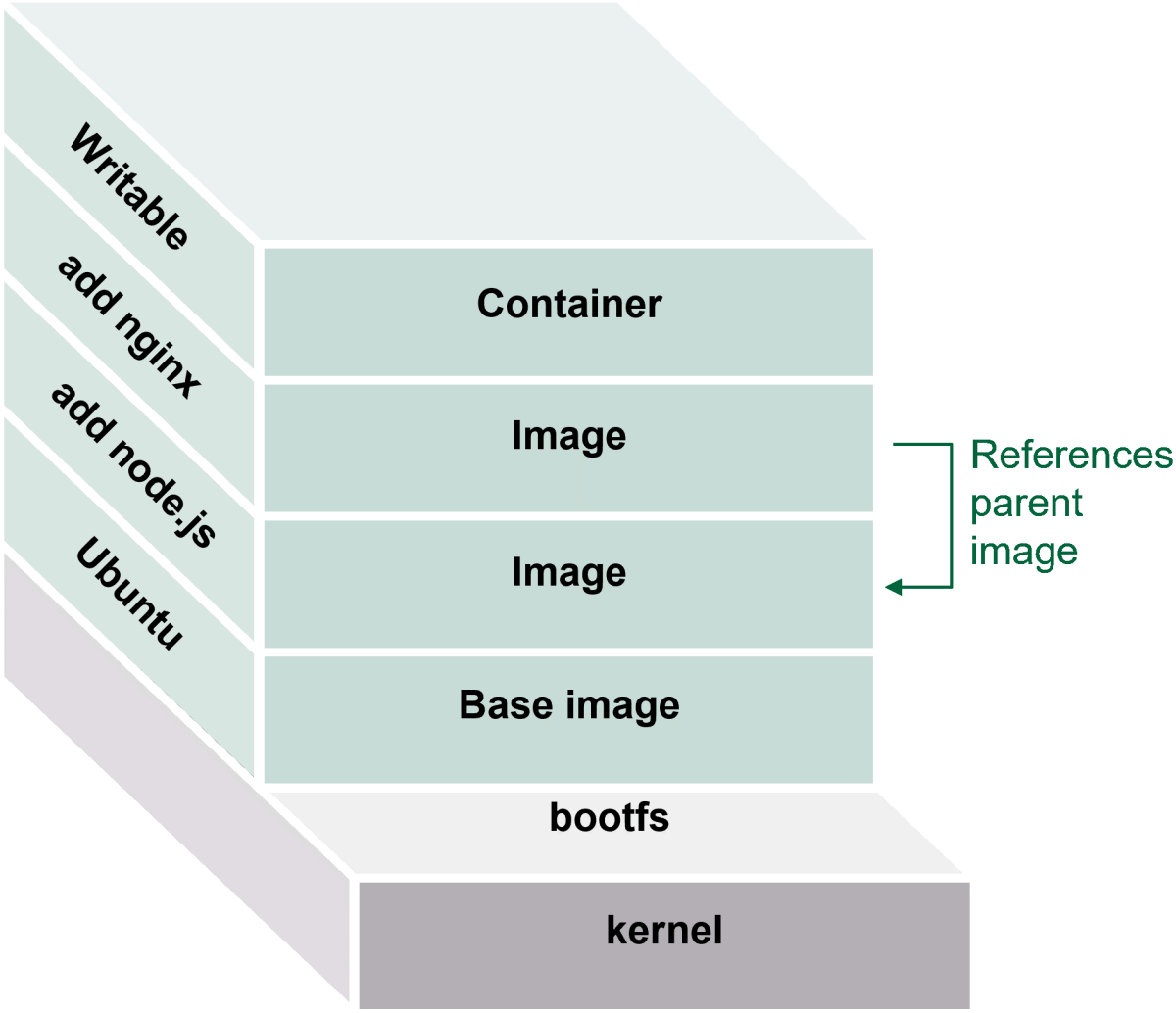
The approach to avoid that undesirable solution, other than carefully maintaining identical versions of each environment, is to containerize the application with its runtime environment and dependencies, using a container engine such a Docker.



This is a much more elegant and versatile solution. It allows developers to work in any suitable environment without worrying about compatibility issues when deploying and migrating between environments.

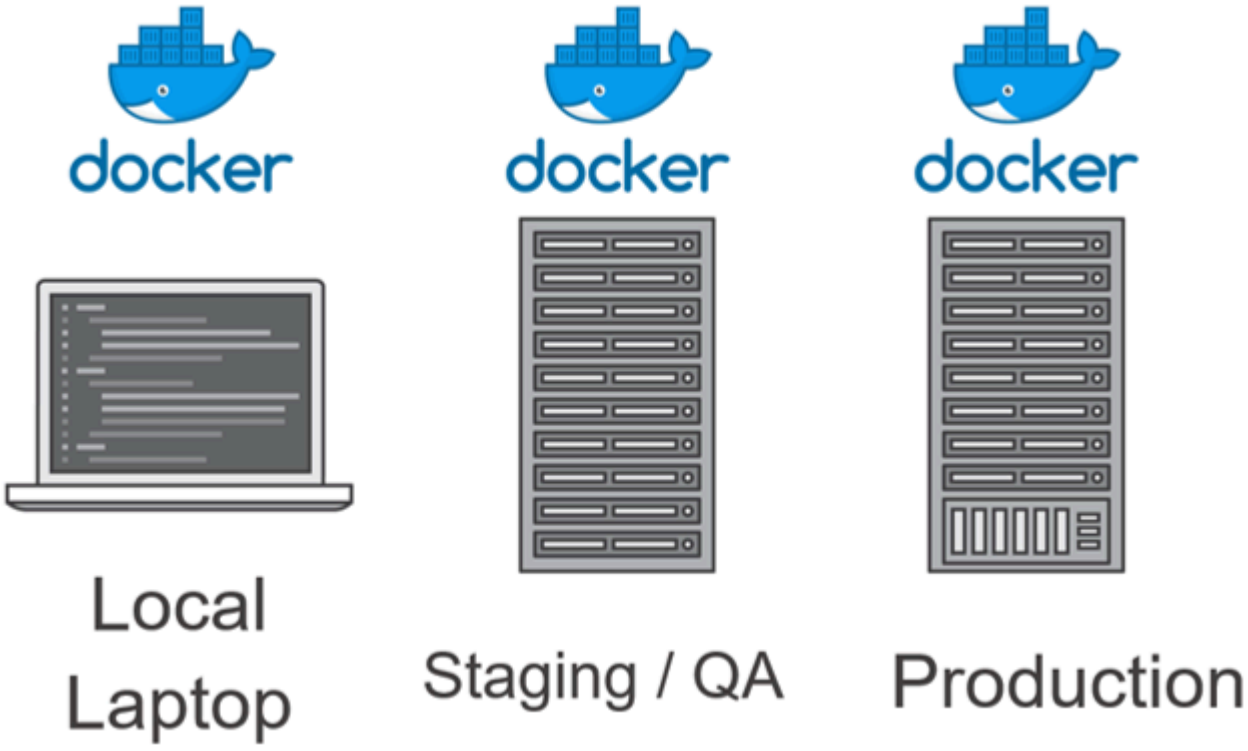
Docker Container Image

Using Docker as an example, let's have a closer look at how the container engine works. As mentioned, the components of the application are packaged into an image. Docker provides many base images that have the underlying dependencies in place. From there, dependencies can be added to the container, on top of the base image. The container holds the code and configuration of the application. A docker file is created for easy deployment and launch, independently from the host environment.



Deployment via container

With the application and its dependencies packaged in a container (docker file), the migration to different environments is no longer dependent on the settings and configuration of the host, allowing for a smooth and easy deployment of the application.



Container & Docker Benefits

As you can gather, there are several benefits to using containers to deploy and migrate applications. The docker file creates a portable application artifact that runs reliable everywhere. The same host can run different applications, or different versions of an application, with different dependencies simultaneously. There is better resource utilization since the application required only a lightweight container instead of a dedicated host operating system.

In the next section, we will examine the relation between containerization and microservices.

Containers support both Linux and Windows workloads. While several container vendors exist, Docker has captured the lion's share of the market. The company has been driving the software container movement, and has become the de facto standard for packaging, deploying, and running cloud-native applications.

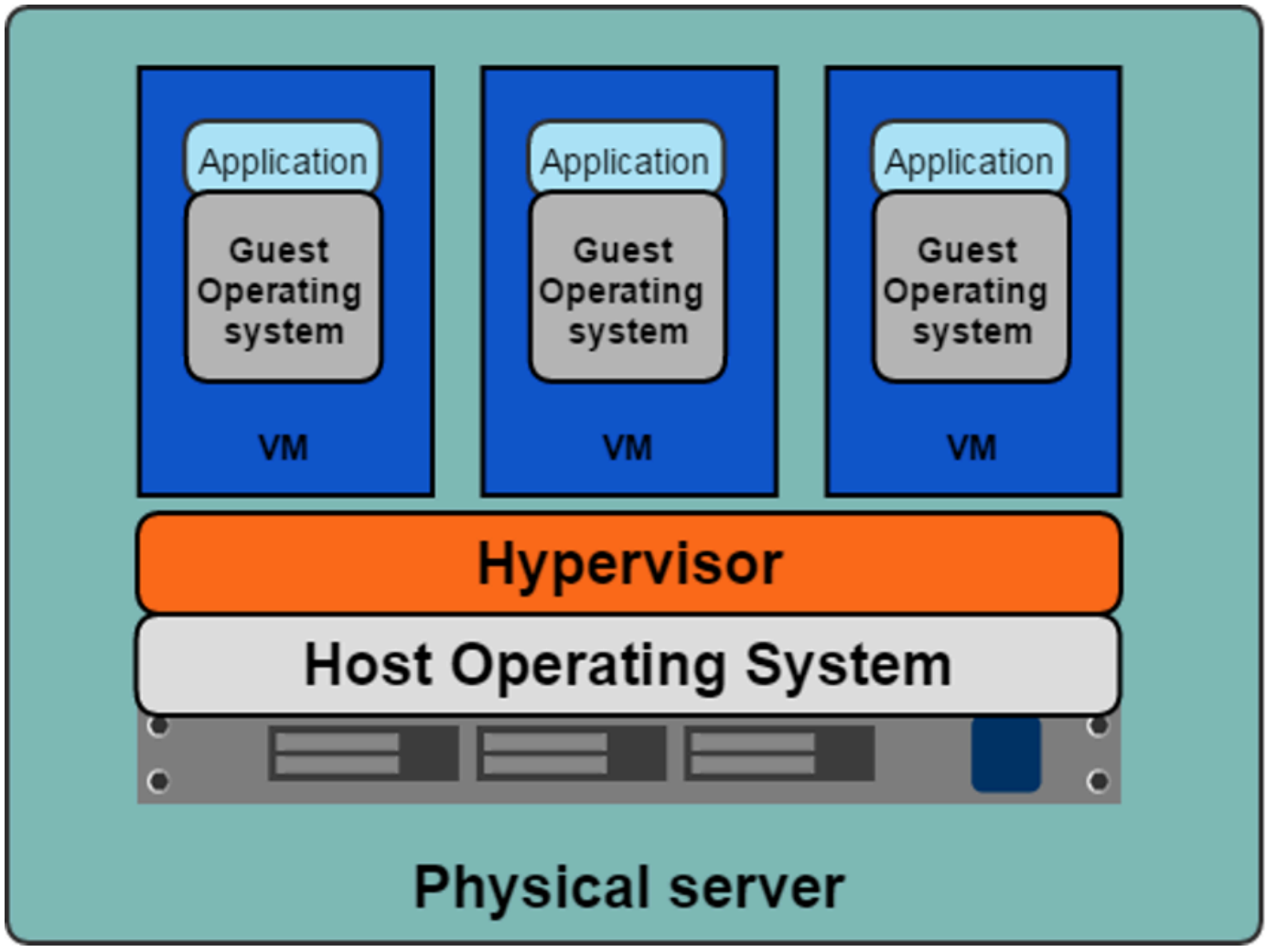
Learn more about Docker at the following resource: **(https://docs.docker.com/get-started/overview/)**

DOCKER OVERVIEW (HTTPS://DOCS.DOCKER.COM/GET-STARTED/OVERVIEW/)

Hypervisor-based Virtualization

As you have learned before, a hypervisor, or a virtual machine monitor, is software, firmware, or hardware that creates and runs virtual machines, or VMs. It's what sits between the hardware and the virtual machine and is necessary to virtualize the server.

Within each virtual machine runs a unique guest operating system. VMs with different operating systems can run on the same physical server—a UNIX VM can sit alongside a Linux VM, and so on. Each VM has its own binaries, libraries, and applications that it services, and the VM may be many gigabytes in size.



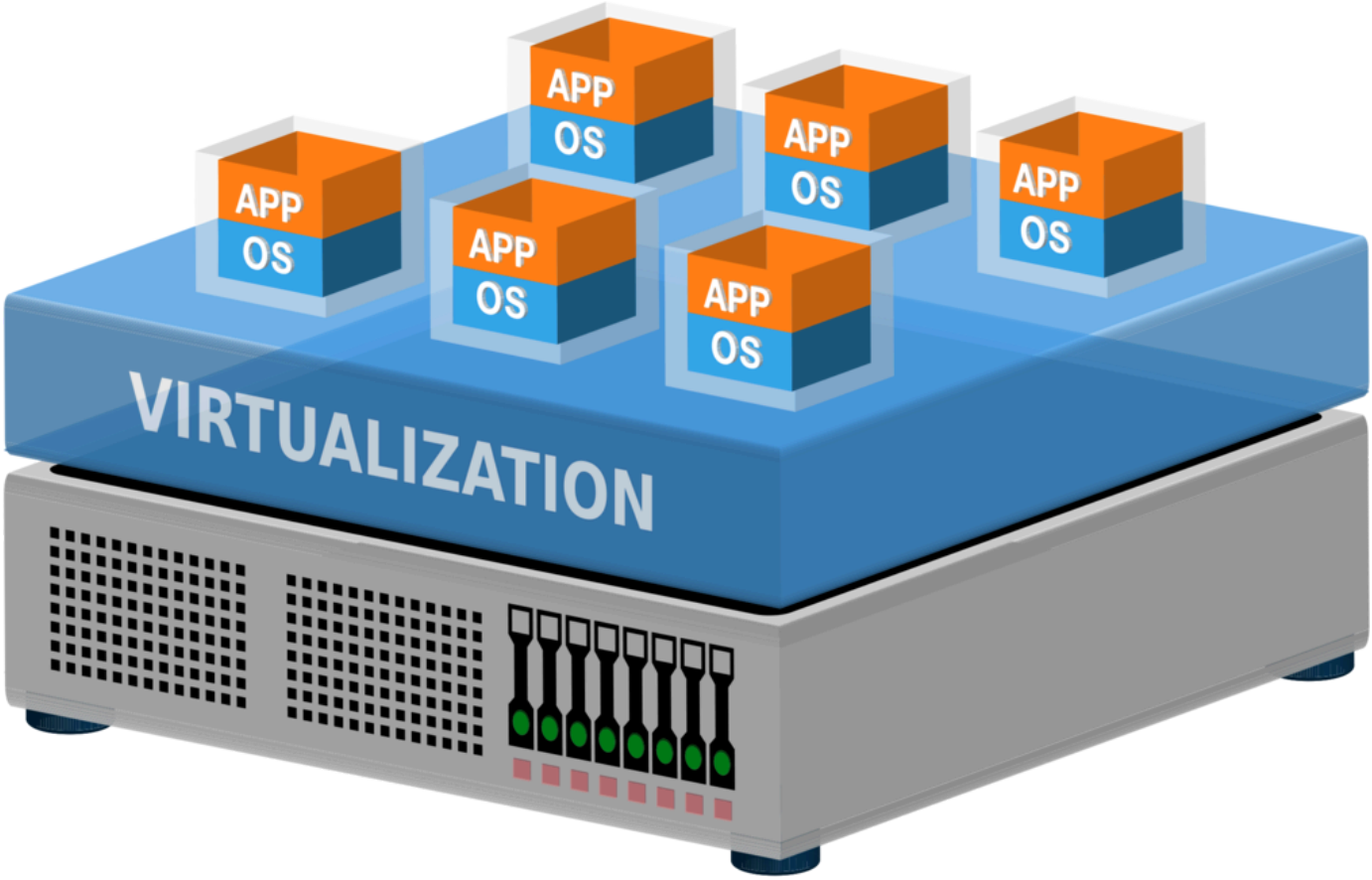
One physical server can contain multiple applications. Each application runs in a virtual machine (VM).

Server virtualization has provided a variety of benefits, one of the biggest being the ability to consolidate applications onto a single system. This eliminated the need of having a single application running on a single server. Virtualization ushered in cost savings through reduced footprint, faster server provisioning, and improved disaster recovery (DR), because the DR site hardware no longer had to mirror the primary data center.

Benefits of virtual machines

You are already familiar with the benefits that come with the use of virtual machines instead of relying on physical servers. Some of these benefits are:

- Better resource pooling.
- Optimized use of one physical machine divided into multiple virtual machines.
- Easier to scale since the virtual machines are software-based.
  When applying this concept to cloud hosting, the versatility of virtual machines allows for:
- Rapid elasticity of allocated resources.
- Using a Pay-as-you-go pricing model.

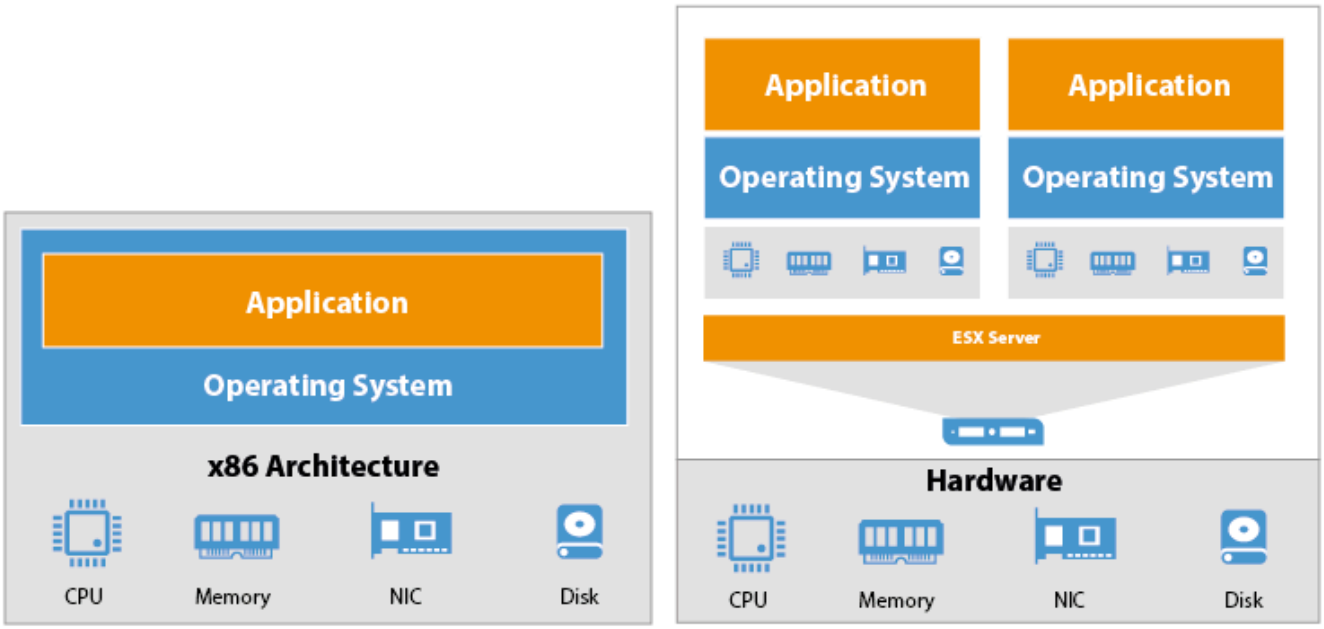


One physical server can provision several easily scalable virtual machines, each having its own independent operating system to run applications in a cloud environment.

Limitations of VMs

Even though this setup is a great improvement over dedicated physical servers, this approach still has its drawbacks. Each VM includes a separate operating system image, which adds overhead in memory and storage footprint. As it turns out, this issue adds complexity to all stages of a software development lifecycle—from development and test to production and disaster recovery. This approach also severely limits the portability of applications between public clouds, private clouds, and traditional data centers.

Each VM still requires CPU and memory allocation, as well as storage for the operating system. Also, the entire guest operating system is dedicated to the VM, which could mean a wasted opportunity to share and pool resources. And even though the application is not tied and dependent on the environment of the physical server, it can still be dependent on the guest OS of the VM, meaning that portability is not guaranteed.



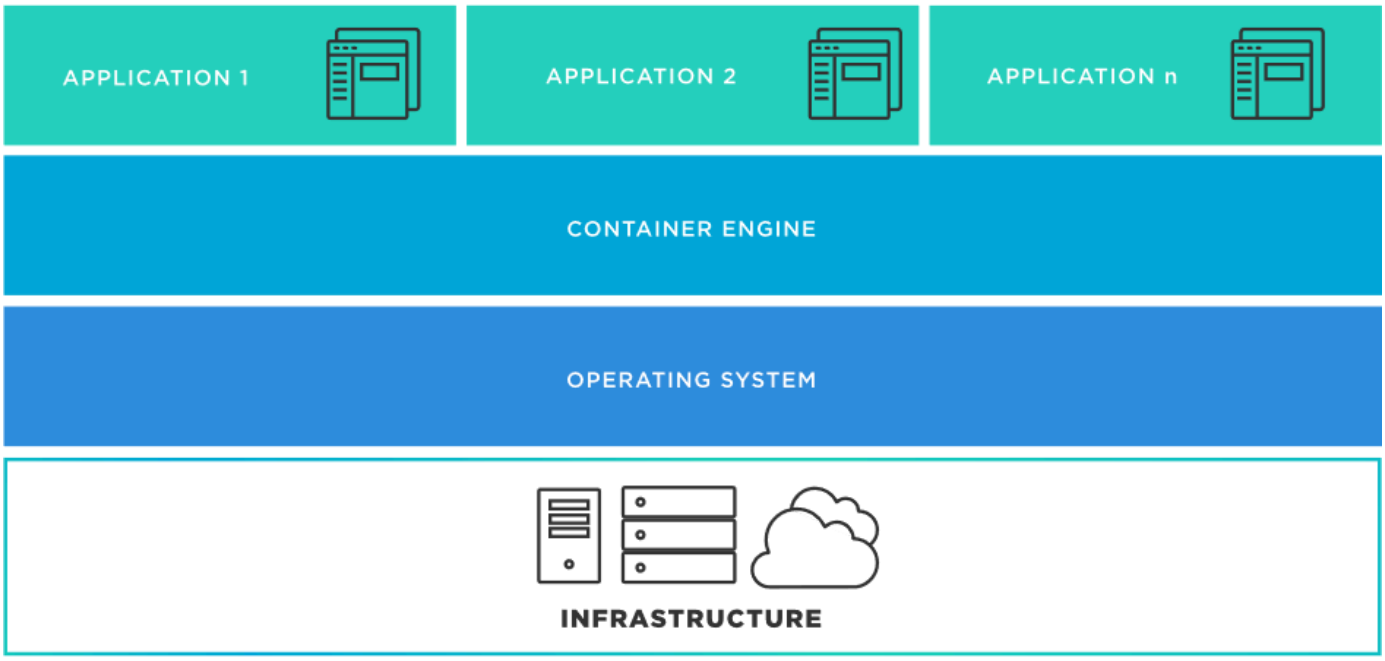
The host server allocates resources to the guest systems to enable them to run applications. This allocation is not optimal since a lot of duplication still occurs.

OS Virtualization: Containerization

Let's look at the notion of containers in this perspective. Our goal is to package and deploy applications in a way that guarantees an optimal use of resources while maintaining portability. An approach towards this goal is to containerize applications and decouple them from the operating system using a container engine.

Containers sit on top of a physical server and its host OS—for example, Linux or Windows. Each container shares the host OS kernel and, usually, the binaries and libraries, too. Shared components are read-only. Containers are thus exceptionally "light"—they are only megabytes in

size and take just seconds to start, versus gigabytes and minutes for a VM.

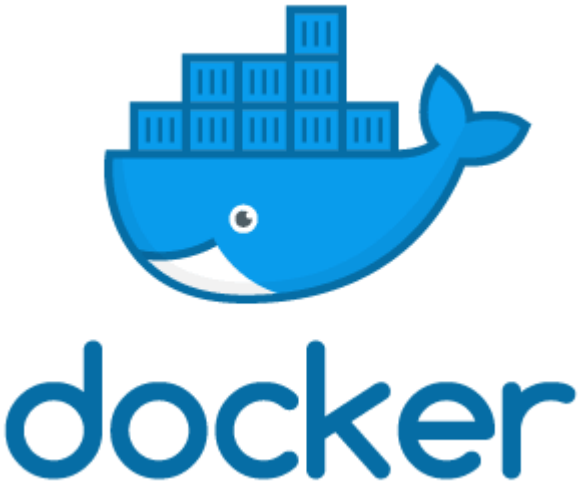


This diagram illustrates the layered approach for containerization. The operating system serves multiple applications, with a container engine providing shared component to the application.

Container engine

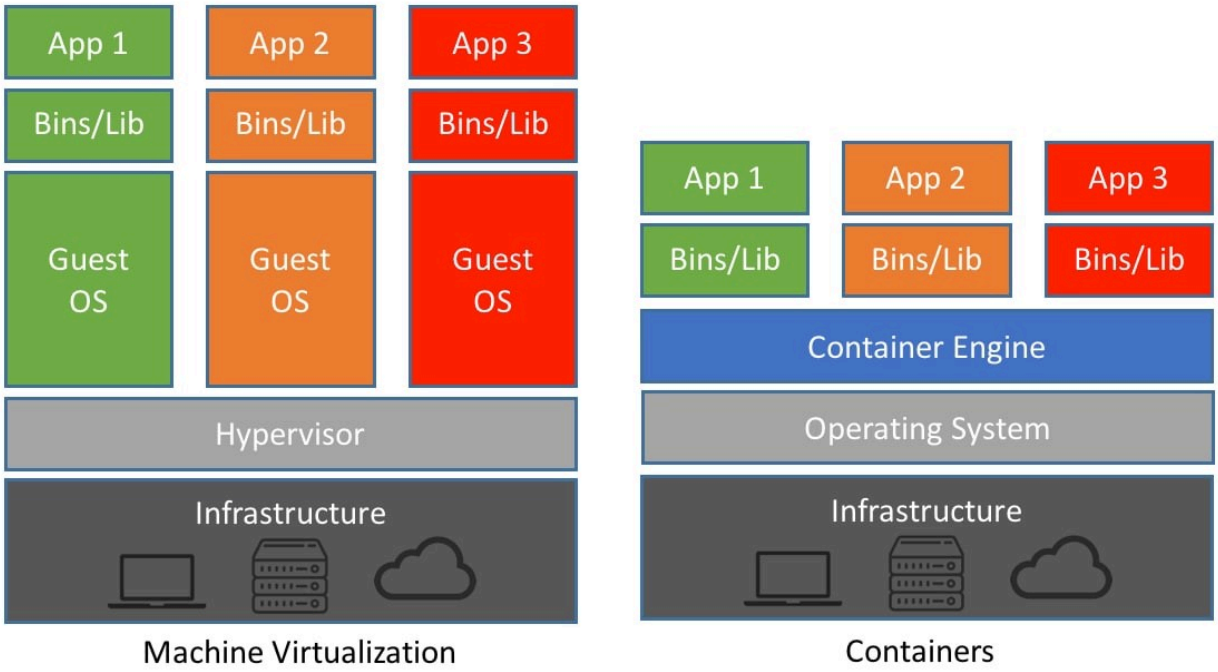
Let's focus on the container engine. As mentioned, it is very light and fast, providing only the required components from the underlying host to the applications in read-only fashion.

A container engine is thus a platform for creating, deploying, and running containerized applications. There are many container engines, including RKT, CRI-O, and LXD. The Docker container engine is a well-known and popular choice and will be used to illustrate the use of a container engine in this lesson.



Containers vs. VMs

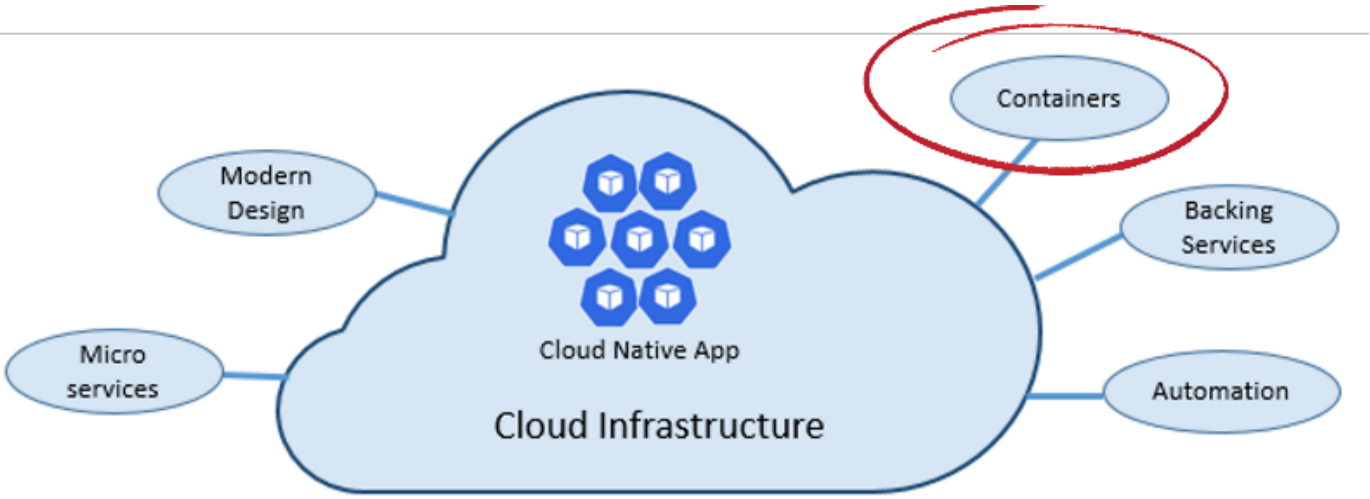
It's important to understand that the use of containers is not a way to avoid virtualization or of saying that VMs do not provide a good solution. In the same way that pooling resources on a physical server to create multiple VMs creates an optimized solution to dedicated physical servers, a container engine provides an optimized approach instead of dedicating a VM operating system to a single application.



Machine Virtualization Containers

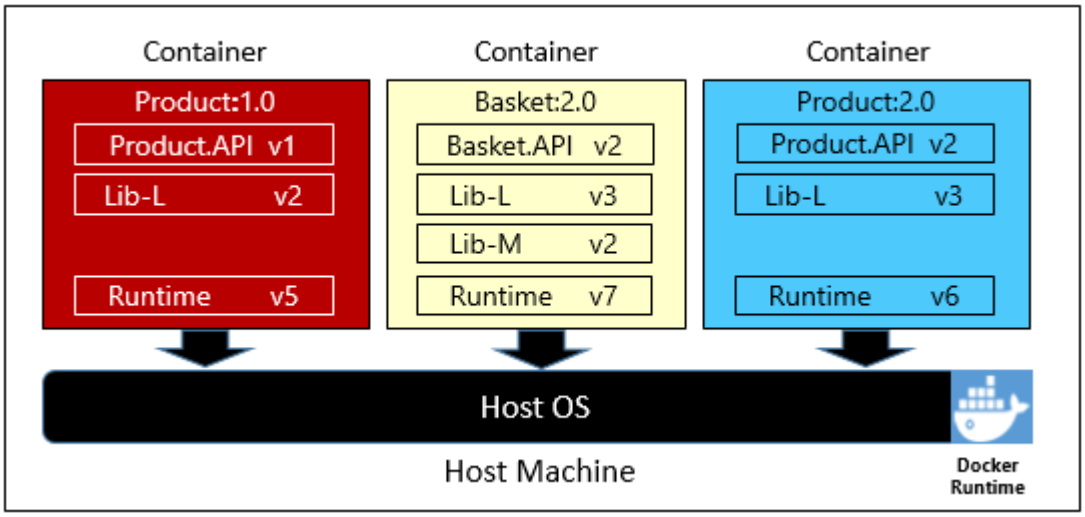
Containers

As you recall from previous lessons, one of the pillars of cloud native app development is the use of containers, which is the subject of this lesson.



Containers are enablers of cloud-native application development. Containerizing a microservice is simple and straightforward.
As was discussed in previous modules, the Cloud Native Computing Foundation (CNCF) places microservice containerization as the first step in their Cloud-Native Trail Map - guidance for enterprises beginning their cloud-native journey.

When using containerization, the code, its dependencies, and runtime are packaged into a binary called a container image. Images are stored in a container registry, which acts as a repository or library for images. A registry can be located on your development computer, in your data center, or in a public cloud. Docker itself maintains a public registry via Docker Hub.



Historical limitations of application deployment

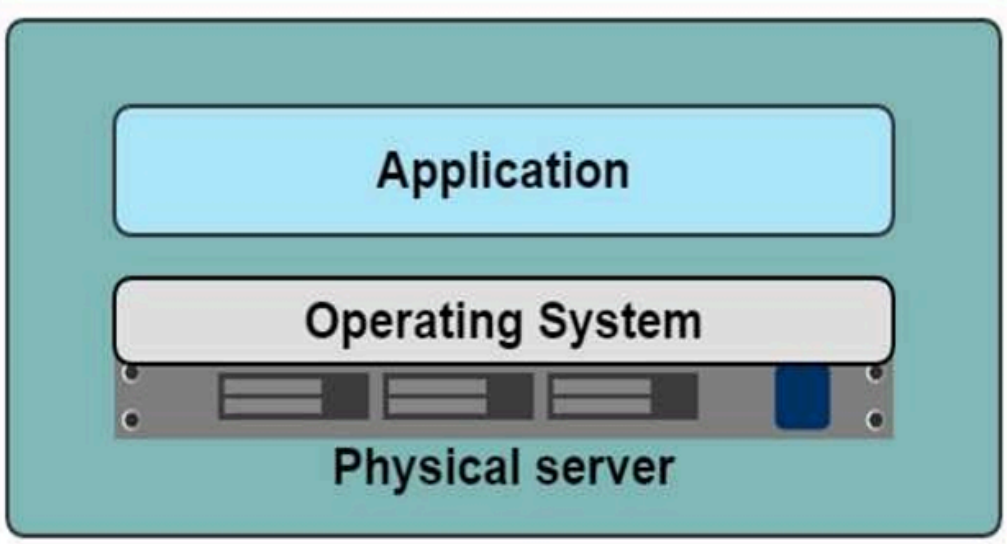
Historically, application deployment has always been highly dependent on creating a suitable and compatible environment to receive the application. Discrepancies between the development environment and others, including production, could derail the release and cause unexpected problems.

Because of this, it was considered best practice to dedicate a physical server to one application, and ensure that updates, upgrades, and any modification to the host environment would not affect the application. This could prove very difficult to achieve sometimes, since the same libraries could be used by the environment and the application. Updating a library for one of the components could jeopardize the other.

Inconveniences causes by the close relationship between the host and application included:

- Slow deployment times due to all the testing to ensure the environment was compatible with requirements.
- Higher costs for the maintenance of the environment.
- Wasted resources, since the environment was dedicated to only one application, and much of the resources went unused.
- Difficulty to scale the application, since vertical scaling was required, creating downtime and provisioning issues.
- Difficulty to migrate the application, since the environment was often required to be highly customized, and this was difficult to reproduce accurately in a different setting because those settings were disassociated from the application itself.
- The close relationship between the environment and the application also created vendor lock-in, preventing the application designers to move to a different environment easily.

One application on one physical server



In the next sections, we will see how containerization can provide a solution to the high dependency on the host environment. But before we do that, we will go over the notion of virtualization and how it relates to containers.

Following up on the discussion on microservices from the previous lesson, we will now look at another pillar of cloud-native development: containers. Containers allow applications to be packaged and deployed to an environment with everything they need. This lesson will explain Virtualization, Containers, and Docker.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Recall the characteristics of virtualization.

02 Define the approach of containerization and its advantages.

03 Apply containerization to microservices.

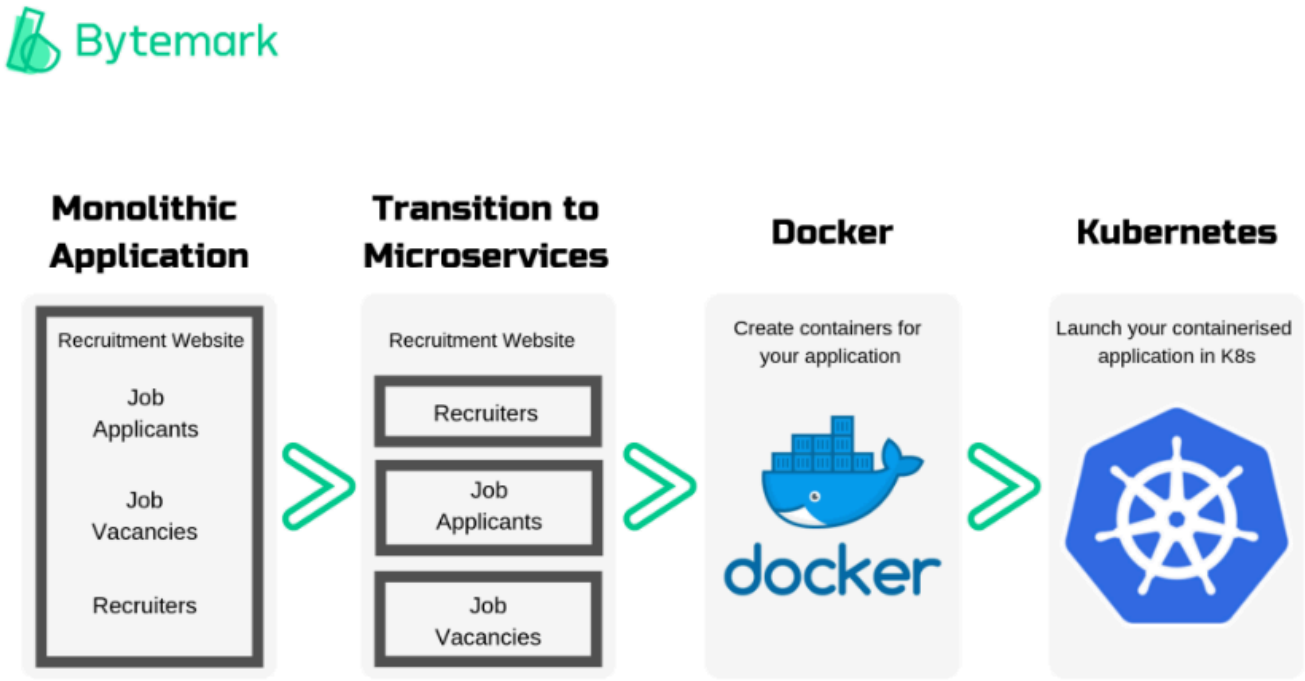
Key Terms and Concepts

Key Term or Concept	Definition
Containers	Packages of software that contain all of the necessary elements to run in any environment. In this way, containers virtualize the operating system and run anywhere, from a private data center to the public cloud or even on a developer's personal laptop.
Virtual Machine	A digital version of a physical computer. Virtual machine software can run programs and operating systems, store data, connect to networks, and do other computing functions, and requires maintenance such as updates and system monitoring.



Containerized microservice

Let's take a step back and go back to the complete picture. If we use the previous example of the recruitment application, we can now add a component to the diagram. The application is broken down to microservices, each a complete application. Each application is containerized using Docker, and the containers are launched and managed by Kubernetes.



Managed Container Orchestrators

You could host your own instance of Kubernetes, but then you'd be responsible for provisioning and managing its resources - which can be complex.

The cloud providers feature Kubernetes as a managed service which enables you to fully leverage the features and power of Kubernetes as a managed service, without having to install and maintain it.



Cloud Providers also offer container orchestration as a proprietary service. For example, Amazon Elastic Container Service (ECS) and Azure Container Service provide these services. Those services are usually easier for teams with limited experience in container orchestration to get started.



Kubernetes

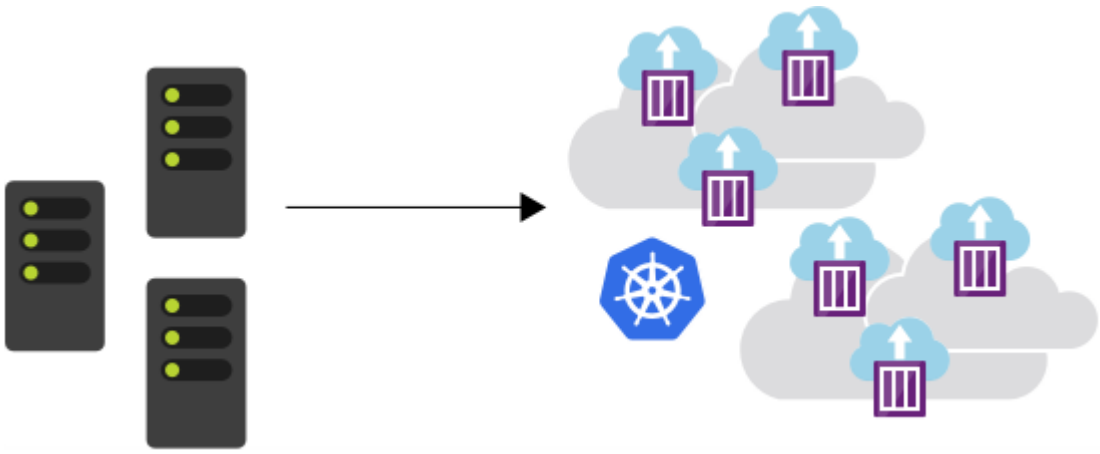
While several container orchestrators exist, Kubernetes has become the de facto standard for the cloud-native world. It's a portable, extensible, open-source platform for managing containerized workloads.

The word “cybernetics” comes from the Greek word kubernētēs (κυβερνᾶν), ‘steersman’, from kubernan ‘to steer’. The word was meant to invoke a conjunction of man and machine, i.e., a cybernetic organism or cyborg. The orchestrator Kubernetes has a nice connotation to cybernetics, going back to the original meaning of the word. Find out more here: **(https://blog.oup.com/2015/03/cyber-word-origins/)**

> **WHERE DOES THE WORD CYBER COME FROM? (HTTPS://BLOG.OUP.COM/2015/03/CYBER-WORD-ORIGINS/)**

Characteristics of Kubernetes

Kubernetes is a portable, extensible open-source platform for managing and orchestrating containerized workloads. It abstracts away complex container management tasks and provides you with declarative configuration to orchestrate containers in different computing environments.



Kubernetes considerations

With Kubernetes, you can view your datacenter as one large compute resource. You don't worry about how and where you deploy your containers, only about deploying and scaling your apps, as needed.
Image

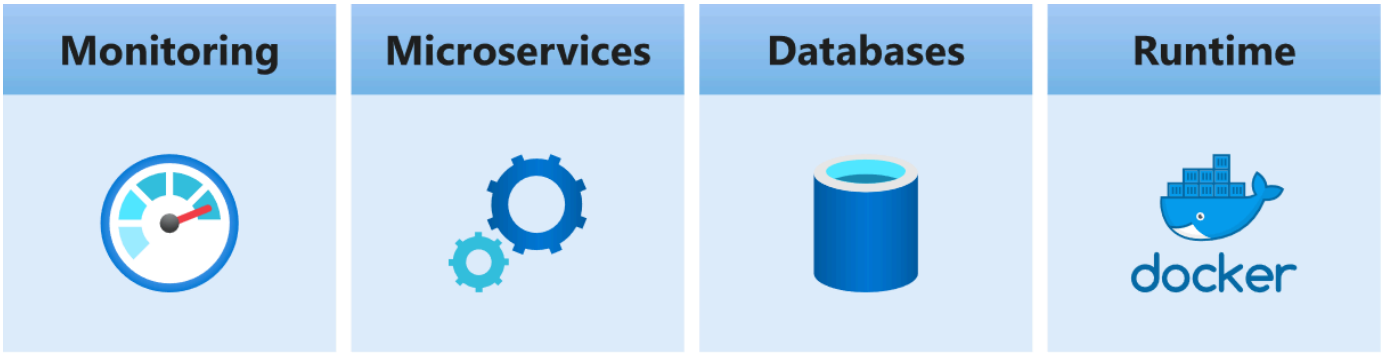
Aspects such as deployment, scaling, load balancing, logging, and monitoring are all optional. You're responsible for finding the best solution that fits your needs to address these aspects.

Kubernetes doesn't limit the types of apps that can run on the platform. If your app can run in a container, it can run on Kubernetes. To make optimal use of containerized solutions, your developers need to understand concepts, such as microservices architecture.

Kubernetes doesn't provide middleware, data-processing frameworks, databases, caches, or cluster storage systems. All these items are run as containers, or as part of another service offering.

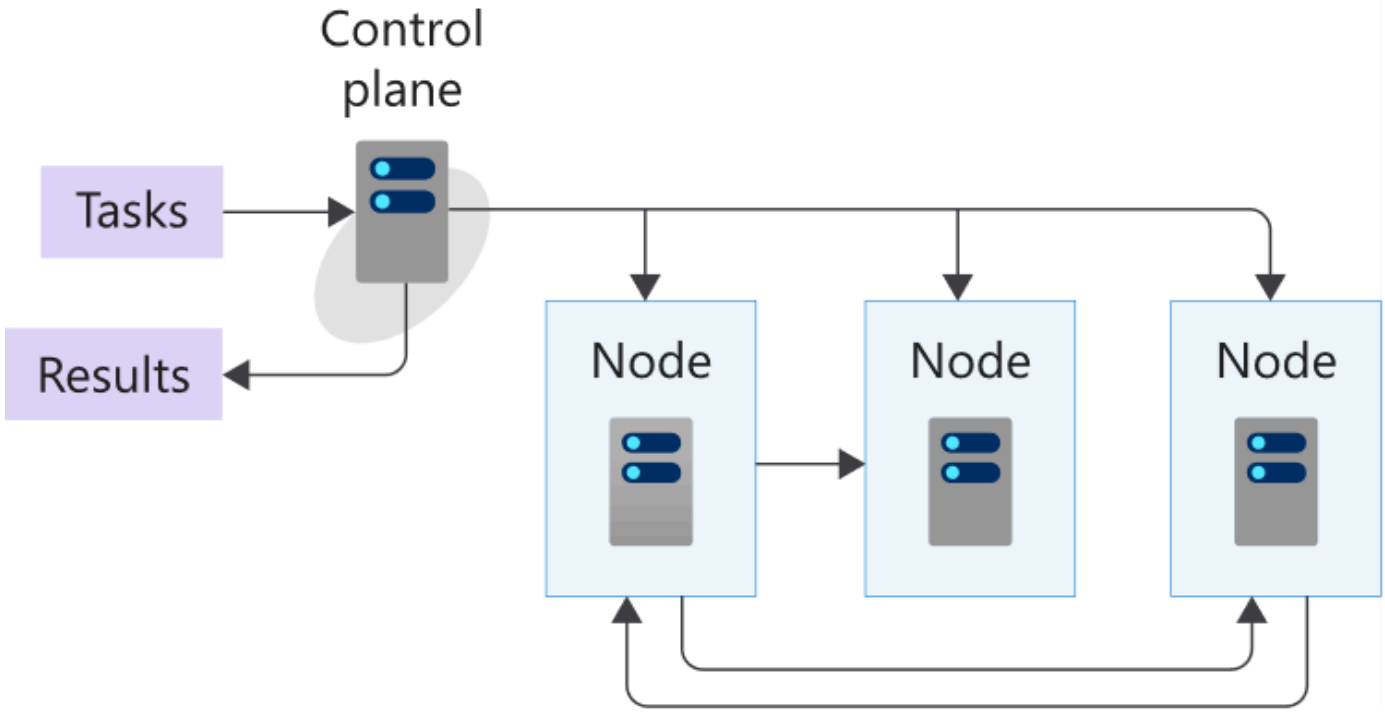
For Kubernetes to run containers, it needs a container runtime, like Docker or containerd. The container runtime is the object that's responsible for managing containers. For example, the container runtime starts, stops, and reports on the container's status.

You're responsible for maintaining your Kubernetes environment. For example, you need to manage OS upgrades and the Kubernetes installation and upgrades. You also manage the hardware configuration of the host machines, such as networking, memory, and storage.



How Kubernetes works

Kubernetes works as a computer cluster. A cluster is a set of computers that you configure to work together and view as a single system. It uses centralized software that's responsible for scheduling and controlling these tasks. The computers in a cluster that run the tasks are called nodes, and the computers that run the scheduling software are called control planes.

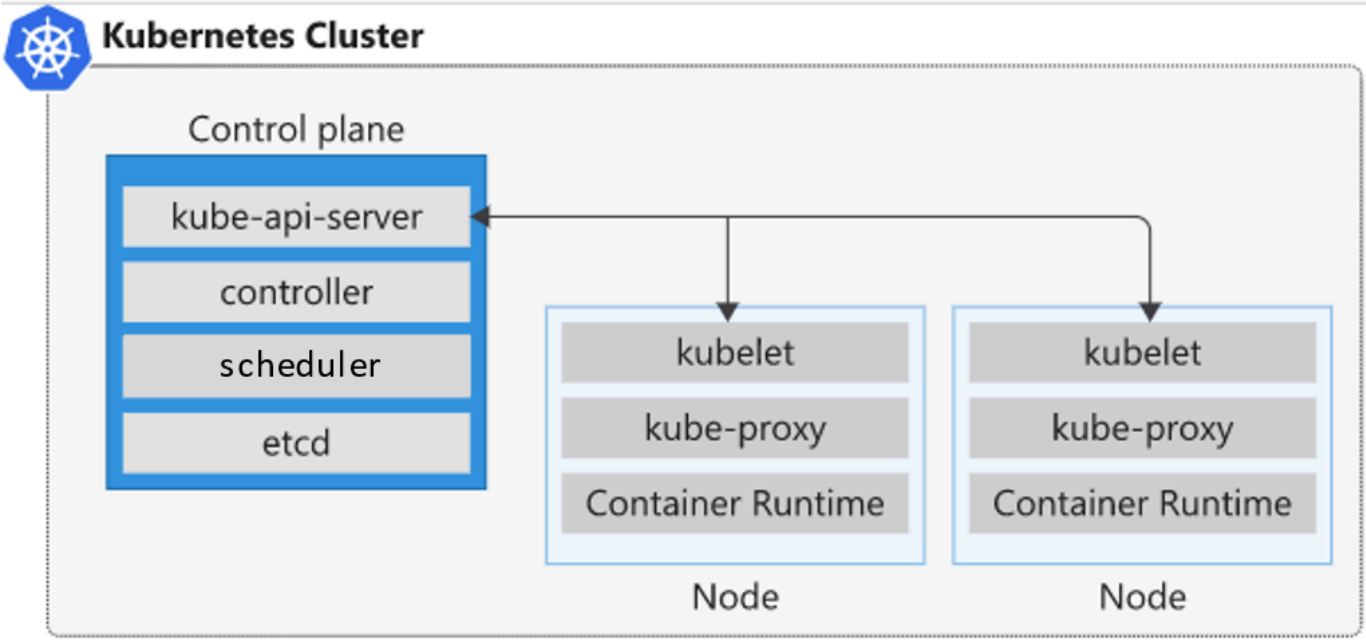


A successfully configured Kubernetes installation depends on a solid understanding of the Kubernetes system architecture. We will look at all the components that make up a Kubernetes installation.

The computers configured in the cluster will typically do the same kinds of tasks. For example, they'll all host websites, APIs, or run computationally intensive work.

Kubernetes architecture

You use Kubernetes as the orchestration and cluster software to deploy your apps and respond to changes in computing resources needed. A Kubernetes cluster contains at least one main plane and one or more nodes. Both the control planes and node instances can be physical devices, virtual machines, or instances in the cloud. The default host OS in Kubernetes is Linux, with default support for Linux-based workloads.



Now, look at both the control planes and worker nodes, and the software that runs on each, in more detail. Understanding the role of each component and where each component runs in the cluster helps you when it comes to installing Kubernetes.

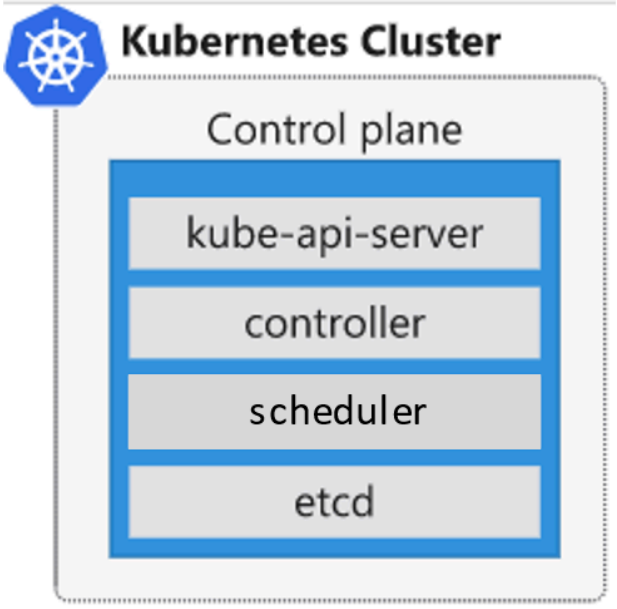
The Kubernetes control plane in a Kubernetes cluster runs a collection of services that manage the orchestration functionality in Kubernetes.

From a learning perspective, it makes sense to use a single control plane in your test environment as you explore Kubernetes functionality. However, in production and cloud deployments, such as Azure Kubernetes Service (AKS), you'll find that the preferred configuration is a high-availability deployment with three to five replicated control planes.

Services that run

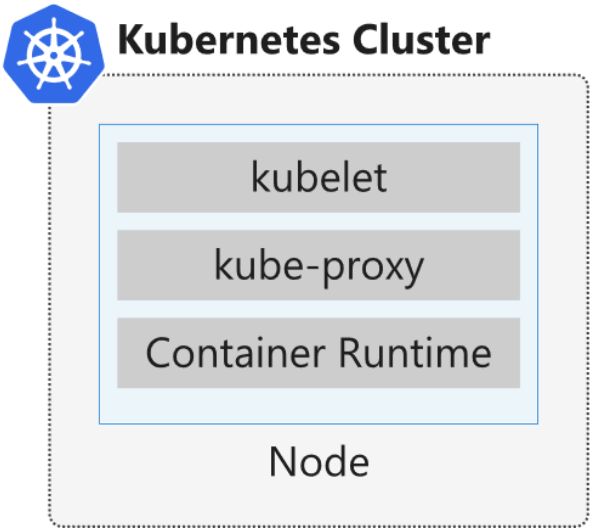
The following services make up the control plane for a Kubernetes cluster:

- API server
- Backing store
- Scheduler
- Controller manager
- Cloud controller manager



The following services run on a Kubernetes node:

- Kubelet
- Kube-proxy
- Container runtime



Interacting with a Kubernetes cluster

Kubernetes provides a command-line tool called kubectl to manage your cluster. You use kubectl to send commands to the cluster's control plane, or fetch information about all Kubernetes objects via the API server.

kubectl uses a configuration file that includes the following configuration information:

- Cluster configuration specifies a cluster name, certificate information, and the service API endpoint associated with the cluster. This definition enables you to connect from a single

workstation to multiple clusters.

- User configuration specifies the users and their permission levels when they're accessing the configured clusters.

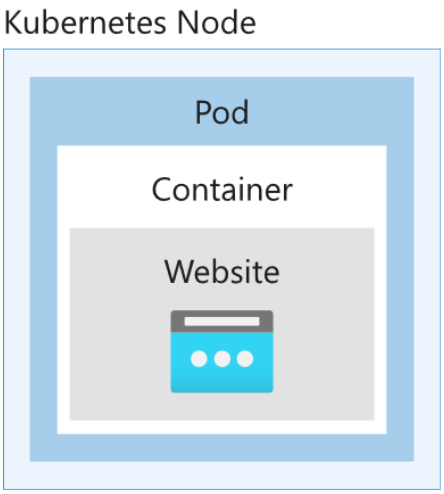
- Context configuration groups clusters and users by using a friendly name. For example, you might have a "dev-cluster" and a "prod-cluster" to identify your development and production clusters.
  You can configure kubectl to connect to multiple clusters by providing the correct context as part of the command-line syntax.

Being handy with commandline interfaces is very useful to become efficient with the management of cloud tools. The following resource provides an introduction to the use of kubectl: **(https://kubernetes.io/docs/tutorials/kubernetes-basics/deploy-app/deploy-intro/)**

USING KUBECTL TO CREATE A DEPLOYMENT (HTTPS://KUBERNETES.IO/DOCS/TUTORIALS/KUBERNETES-BASICS/DEPLOY-APP/DEPLOY-INTRO/)

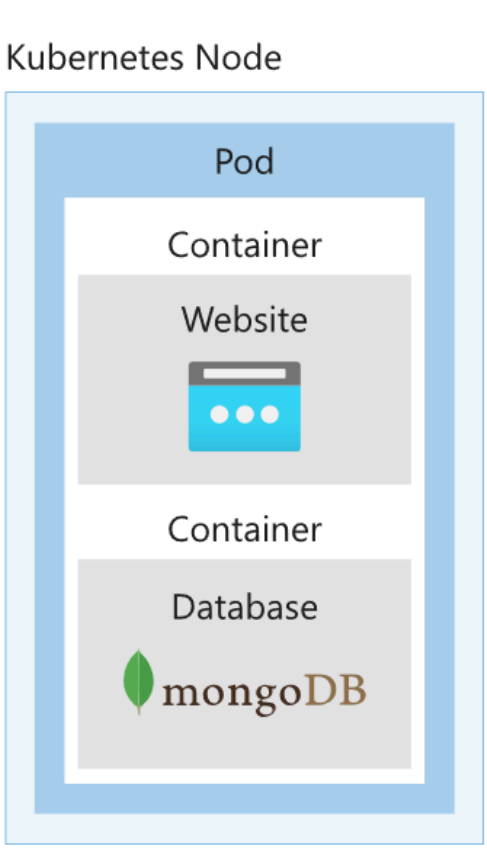
Kubernetes pods

A pod represents a single instance of an app running in Kubernetes. The workloads that you run on Kubernetes are containerized apps. Unlike in a Docker environment, you can't run containers directly on Kubernetes; You package the container into a Kubernetes object called a pod. A pod is the smallest object that you can create in Kubernetes.



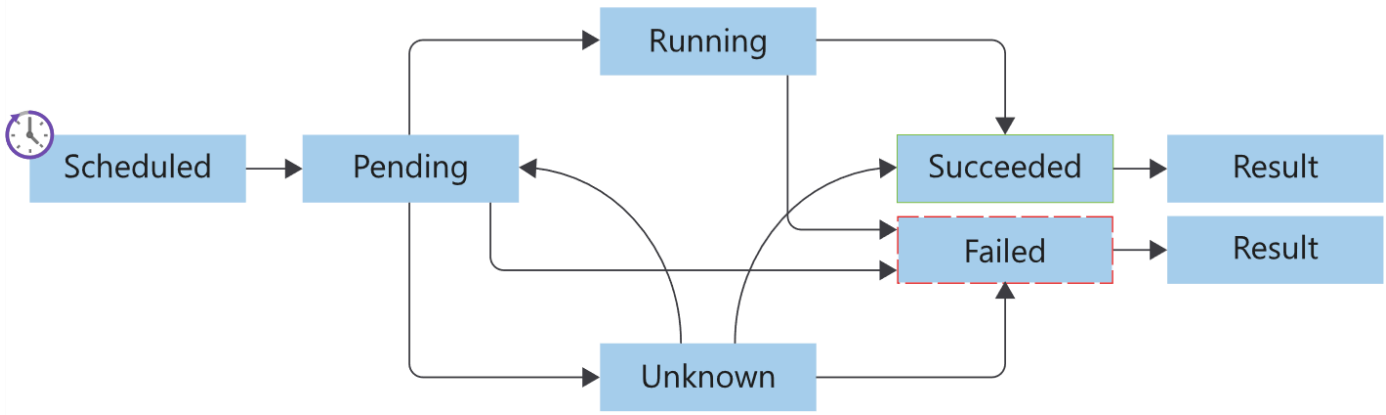
A single pod can hold a group of one or more containers. However, a pod typically doesn't contain multiples of the same app. A pod includes information about the shared storage and network configuration, and a specification about how to run its packaged containers.

You use pod templates to define the information about the pods that run in your cluster. Pod templates are YAML-coded files that you reuse and include in other objects to manage pod deployments.



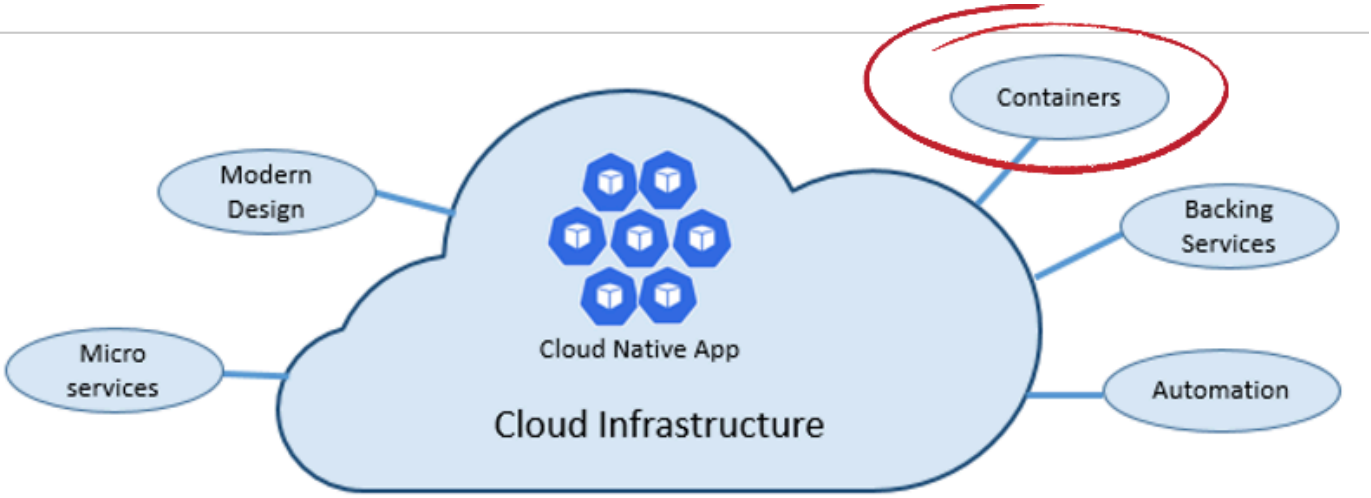
Lifecycle of a Kubernetes pod

Kubernetes pods have a distinct lifecycle that affects the way you deploy, run, and update pods. You start by submitting the pod YAML manifest to the cluster. After the manifest file is submitted and persisted to the cluster, it defines the desired state of the pod. The scheduler schedules the pod to a healthy node that has enough resources to run the pod.



Containers

This lesson builds on the notion of the previous lesson about containers. You will recall that containers are one of the pillars of cloud-native application development. In this lesson, you will learn more about the use of containers and related technology.

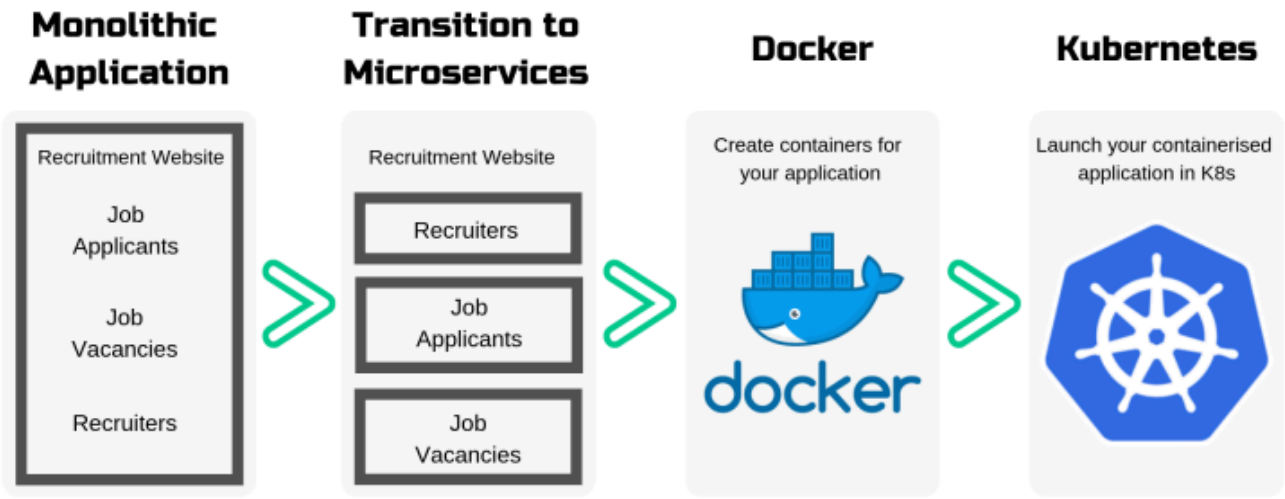


Containerized Microservice

Remember that cloud-native applications avoid the monolithic application model and instead "break down" the application into several independent microservices that communicate in a loosely coupled fashion. These microservices can be talking to other local microservices, but they act as if they were remote to enhance the independence of the microservices. Each microservice is a contained application, following the twelve-factor application methodology. In that respect, containerization of microservices fits with the ideology of cloud-native apps since it promotes the autonomy of the microservices and makes it ready for easy and reliable cloud deployment.

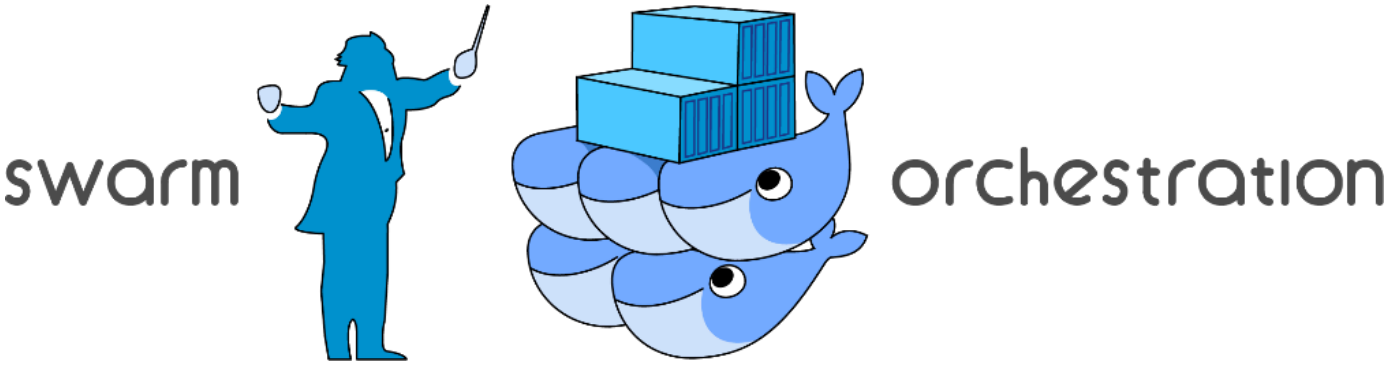
For example, imagine that the goal is to create an application for a recruitment service. The monolithic approach would be to create a single application that has several modules to meet the functionalities required by the application. The cloud-native approach consists into identifying the various microservices that compose the application and create a complete standalone application for each of these microservices. Each microservice is packaged into a docker file and deployed independently. Remember that microservices can grow and evolve at different rates, and changing a microservice does not have to impact the entire application.

Bytemark



Running containers on hosts

This brings us to running the microservices on hosts. With only one container running, or even a few, it is fairly easy to manage the containers in the host. However, when containers get duplicated for horizontal scaling, and several containers are running on the host, it gets very confusing and hard to manage. This is where the orchestrator will come into play to facilitate and manage the containers running on the host.

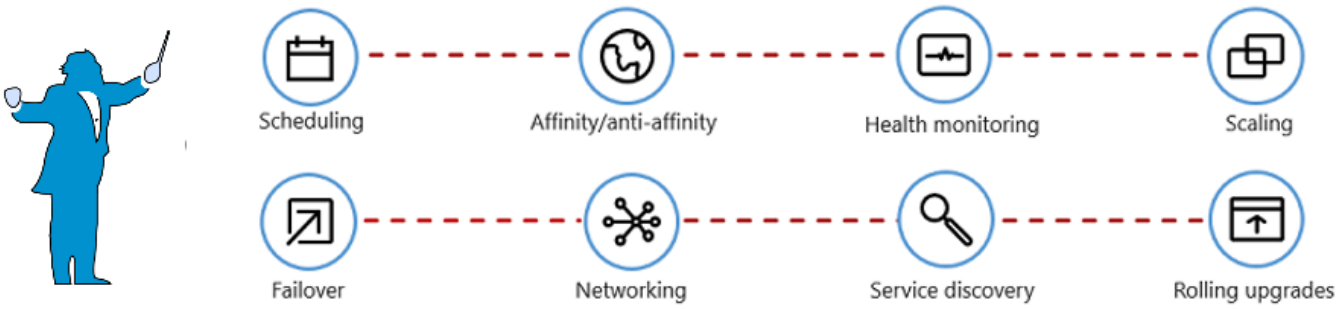


While tools such as Docker create images and run containers, you also need tools to manage them. Container management is done with a special software program called a container orchestrator. When operating at scale with many independent running containers, orchestration is essential.

Orchestration tasks

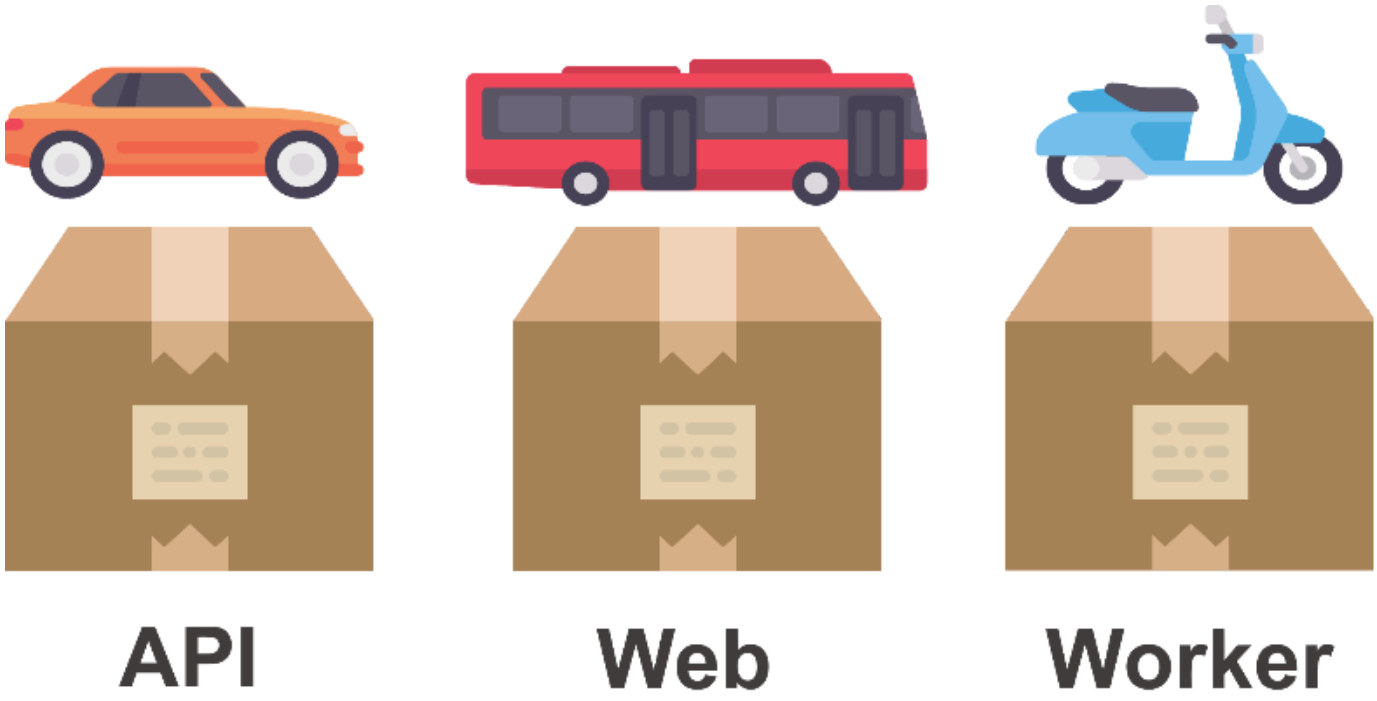
So, what do container orchestrators do? Let's look at the tasks that orchestrators take on:

- Scheduling: Automatically provision container instances.
- Affinity/anti-affinity: Provision containers nearby or far apart from each other, helping availability and performance.
- Health monitoring: Automatically detect and correct failures.
- Failover: Automatically reprovision a failed instance to a healthy machine.
- Scaling: Automatically add or remove a container instance to meet demand.
- Networking: Manage a networking overlay for container communication.
- Service Discovery: Enable containers to locate each other.
- Rolling Upgrades: Coordinate incremental upgrades with zero downtime deployment. Automatically roll back problematic changes.

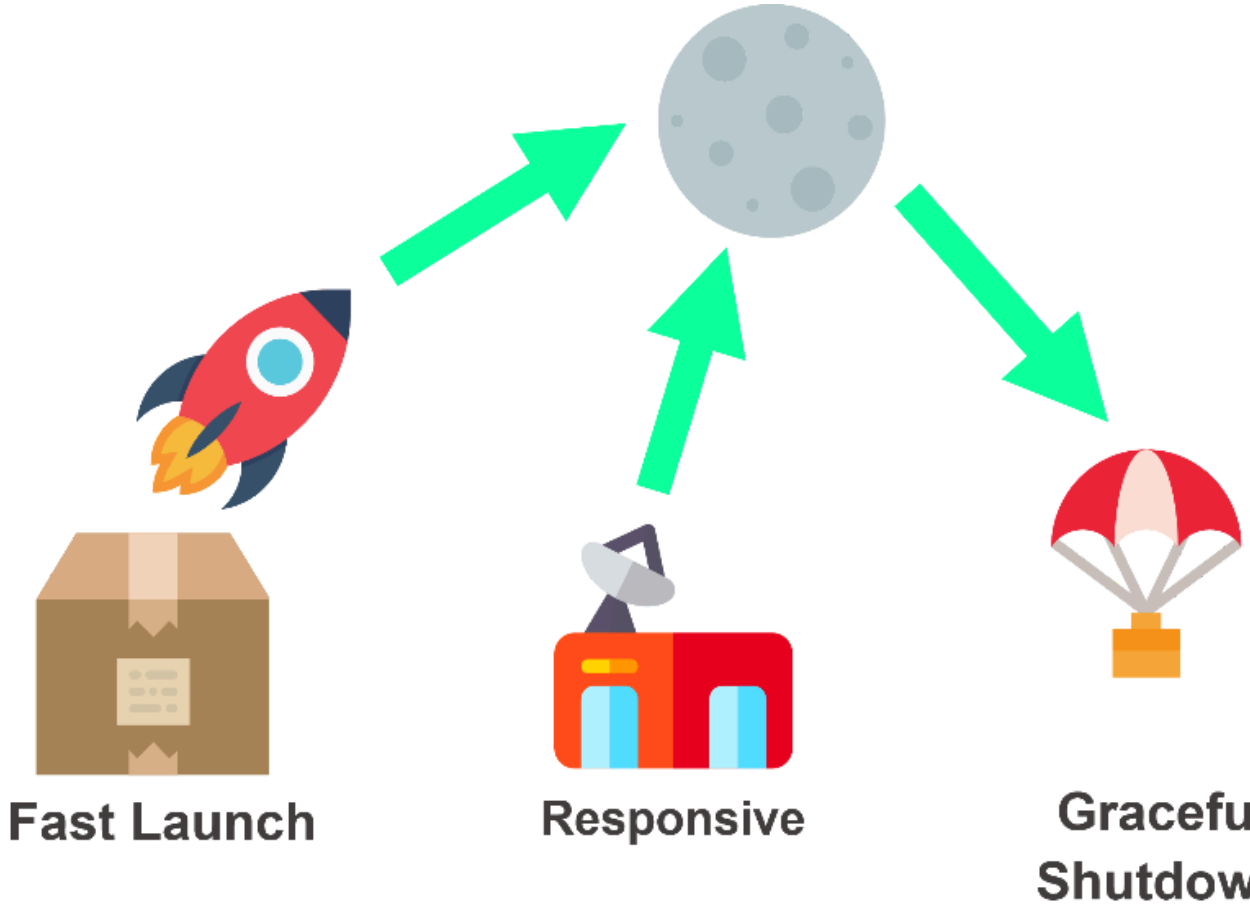


Concurrency and disposability

Notice how container orchestrators embrace the "Concurrency" and "Disposability" principles from the Twelve-Factor Application. Orchestrators scale out services horizontally across multiple identical processes (copies) as opposed to scaling-up a single large instance on the most powerful machine available. Developing the application, i.e., the microservice, to be concurrent makes scaling out in cloud environments seamless.



Service instances should be disposable. When programming applications, favor fast startup to increase scalability opportunities and graceful shutdowns to leave the system in a correct state. Docker containers along with an orchestrator inherently satisfy this requirement.



This lesson will continue the exploration of the "containers" pillar of cloud-native app development. You were introduced to the concept of containers in the previous lesson, as a mechanism to package apps with their runtime and dependencies for easy and reliable migration and deployment. This lesson will pick up on these notions where we left off and explain how to scale containers and use container orchestrators.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Define how to scale containers to allocate required resources to microservices.

02 Recall the basic approach to use a container orchestrator.

03 Apply the use of containerization to package, deploy, and orchestrate containers.

Key Terms and Concepts

Key Term or Concept	Definition
Containerd	Containerd is a container backend. It is an abstraction of kernel features that provides a relatively high-level container interface. Containerd started when Docker extracted its container runtime out into a new project.

Key Term or Concept	Definition
Yet another markup language (YAML)	YAML is a human-readable data-serialization language. It is commonly used for configuration files and in applications where data is being stored or transmitted. YAML targets many of the same communications applications as Extensible Markup Language but has a minimal syntax which intentionally differs from Standard Generalized Markup Language.

Databases

Databases have traditionally been central to the functioning of an organization. In some cases, centralized databases contain decades of history pertaining to an organization's operations, clients, inventory, and intellectual property. The monolithic design of a web application can be described as "database-centric", meaning that the application is built to provide user interfaces for the management of data back-and-forth to the database.

Modern designs recognize that databases cannot be so centralized anymore. The notion of microservices would tend to associate smaller, independent data storage to each service, as required for the service to perform, instead of making the service reach out to a centralized database in a tightly coupled way.

CENTRALIZED DATABASE (HTTPS://WWW.EDUCBA.COM/CENTRALIZED-DATABASE/)

When looking for flexibility, scalability, and the avoidance of a single-point-of-failure, a centralized database does not appear as an ideal solution. The following article defines the concept of a distributed database system:

DISTRIBUTED DATABASE SYSTEM (HTTPS://WWW.EDUCBA.COM/DISTRIBUTED-DATABASE-SYSTEM/)

Database as a service

When using microservices that act independently and fetch resources using URLs, the use of databases typically becomes a more fragmented and service-oriented proposition. This is embodied by the concept of a database as a service, or DBaaS.

As for other backing services, you could implement and deploy your own virtual machine that runs a database server so that you can deploy a custom-defined database of your choice. While you'd have full control over the environment, you'd forgo many built-in features that are provided through cloud platforms. You'd also be responsible for managing the virtual machine and database for each service. This approach could quickly become time-consuming and expensive.

Databases are so ubiquitous in applications that it comes with no surprise that cloud providers offer ready to use databases as a service, fully provisioned, maintained and monitored by the provider.

Cloud-native applications and databases

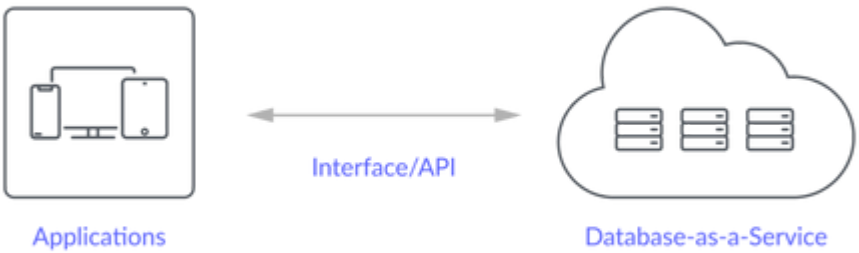
Cloud-native applications favor data services exposed as a Database as a Service (DBaaS). Fully managed by a cloud vendor, these services provide built-in security, scalability, and monitoring. Instead of owning the service, you simply consume it as a backing service. The provider operates the resource at scale and bears the responsibility for performance and maintenance.



Database as a Service

Communications with DBaaS

As with other microservices and backing services, communications are done at the network level using URLs and APIs.



This is in stark contrast to the traditional approach of using a connection string to the database and interacting directly using SQL transactions. This approach is very dependent on the database server and the architecture of the data. The data transferred to the application is packaged in a format that is directly tied to the database. There are numerous problems with the approach:

- The web application source code often contains credentials to connect to the database.
- Changes to the database need to be reflected in the code, sometimes at many different places, and risk breaking down the application.
- Switching from one database provider to another is a very risky operation.

In the Twelve-Factor approach, the database is no longer a massive repository of data, acting as a single entity. There are instead multiple stores of data associated to the microservices and other backing services, for better versatility and resilience.

Benefits of a DBaaS

Let's look more closely at the advantages of using DBaaS. Compared to deploying a database management system on-premises, DBaaS offers your organization significant financial, operational, and strategic benefits:

- **Cost savings**: Laying down infrastructure for database management is expensive; scaling it as needed is costly and often wasteful. With DBaaS, your organization pays a predictable periodic charge based on the resources you consume—there's no need to purchase additional capacity to have on hand for hypothetical future needs.
- **Scalability—up and down**: You can quickly and easily provision additional storage and computing capacity at run time if you need it, and you can scale down your database cluster during non-peak usage times to save cost.
- **Simpler, less costly management**: To manage and maintain a database on-premises, you'd need an in-house administrative team. With DBaaS, the cloud provider manages everything (although you can choose to manage certain aspects yourself if you wish). DBaaS lightens the administrative burden on your existing IT staff and frees them to work on applications and innovation.
- **Rapid development and faster time-to-market**: With an on-premises database system, development teams typically need to request access through IT, a process that can take days or weeks. In contrast, with DBaaS, developers can help themselves to database capabilities and spin up and configure a database that's ready to integrate with their application in minutes.
- **Data and application security**: Cloud database providers typically offer enterprise grade security, including features like default encryption of data at rest and in-transit and integrated identity and access management controls. Some also meet specific regulatory compliance standards.

- **Reduced risk**: DBaaS offerings from major cloud providers typically include a service-level agreement (SLA) guaranteeing a certain amount of uptime. In the unlikely event that your provider doesn't meet the requirements stipulated in the SLA, you'll be compensated for any excess downtime you experience.

- **Software quality**: The major cloud providers offer a wide variety of highly configurable DBaaS options—each preselected for quality, so you won't have to worry about wading through hundreds of different databases.

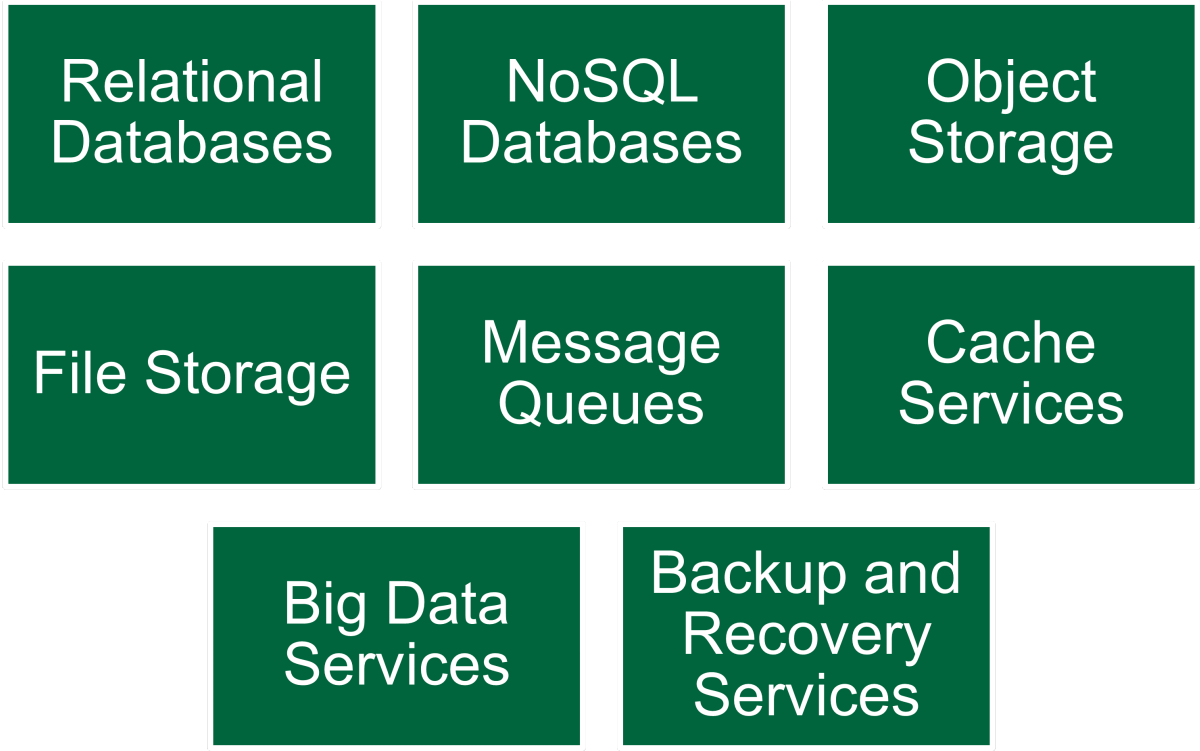


Common backing services

As we defined previously, backing services are external services that provide data storage, management, and processing capabilities to software applications.

Here are some of the most common types of backing services:

- Relational databases: These are traditional databases that store data in tables and use structured query language (SQL) to retrieve and manipulate data. Examples include MySQL, PostgreSQL, and Microsoft SQL Server.
- NoSQL databases: These databases are designed to handle large amounts of unstructured or semi-structured data, such as documents, images, or audio and video files. Examples include MongoDB, Cassandra, and Amazon DynamoDB.
- Object storage: Object storage is a type of data storage architecture that is optimized for storing large amounts of unstructured data, such as photos, videos, and audio files. Examples include Amazon S3 and Microsoft Azure Blob Storage.
- File storage: File storage provides a centralized location for storing and sharing files. Examples include Microsoft OneDrive, Google Drive, and Dropbox.
- Message queues: Message queues provide a way to decouple the components of a software system and ensure that messages are delivered in the right order. Examples include RabbitMQ, Apache Kafka, and Amazon Simple Queue Service (SQS).
- Cache services: Cache services provide a temporary storage area for frequently used data, allowing applications to retrieve data faster and reducing the load on the backing database. Examples include Redis and Memcached.
- Big data services: Big data services provide the tools and infrastructure necessary for processing and storing large amounts of data. Examples include Apache Hadoop, Apache Spark, and Amazon EMR.
- Backup and recovery services: Backup and recovery services provide a way to protect data by creating and storing backups and allowing for quick recovery in the event of data loss. Examples include AWS Backup, Veeam, and Microsoft Azure Backup.



These are just a few of the many types of backing services available. When choosing a backing service, it's important to evaluate your specific requirements and choose the right type of service to meet those needs.

How to choose backing services?

Choosing the right backing service can be a crucial factor in the success of a software application. Here are some steps to help you choose the right backing service for your needs:

- Identify your requirements: Define what you need the backing service to do and the data that it needs to store. This will help you determine the right type of backing service, such as a database, file storage, or message queue.
- Consider scalability: Will your application require more resources as it grows? Will you need to scale up or down? The right backing service should be able to handle changes in resource requirements and be scalable to meet the needs of your application.
- Evaluate performance: The backing service should be fast and efficient, especially if your application requires real-time data access.
- Evaluate reliability: Consider the level of availability that your application requires and choose a backing service that is reliable and can provide the level of uptime you need.
- Consider cost: Different backing services have different pricing models, so it's important to choose a service that fits within your budget.

- Evaluate ease of use: Make sure the backing service is easy to use and has good documentation and support.
- Security: If your application handles sensitive data, it's important to choose a backing service that provides robust security features to protect the data.
- Evaluate vendor support: Choose a vendor that provides good support and has a strong reputation in the market.

It's also important to test the backing service before making a final decision. This will help you to determine if it meets all of your requirements and if it is a good fit for your application.

Owned vs Managed Backing Services

You could host your own backing services, but then you'd be responsible for licensing, provisioning, and managing those resources. This is a decision that can impact the versatility and scalability of the application that depends on these services.

Remember that the first pillar of cloud-native app development is to leverage the advantages of cloud computing. This would suggest that using cloud-provided backing services and consuming them as any other cloud service would be a desirable option. However, other factors can come into play that will influence that decision.

Among the characteristics to consider is the level of independence from the application of the backing service. For instance, if we are talking about arbitrary object storage, this is a very stand-alone type of service. There would be little incentive to set up, manage, and provision a cloud-based storage service instead of using a third-party cloud-provisioned service. On the other hand, if the objects stored are confidential documents and constrained by geographical and security requirements, a custom solution might be required. This is the type of analysis that will determine how to select the type of backing service that will fit the solution requirements.

The following article provides a perspective that contrasts the Twelve-Factor app principle to organization-specific constraints for provisioning:

BACKING SERVICES FOR MODERN APPLICATION DEVELOPMENT (HTTPS://WWW.OREILLY.COM/CONTENT/BACKING-SERVICES-FOR-MODERN-APPLICATION-DEVELOPMENT/)

Managed Backing Services

Cloud providers offer a rich assortment of managed backing services. Instead of owning the service, you simply consume it. The cloud provider operates the resource at scale and bears the responsibility for performance, security, and maintenance. Monitoring, redundancy, and availability are built into the service. Providers guarantee service level performance and fully support their managed services - open a ticket and they fix your issue.

Cloud-native systems favor managed backing services from cloud vendors. The savings in time and labor can be significant. The operational risk of hosting your own and experiencing trouble can get expensive fast.

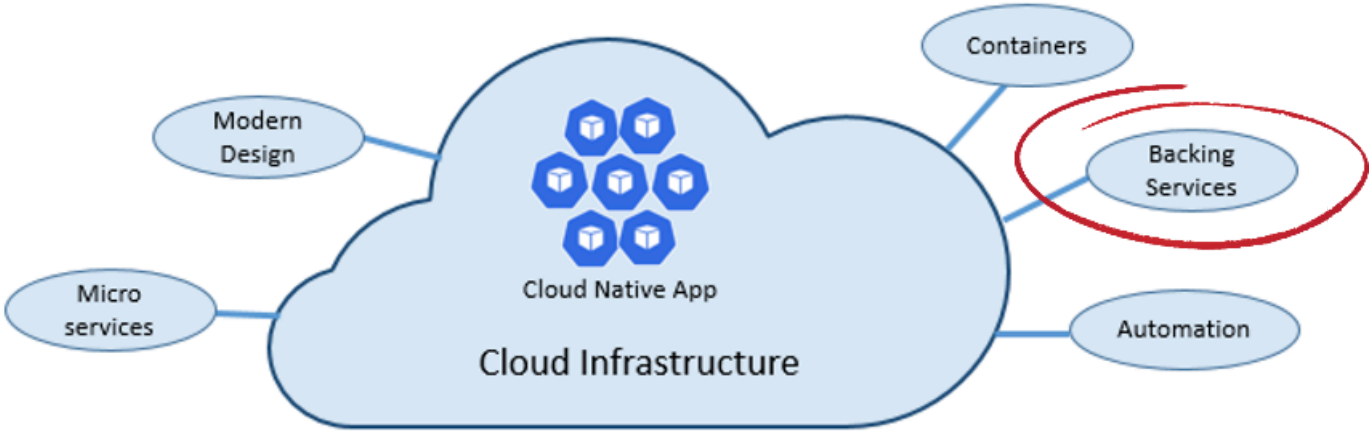


In the absence of particular constraints, using managed services from industry-leading cloud service providers can represent savings, workload reduction, and optimized service delivery.

Backing services

This lesson continues in the exploration of the pillars of cloud-native applications, this time focusing on the “backing services” pillar. Backing services provides background functionalities to cloud-native applications and other cloud services.

Backing services are also directly part of the Twelve-Factor application development methodology for cloud-native application, as factor 4.

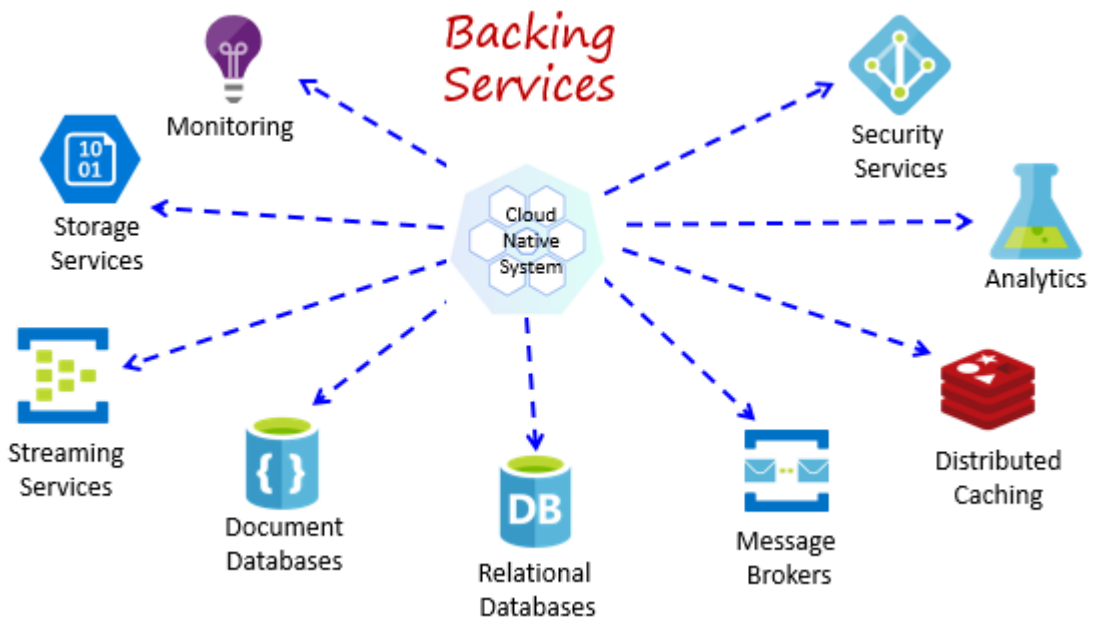


Common backing services

As you will recall from the other pillars that we covered so far, cloud-native applications do not use a monolithic design to produce a centralized application, preferring to break up the application into microservices, each acting as a dedicated standalone application. This allows cloud-native apps to leverage many advantages made available by cloud computing technologies:

- Decentralized infrastructure and deployment.
- Resilience through multiple instances.
- Easier to maintain, enhance, and fix the application by working on separate components.

A drawback of this approach it becomes harder to have an overview of the application, to monitor it as one entity, and to provide secondary services in a seamless fashion.



For example, the traditional three-tier web application design defines the user interface (web pages), the business logic (web application), and the persistent data (backend database). The interaction between these components is tightly coupled, where the business logic produces the interfaces by talking directly to the database to extract or modify data.

When this is broken down into microservices, the interaction becomes loosely coupled. The microservices talk to each other through messages while exchanging formatted blocks of information. Through all this, how do administrators manage correlated data? How do they manage bottlenecks and other application deficiencies? How can they overview the interactions and define potential enhancements to the interactions? This is where backing services come in.

Common backing services include:

- Monitoring tools like dashboards
- Storage services
- Streaming services
- Document databases
- Relational databases
- Message brokers
- Distributed caching for optimization
- Analytical tools
- Security services

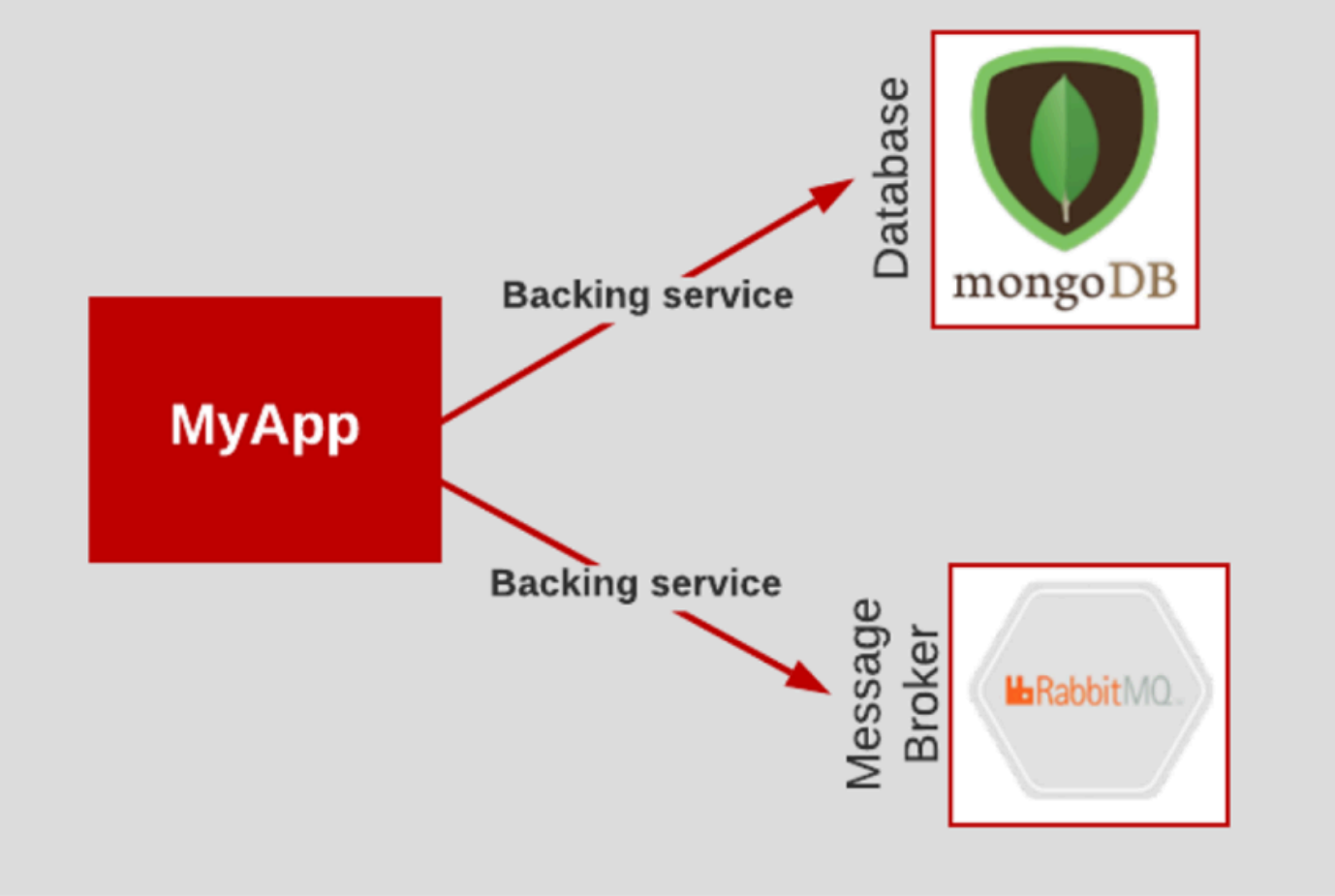
Backing service definition

Let's establish a formal definition for a backing service:

> A backing service is any service the app consumes over the network as part of its normal operation.

As you will recall, microservices run in isolation at the network level, usually with their dedicated port number. Any service that is accessed in this fashion is thus a backing service.

In traditional operating systems, these services could have been accessed over the network, but not exclusively as such. For instance, a database can be accessed by an application through a network connection or a UNIX socket. A file storage service could be accessed through the network or even be a sub-process on the server.



For cloud-native applications, these functionalities are made available as microservices and reached via the network. Examples include:

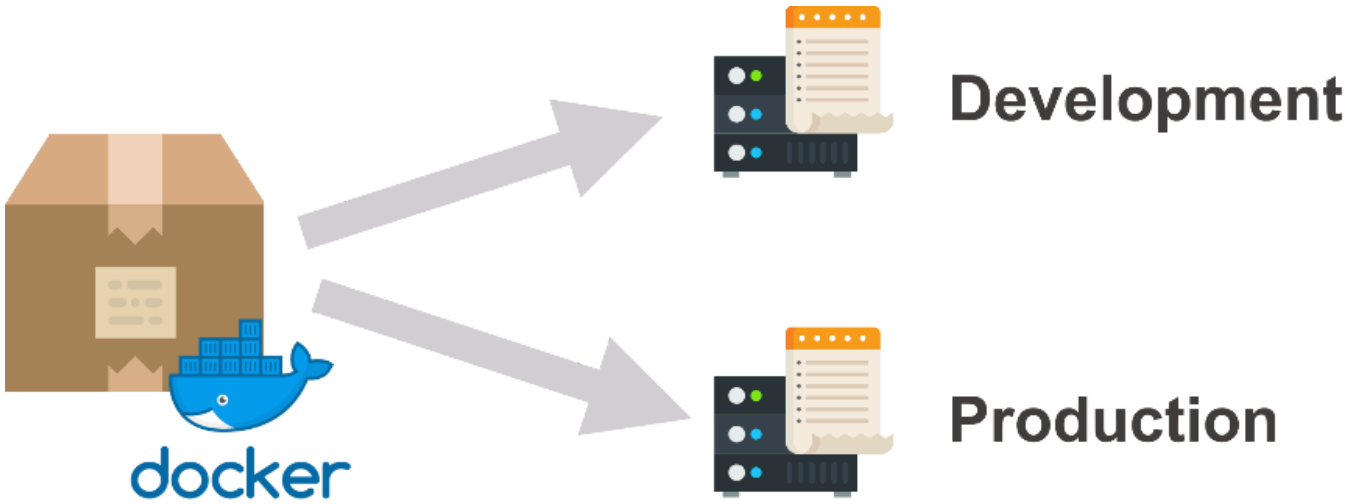
- Databases (e.g., MySQL, PostgreSQL, Cloud Spanner)
- File storage (e.g., NFS, FTP, Cloud Filestore)
- Logging services (e.g., syslog endpoints, Cloud Logging)
- Traditional HTTP APIs (e.g., Google Maps, WikiData, Parcel Tracking APIs)

Connecting to backing services over the network rather than installing them all into the same machine allows developers to focus on their App, independent security upgrades for different components, and flexibility to swap implementations.

Correlation to Factor 3: Configurations

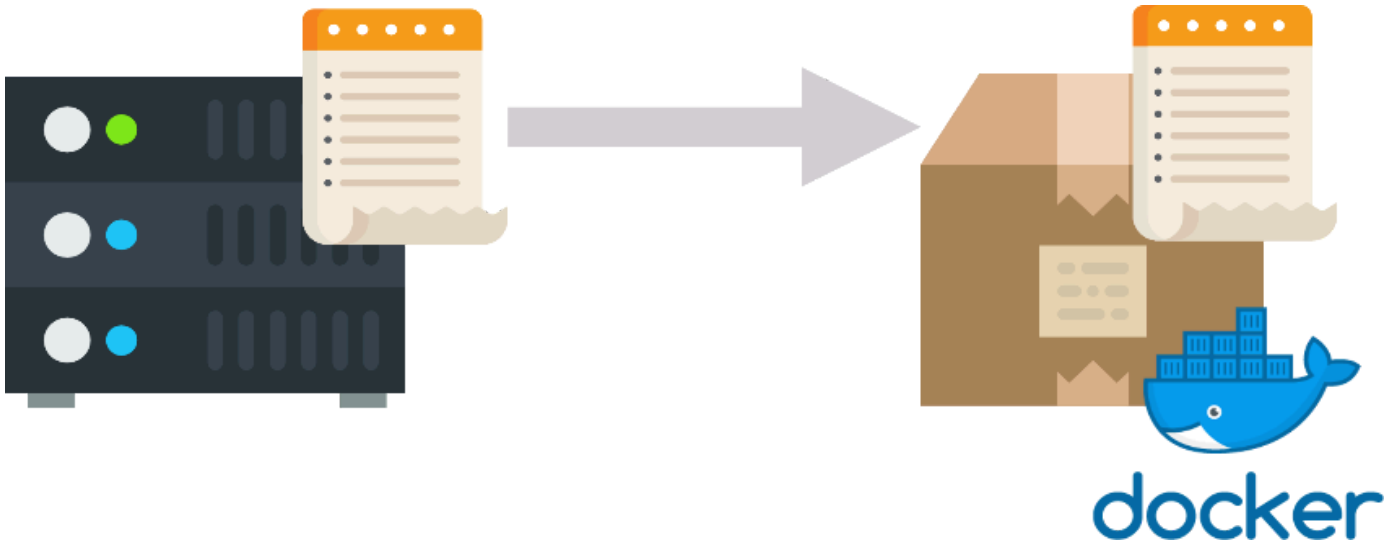
Recall the Twelve-Factor application methodology. Factor 3 is about configurations and states:

- Configuration information is moved out of the microservice and externalized through a configuration management tool outside of the code.
- The same deployment can propagate across environments with the correct configuration applied.
- At runtime the container gets its configuration from the environment.



The same container is deployed to both environments. The configuration is part of the environment on the host.

This factor takes on an extended meaning when considering backing services. Connecting to a backing service should not be hard coded into the application. Instead, the connectivity should be defined through configuration files, as per Factor 3. Defining an environment variable that states the environment in use and how to find and consume microservices is much more flexible.



With this pattern, a backing service can be attached and detached without any changes to the code. You might promote a microservice from QA to a staging environment, in which case you update the microservice configuration to point to the backing services in staging and inject the settings into your container through an environment variable.

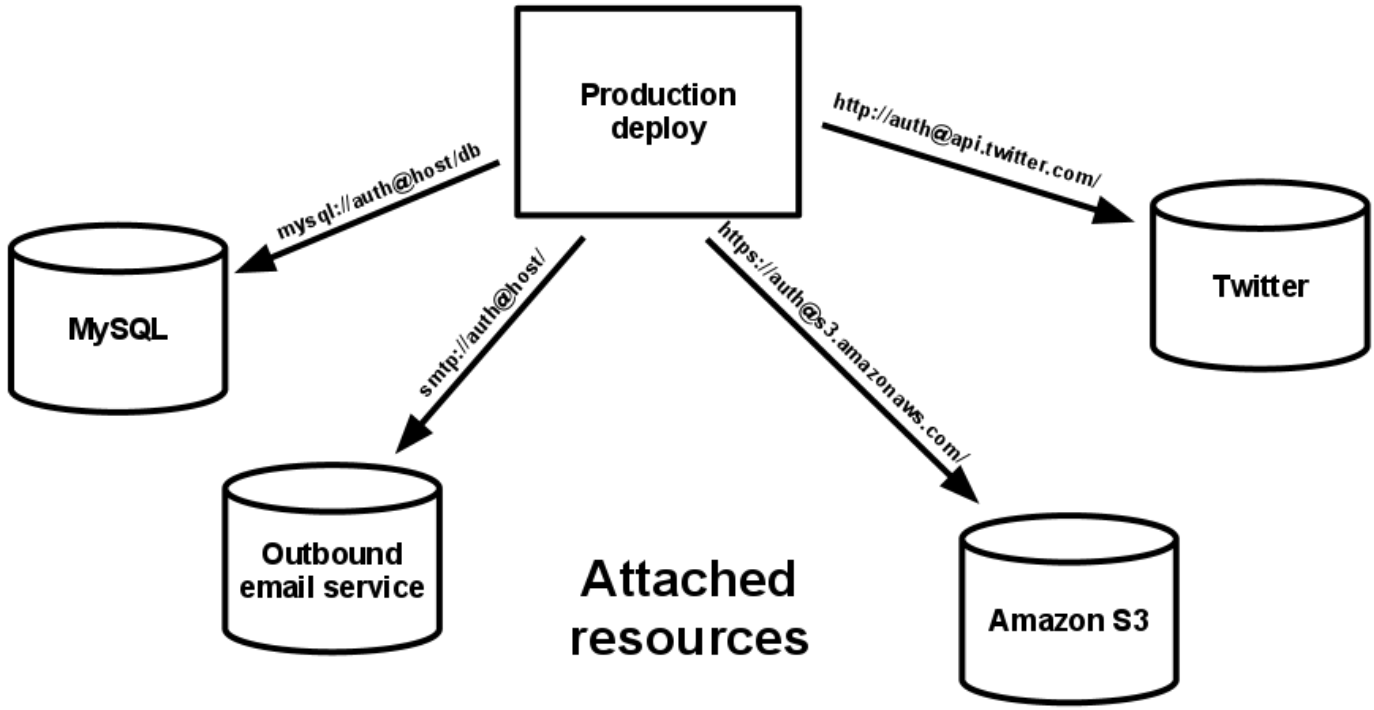
Correlation to Factor 4: Backing Services

As mentioned at the beginning of the lesson, factor 4 of the Twelve-Factor application methodology is “Backing Services”. This principle states that:

- Ancillary resources (data stores, caches, message brokers) should be exposed via an addressable URL.
- The resource should be interchangeable by decoupling it from the application.
- Local services should be treated just like remote third-party ones.

This means that the code for a twelve-factor app makes no distinction between local and third-party services. To the app, both are attached resources, accessed via a URL or other locator/credentials stored in the configuration of the application. A deployment of the twelve-factor app should be able to swap out a local MySQL database with one managed by a third party (such as Amazon RDS) without any changes to the app’s code. Likewise, a local SMTP server could be swapped with a third-party SMTP service (such as Postmark) without code changes. In both cases, only the resource handle in the configuration needs to change.

Each distinct backing service is a resource. For example, a MySQL database is a resource; two MySQL databases (used for sharing at the application layer) qualify as two distinct resources. The twelve-factor app treats these databases as attached resources, which indicates their loose coupling to the deployment they are attached to.



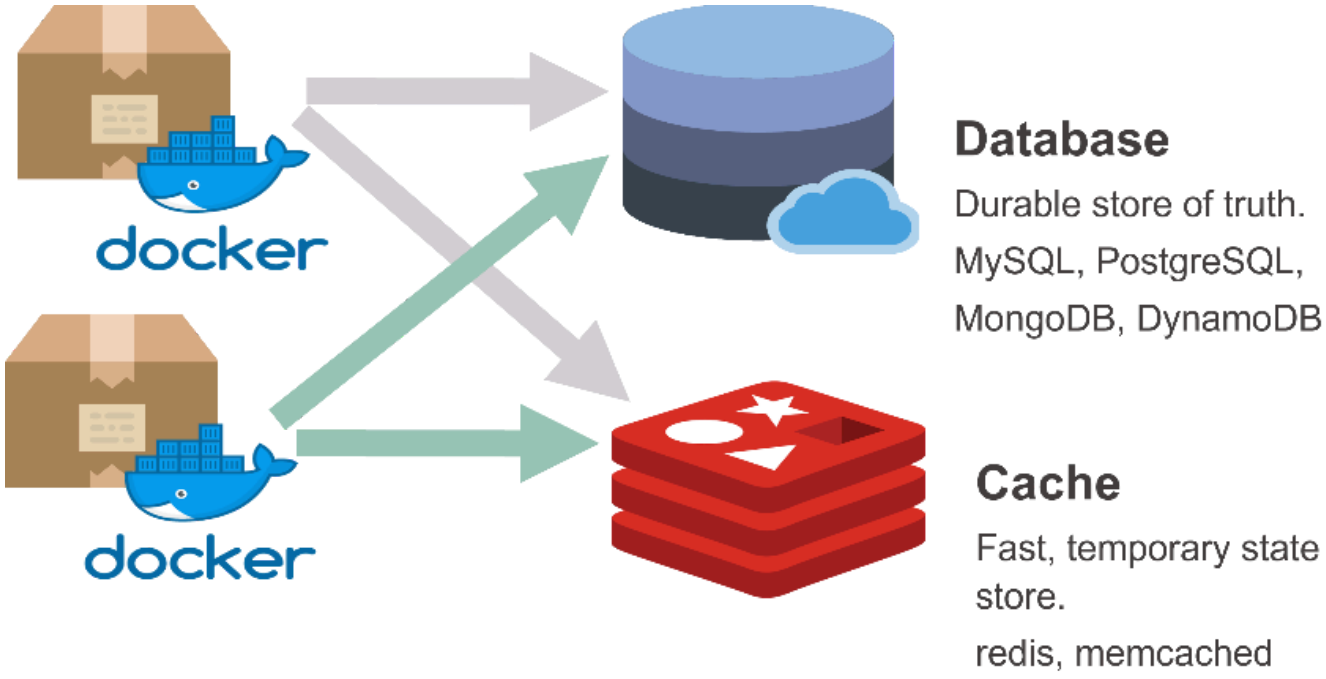
In this diagram, the application deployed in production consumes backing services as attached resources through URLs defined in configuration files.

Correlation to Factor 6: Processes

Let's continue with the analysis of the Twelve-Factor app in relation to backing services. Factor 6 is "Processes" and states that:

- Each microservice should execute in its own process, isolated from other running services.
- The require state should be externalized to a backing service, such as a distributed cache or data store.

Microservices create processes when they run, and the efficiency of those processes will determine the capacity of the overall application. If key processes are slow or bogged down, this can create a bottleneck, even with the use of microservices. To optimize the speed and efficiency, microservices will leverage localized temporary and volatile caches of data. The durable and long-term storage is provided by relational databases.



Twelve-Factor App

As you can see, the Twelve-Factor app methodology integrates well with the use of backing services, acting independently, consumed through the network, and defined through configuration files.

In the next section, we will discover some of the more common types of backing services.

Cloud-native systems depend upon many different ancillary resources such as data stores, message brokers, monitoring, and identity services. These services, known as backing services, will be explored during this lesson.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Define what is a backing service.

02 Relate how the twelve-factor application principles apply to backing services.

03 Recall the most common types of backing services.

Key Terms and Concepts

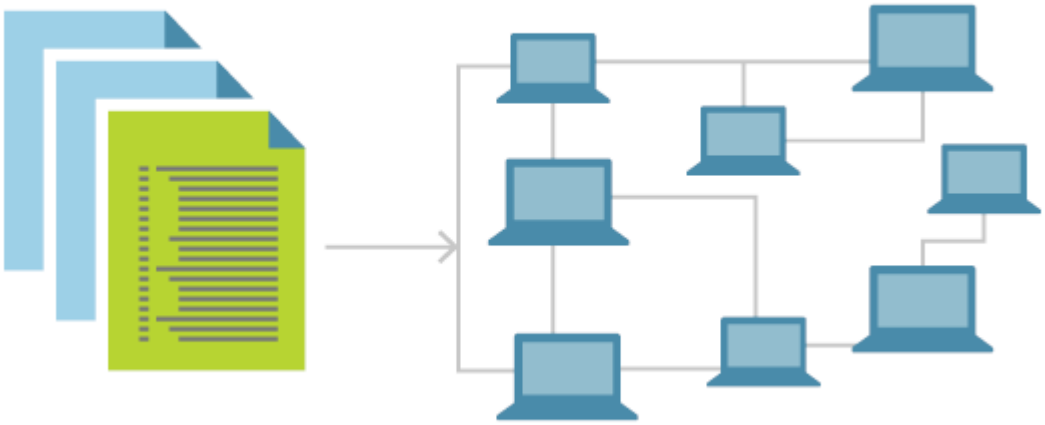
Key Term or Concept	Definition
Hard coding	Hard coding means that the information is set within the code of the application. For example, defining an access path to “C:\App_Data” directly in the code. This means that if the access path needs to be modified, a new version of the app with modified source code will be required.

Key Term or Concept	Definition
NoSQL	NoSQL stands for “Not-only SQL”. A NoSQL database provides a mechanism for storage and retrieval of data that is modeled in means other than the tabular relations used in relational databases. Such databases have existed since the late 1960s, but the name "NoSQL" was only coined in the early 21st century, triggered by the needs of Web 2.0 companies.
Structured Query Language	SQL stands for “Structured Query Language”. Structured Query Language is a domain-specific language used in programming and designed for managing data held in a relational database management system, or for stream processing in a relational data stream management system. It is particularly useful in handling structured data, i.e., data incorporating relations among entities and variables.

Automating Infrastructure

Infrastructure as code, or IaC, uses DevOps methodology and versioning with a descriptive model to define and deploy infrastructure, such as networks, virtual machines, load balancers, and connection topologies.

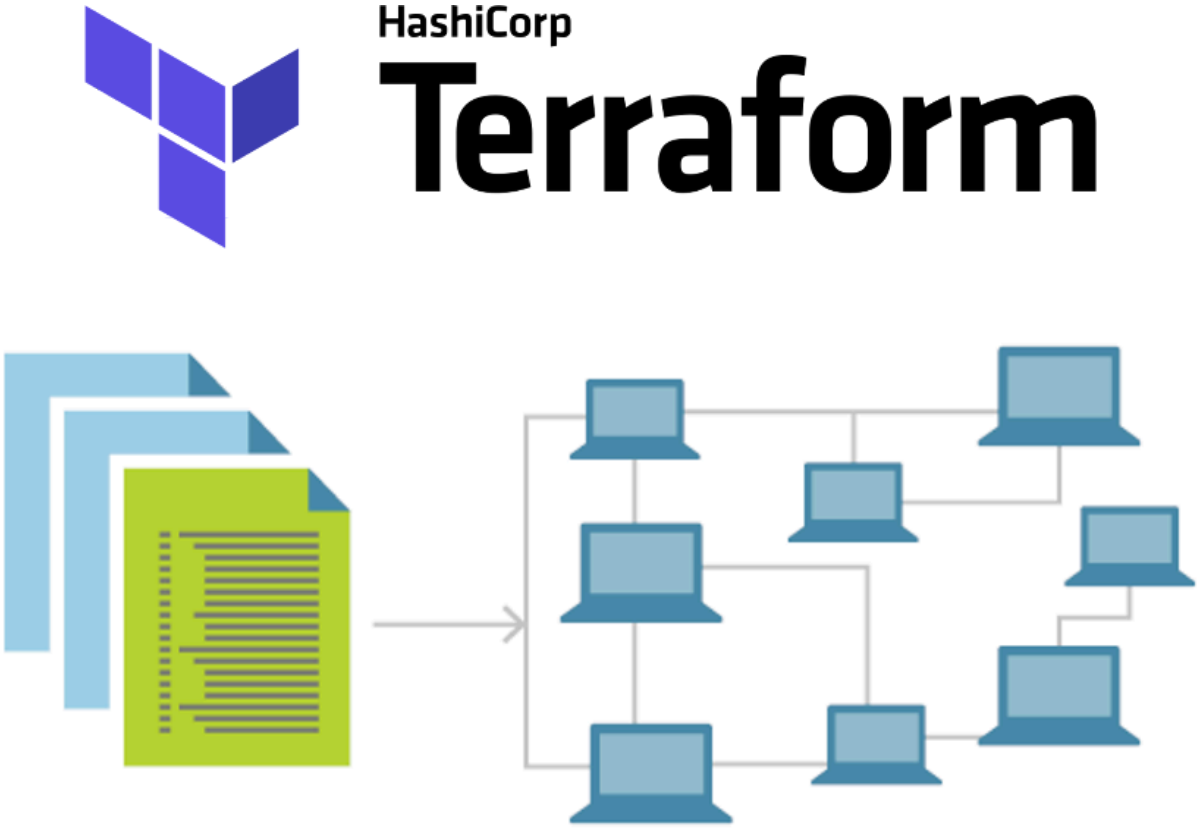
Just as the same source code always generates the same binary, an IaC model generates the same environment every time it deploys.



Infrastructure as code

Tools like Terraform from HashiCorp enable you to declaratively script the cloud infrastructure you require. Resource names, locations, capacities, and secrets are parameterized and dynamic. The script is versioned and checked into source control as an artifact of your project.

You invoke the script to provision a consistent and repeatable infrastructure across system environments, such as QA, staging, and production.



IaC and DevOps

Under the hood, IaC is idempotent, meaning that you can run the same script over and over without side effects. If the team needs to make a change, they edit and rerun the script, and only the updated resources are affected.

Teams who implement IaC can deliver stable environments rapidly and at scale. They avoid manual configuration of environments and enforce consistency by representing the desired state of their environments via code.

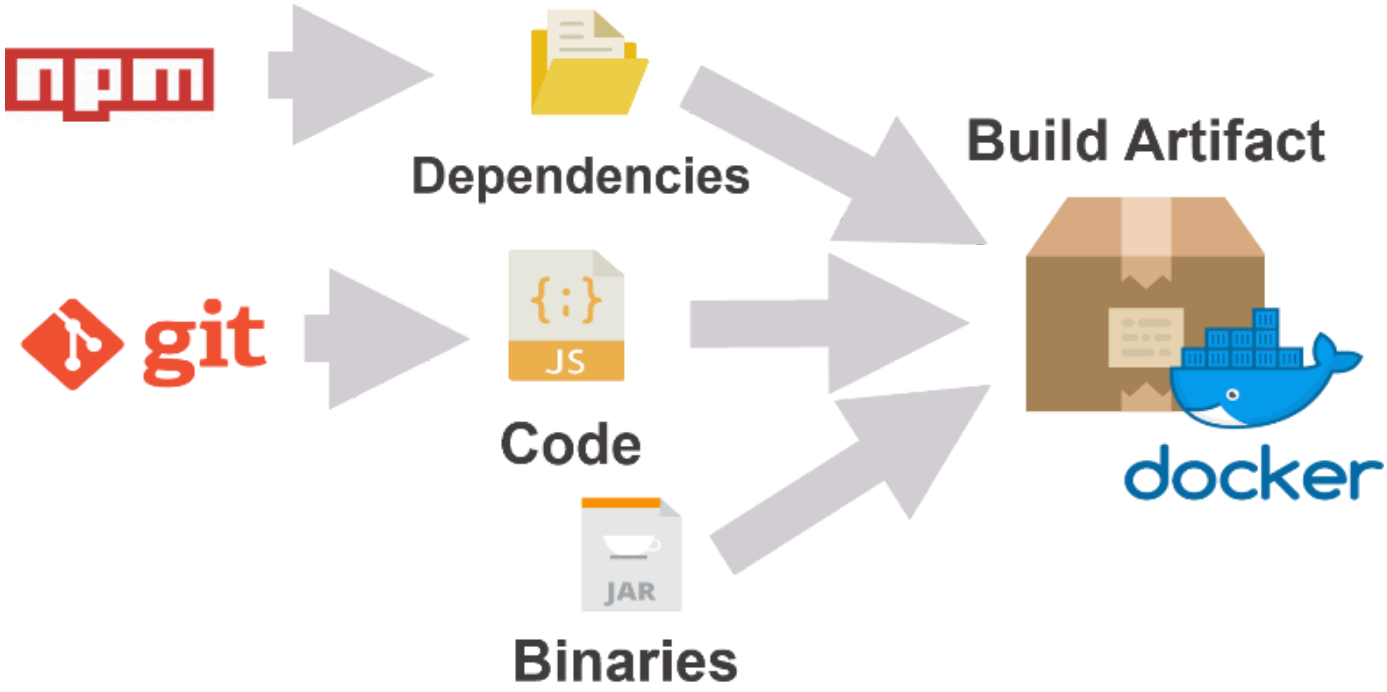
Infrastructure deployments with IaC are repeatable and prevent runtime issues caused by configuration drift or missing dependencies. DevOps teams can work together with a unified set of practices and tools to deliver applications and their supporting infrastructure rapidly, reliably, and at scale.

Twelve-Factor app: Factor 5

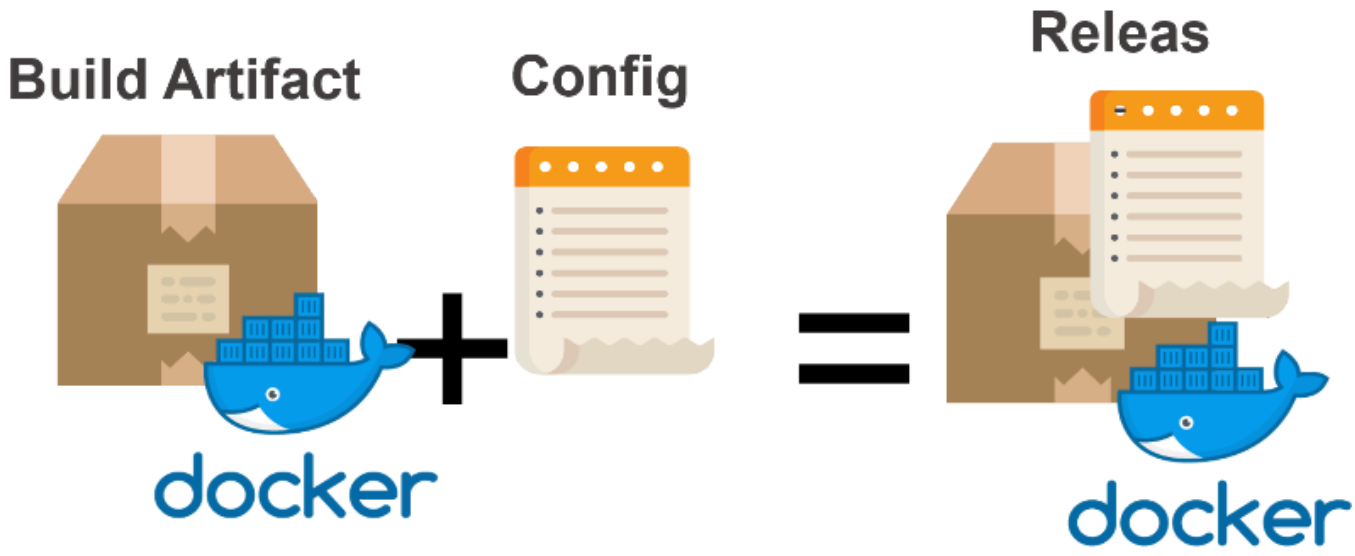
Let's revisit the Twelve-Factor app in light of new knowledge regarding automation of the CI/CD pipeline. You will recall that Factor 5 is: Build, Release, Run. This factor states that:

- Each release must enforce a strict separation across the build, release, and run stages.
- Each should be tagged with a unique ID and support the ability to roll back.

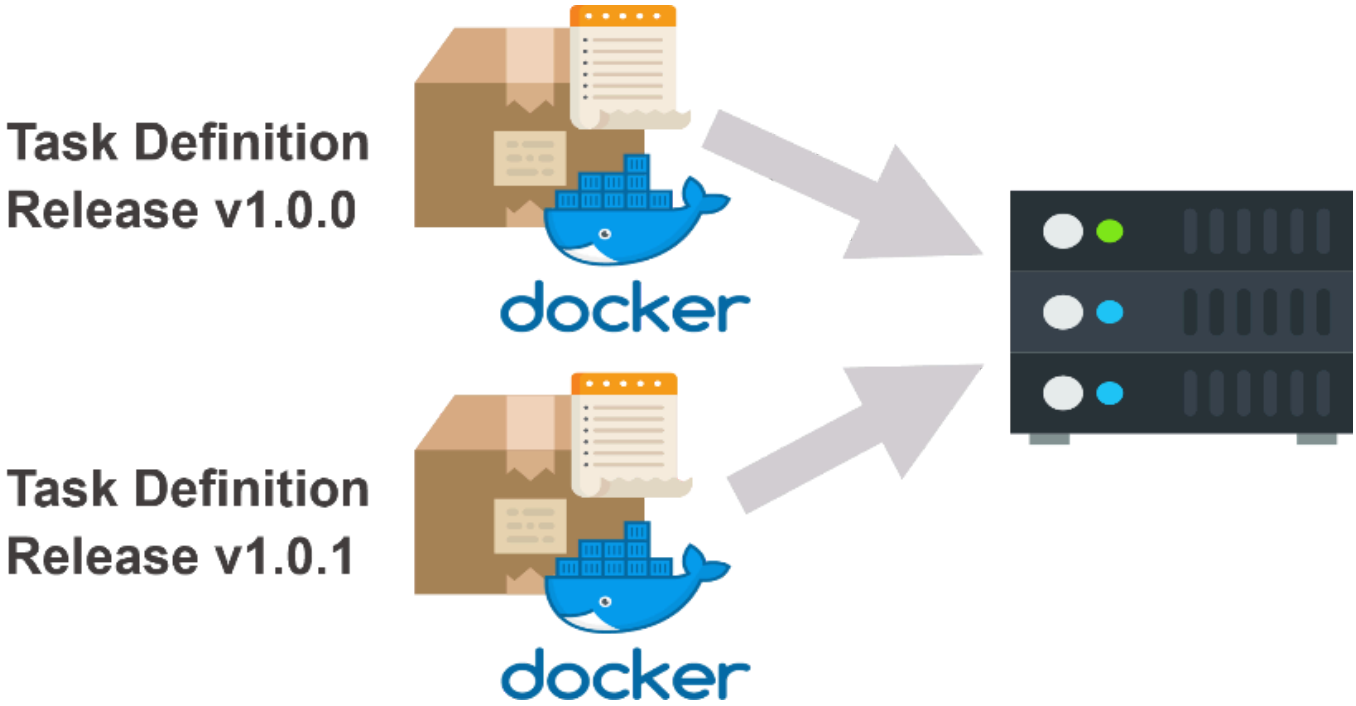
Modern CI/CD systems help fulfill this principle.



The development team, or multiple teams working on different aspects of the software, create and combine code, dependencies and binaries to build a containerized artifact.



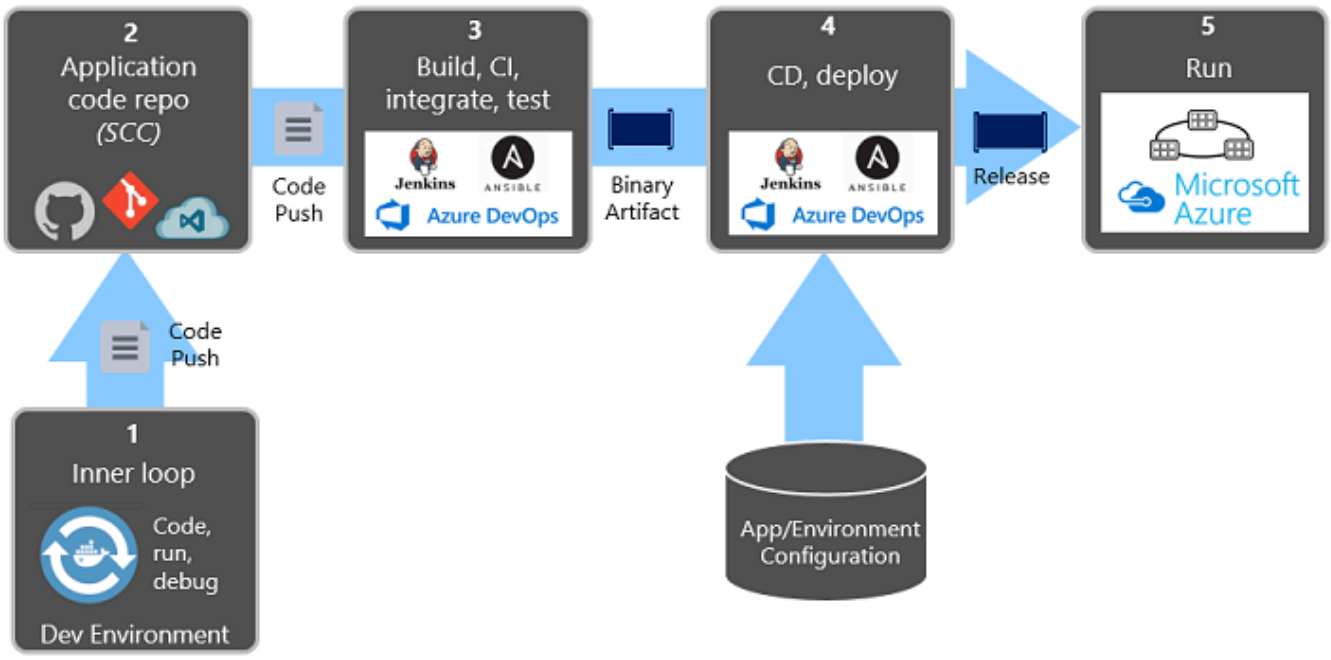
The artifact is combined with the environment configuration file required to deploy independently, building a release.



Subsequent releases are produced by the pipeline, identified by a version number and deployed to an environment. Monitoring ensures smooth deployment and rollbacks are initiated in case of issues.

Automating deployments

The Twelve-Factor Application, discussed earlier, calls for separate steps when transforming completed code into a running application. By design, modern CI/CD systems help fulfill this principle since they provide separate build and delivery steps that help ensure consistent and quality code that's readily available to users.



In the figure, pay special attention to separation of tasks:

1. The developer constructs a feature in their development environment, iterating through what is called the "inner loop" of code, run, and debug.

2. When complete, that code is “pushed” into a code repository, such as GitHub, Azure DevOps, or BitBucket.

3. The push triggers a build stage that transforms the code into a binary artifact. The work is implemented with a Continuous Integration (CI) pipeline. It automatically builds, tests, and packages the application.

4. The release stage picks up the binary artifact, applies external application and environment configuration information, and produces an immutable release. The release is deployed to a specified environment. The work is implemented with a Continuous Delivery (CD) pipeline. Each release should be identifiable. You can say, "This deployment is running Release 2.1.1 of the application."

5. Finally, the released feature is run in the target execution environment. Releases are immutable meaning that any change must create a new release.

Automation

By applying these practices, organizations have radically evolved how they ship software. Many have moved from quarterly releases to on-demand updates.

The goal is to catch problems early in the development cycle when they're less expensive to fix. The longer the duration between integrations, the more expensive problems become to resolve. With consistency in the integration process, teams can commit code changes more frequently, leading to better collaboration and software quality.

WHAT ARE THE BENEFITS OF CI/CD? (HTTPS://WWW.JETBRAINS.COM/TEAMCITY/CI-CD-GUIDE/BENEFITS-OF-CI-CD/)

CI/CD pipeline

We now have all the components to look at the full picture of cloud-native app deployment: the CI/CD pipeline. As we mentioned already, CI/CD stands for “Continuous Integration and Continuous Delivery/Deployment”. The concept is central to the DevOps software development lifecycle, which is based on Agile principles. To better understand the appeal of Agile in software development, we need to go back and look at previous software development lifecycle approaches.

Waterfall approach

The Waterfall model could be seen as a “theoretical” model, used to study the process of software development. However, it was and still is used as a model in some organizations, so to label it as theoretical is not accurate.

The waterfall model takes its name from the way that each step flows into the next. The model states that there are six phases to the creation of a software:

1. Requirements gathering
2. Analysis
3. Design
4. Coding
5. Testing
6. Deployment to operations

The model states that each step starts when the previous step is finished and verified, and that you cannot go back to any previous step to modify anything.

The benefits to the approach are rigour and control. Everything is determined and predictable. Timelines are strict, every step is fully documented and approved before moving on. The goal is to keep the project within timelines and budget, without surprises.

However, the reality of using the Waterfall model contrasts sharply with the expectations. It turns out that software creation does not yield itself well to a linear approach. Adjustments need to be made, requirements need to be edited, unexpected constraints will pop up, etc. Rigidly sticking to what was approved in earlier phases does not provide the flexibility required to adjust and fix the problems and issues arising during creation.

Major issues and failures in software development are frequent. Wikipedia hosts a list of software project that went over budger and/or failed completely.

LIST OF FAILED AND OVERBUDGET CUSTOM SOFTWARE PROJECTS

(HTTPS://EN.WIKIPEDIA.ORG/WIKI/LIST_OF_FAILED_AND_OVERBUDGET_CUSTOM_SOFTWARE_PROJECTS)

Evolution of software development lifecycles

On paper, the Waterfall approach makes sense, but it does not deliver good results when applied to real life projects. So, what does? Over the years, development lifecycles evolved, and many different approaches were used. Some examples of software development lifecycles include:

- **Rapid Application Development (RAD)**: This approach puts less emphasis on planning and instead start the creation early. This approach is popular with software that is driven by graphical user interfaces. The goal is to be able to quickly show the client what the software would “look like” and get quick feedback to adjust accordingly.

- **Iterative and incremental**: This approach shares similarities with RAD, where the creation is accelerated in order to have a prototype to show to the client quickly. Each cycle adds functionality to the software and is validated by the client. Adjustments can be made quickly so that the design does not stray from what the client wants.

- **Agile**: This is more of a paradigm shift than a specific lifecycle model. The notion of Agile software development is based on being able to introduce modification to software on a continual basis, without a long process to fix issues and bugs. As such, it borrows from the incremental approach and adds to the culture of software development by stating that “software is never complete, it always needs to be enhanced, tweaked, and maintained”. It removes the idea of a “finished” product that is meant to be in operations without maintenance and upkeep.

- **DevOps**: A model that uses the Agile paradigm and pushes towards a continuous cycle of development and operations, where the phases are not separate but managed by multi-skilled teams. DevOps is a natural fit for cloud-native apps and is rapidly becoming an established model for modern application design and development.

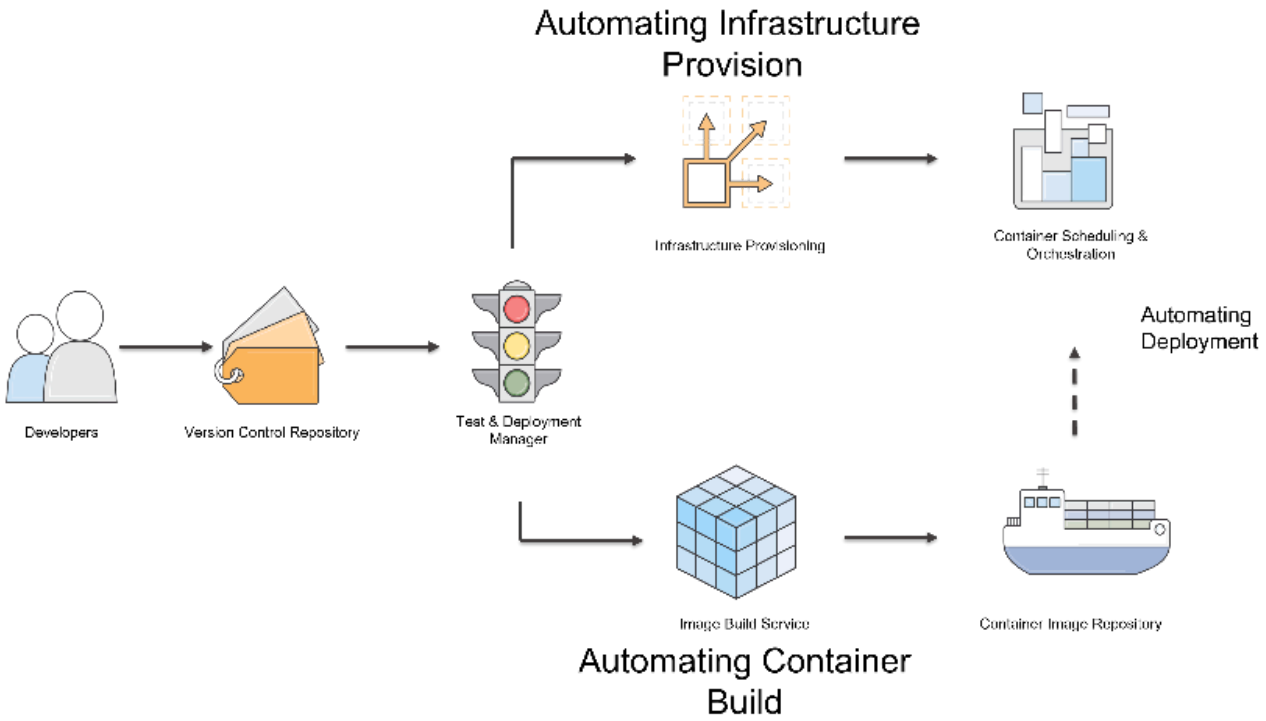


DevOps and CI/CD

DevOps has made possible the approach of multiple releases to a software. With microservices, different teams can work on different services of the application, and there can be a high number of releases every day. Bugs are quickly addressed, as are new requirements and enhancements.

This cycle of integration fixes, requirements, enhancements into the cycle of build, release, run is what creates the CI/CD pipeline.

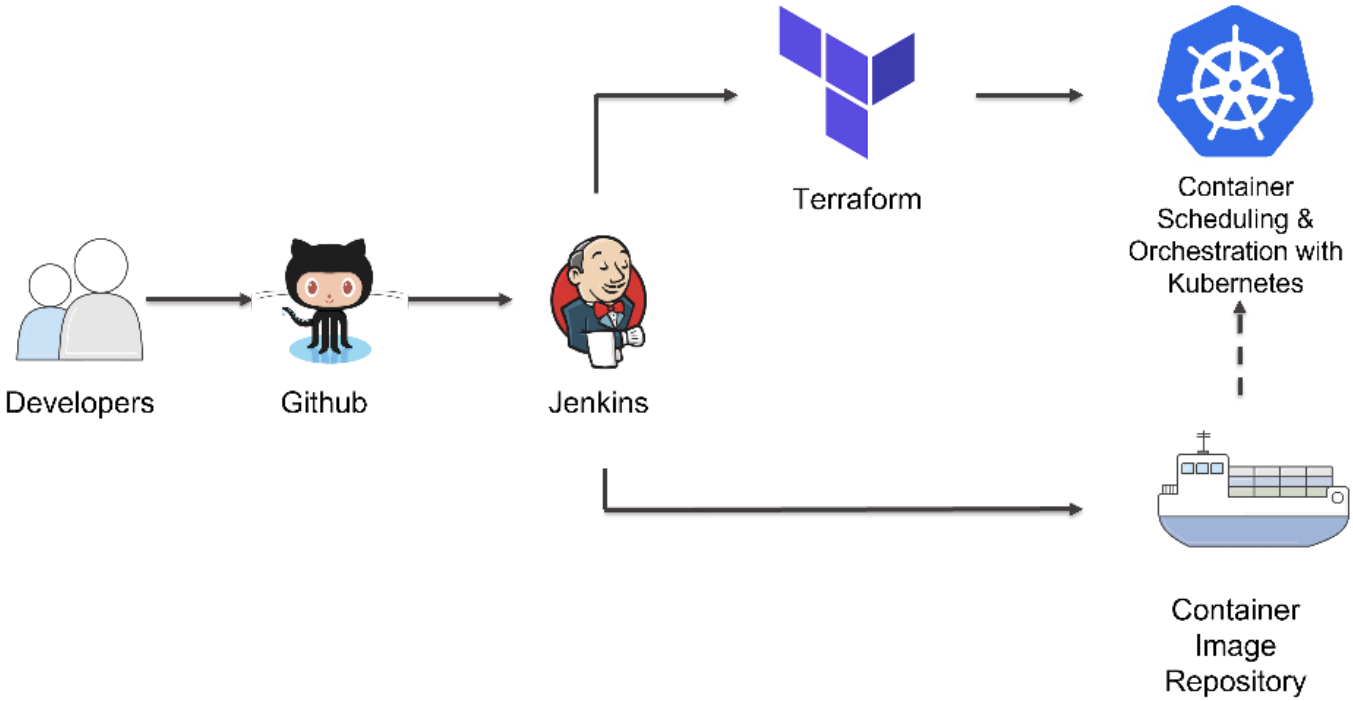
For cloud-native applications, this automation process can be extended to deploying servers and releasing containerized microservices to the environments.



Example of CI/CD Pipeline

Depending on other choices made by the software development team, certain tools will provide a more suitable choice than others when it comes to integrating into the environment.

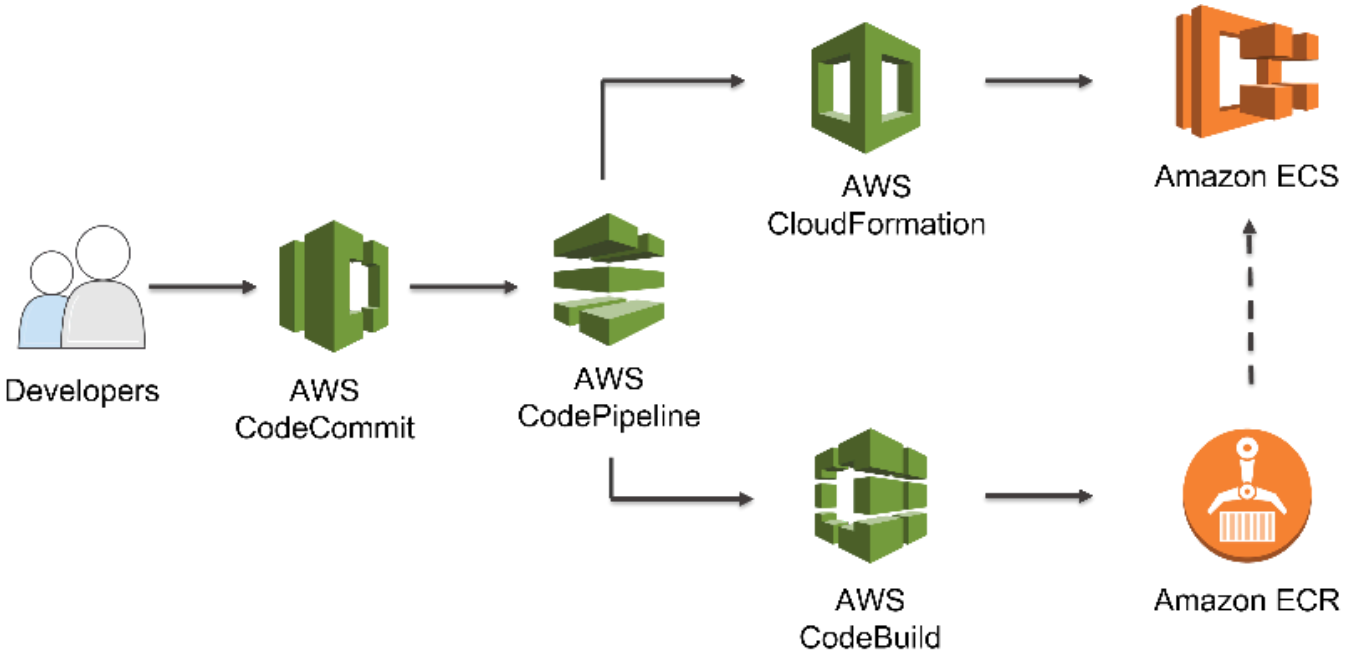
For example, a versatile set of tools that adapt to several different environment could be the following.



Leveraging the cloud provider tools

Certain teams will opt to use tools that are integrated into the cloud environment that will be hosting their microservices. This can offload some concerns and work to the cloud provider and generating a more scalable and elastic solution.

For example, AWS offers a suite of tools for most of the activities required in a complete CI/CD pipeline.



Automation tasks

To be in accordance with DevOps principles, software development, testing, and release should be as automated as possible, with full automation being the ultimate goal. To achieve this, you must:

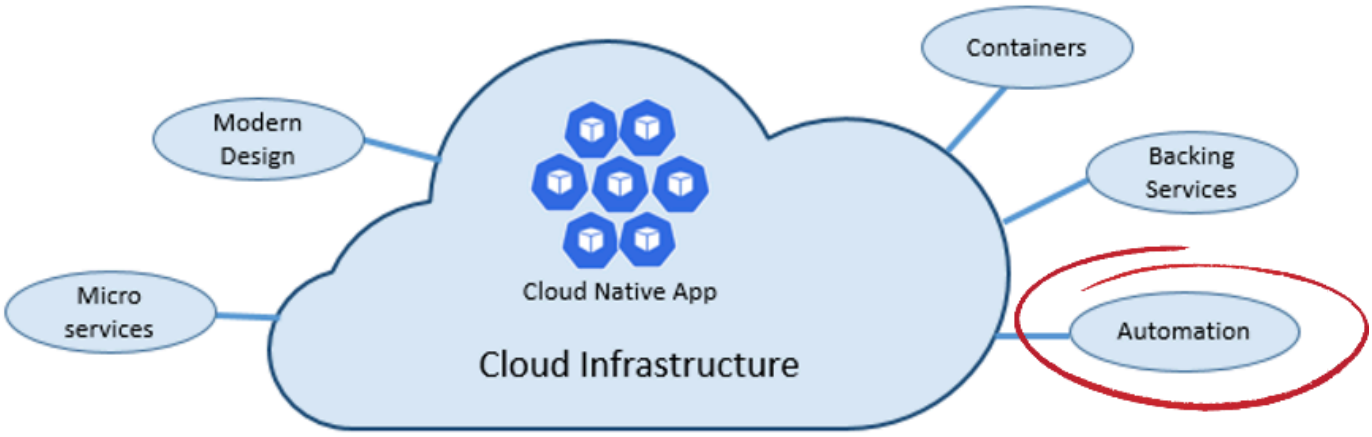
- Automate platform provisioning and application deployment.
- Apply software engineering practices such as testing and versioning to your DevOps practices.
- Ensure that your infrastructure and deployments are automated, consistent, and repeatable.

In the next section, we will examine the automation of the infrastructure, i.e., infrastructure as code.



Automation

We have reached the last pillar of cloud-native app development: automation. In a lot of ways, automation is a continuity of the other pillars that we have already studied. After all, automation plays a tremendous role in cloud computing technology, so it should not be surprising to see automation being a key component of cloud-native app development. As you have experimented with already, containerization is about automation. It relies on instructions embedded in configuration files to facilitate and accelerate the work of moving applications from one environment to another, with les risk and need for modifications.



In this lesson, we will go over one of the main components of DevOps: the continuous integration and continuous delivery and deployment pipeline, i.e., the CI/CD pipeline. The CI/CD pipeline can automate multiple aspects of the creation, testing, and deployment of software.

Automation in cloud-native apps

Cloud-native systems embrace microservices, containers, and modern system design to achieve speed and agility. But that's only part of the story. To achieve an agile pipeline through automation, you must determine:

- How do you provision the cloud environments upon which these systems run?
- How do you rapidly deploy app features and updates?
- How do you round out the full picture?

Automate the container build process

Automating the container build process involves creating a repeatable, reliable, and scalable process for building containers from your source code. Here are the steps you can follow to automate the container build process:

1. **Choose a Container Registry**: Choose a container registry where you will store your built containers. Popular options include Docker Hub, Google Container Registry, and Amazon Elastic Container Registry (ECR).

2. **Define your Build Configuration**: Define your build configuration, including the base image to use, build context, and any environment variables required by your application. You can define your build configuration using a Dockerfile, a YAML file, or a similar configuration format.

3. **Set up a Continuous Integration (CI) tool**: Choose a CI tool that will be responsible for building your containers. Popular CI tools include Jenkins, Travis CI, and AWS CodeBuild.

4. **Create a Build Pipeline**: Create a build pipeline in your CI tool that will automatically build and test your containers whenever changes are pushed to your source code repository. The build pipeline should include the following steps:

- Checkout the source code from the repository
- Build the container using the defined build configuration
- Push the built container to the container registry
- Run tests on the built container to ensure it is functional

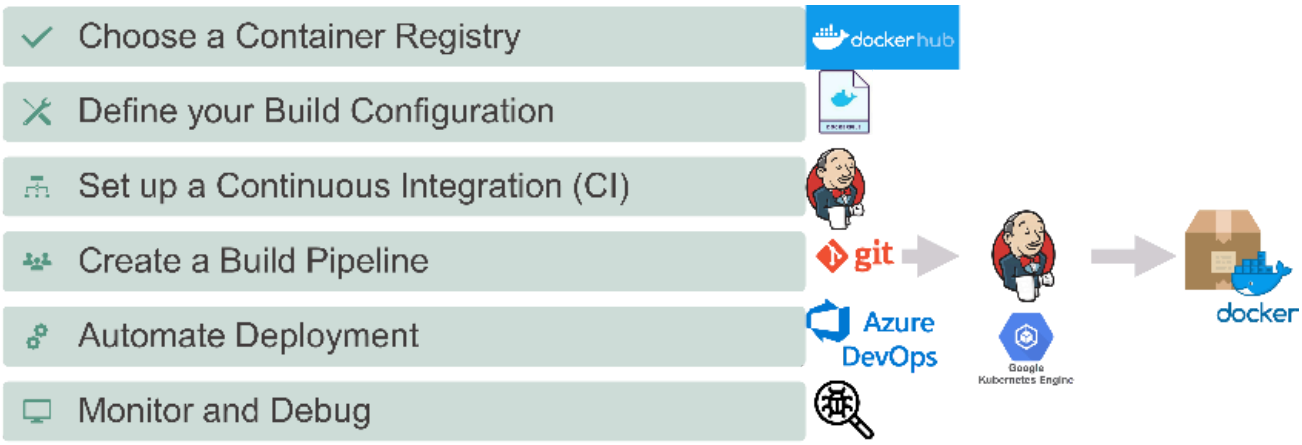
5. **Automate Deployment**: Automate the deployment of your containers to a production environment using a CD tool like AWS CodeDeploy, Google Kubernetes Engine, or Heroku.

6. **Monitor and Debug**: Set up monitoring and debugging tools to track the health and performance of your containers in production.



Automating a build pipeline can create a process that goes from the source code versioning repository to a built and deployed container.

By automating the container build process, you can ensure that your containers are built consistently and reliably every time. This helps you to quickly identify and resolve issues and enables you to release new features and bug fixes faster.

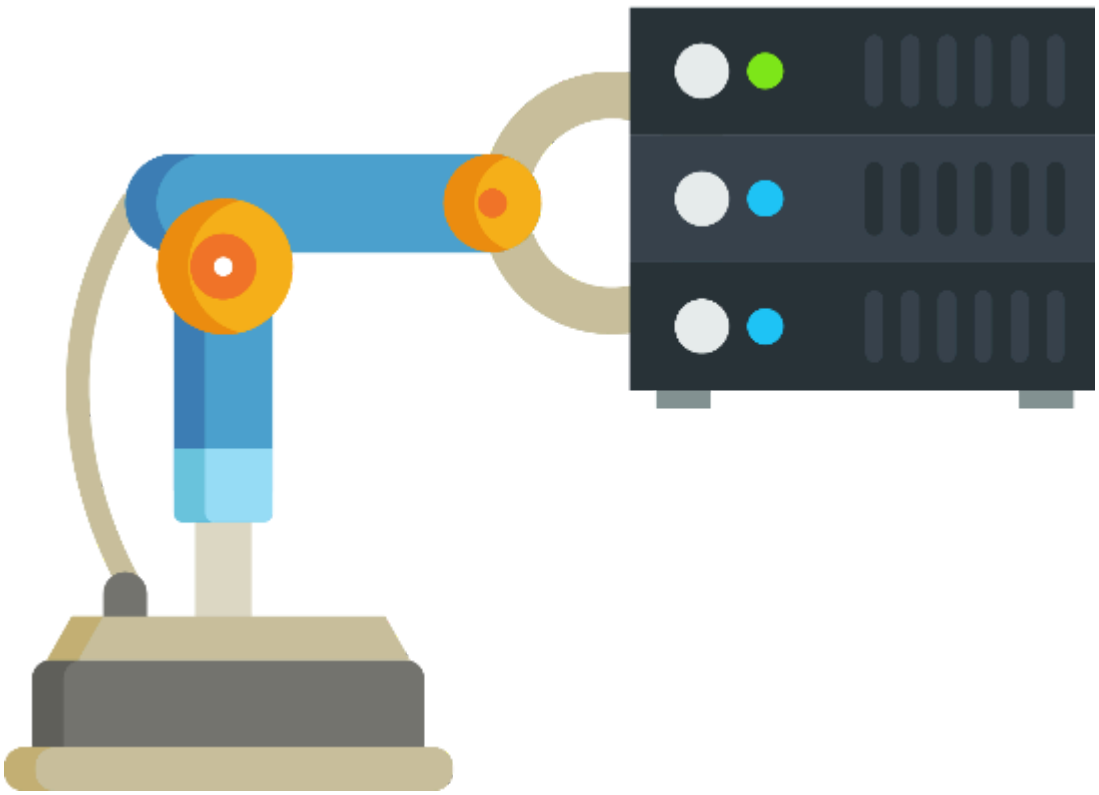


9

Sample selection of tools to create the entire CI/CD pipeline.

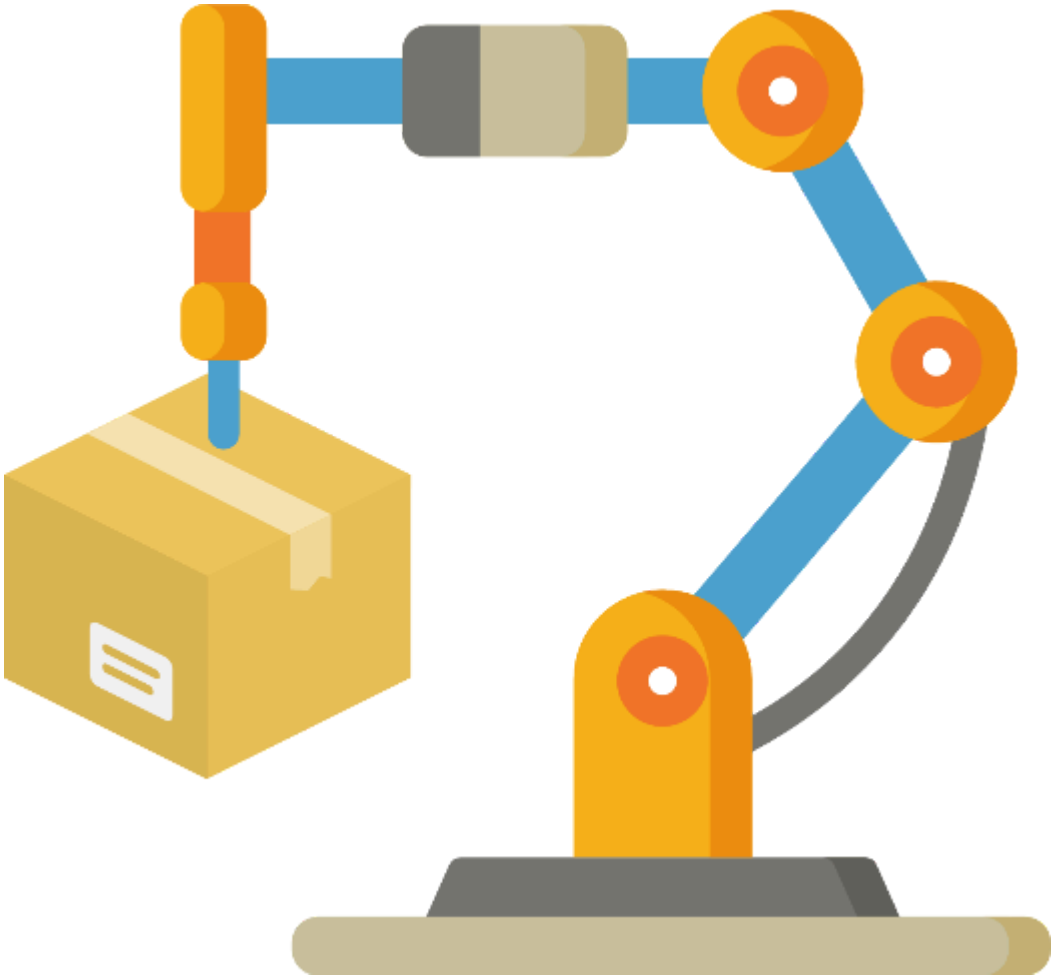
Infrastructure automation

Automation is not only for the code that creates the software: you can automate the provisioning of the servers that host microservice containers. This is known as: Infrastructure as code (IaC). Tools that provide this service include Terraform, Ansible, and Amazon CloudFormation.



Provisioning a server automatically using infrastructure as code.

With automated containers being created, and infrastructure as code provisioning servers to host the microservices, the last step remaining is to automate the placement of containerized service processes onto hosts. Tools that provide this service include Amazon Elastic Container Service, Kubernetes, and Docker Swarm.



Automatically deploying a containerized microservice on a provisioned server.

Cloud computing technology is defined by many characteristics which include on-demand provisioning, convenience, and being released with minimal management effort or service provider interaction. A key component to accomplish these goals is the heavy use of automation. In cloud-native application design and development, automation also plays a key role to reduce error, decouple the application from its environment, and speed up and facilitate the deployment of multiple releases of software versions to multiple different server instances. In this lesson, you will discover the role that automation plays in modern application design and development.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Recall the key concepts of automation.

02 Define the principles involved in the automation of software infrastructure.

03 Apply automation to a software delivery pipeline.